

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA VẬT LÝ – VẬT LÝ KỸ THUẬT



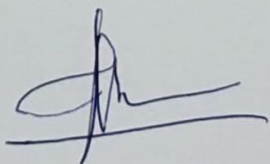
KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**THIẾT KẾ CHẾ TẠO QUADCOPTER
NHẬN DẠNG NGƯỜI BẰNG ẢNH NHIỆT**

SVTH: Vũ Thạch Thanh Huy

CBHD: ThS. Phạm Xuân Hiền



Phạm Xuân Hiền

TP HỒ CHÍ MINH – 2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA VẬT LÝ – VẬT LÝ KỸ THUẬT



KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**THIẾT KẾ CHẾ TẠO QUADCOPTER
NHẬN DẠNG NGƯỜI BẰNG ẢNH NHIỆT**

SVTH: Vũ Thạch Thanh Huy

CBHD: ThS. Phạm Xuân Hiền



TP HỒ CHÍ MINH – 2026

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp cử nhân ngành Vật lý học – chuyên ngành Vật lý điện tử, với đề tài: **“Thiết kế chế tạo quadcopter nhận dạng người bằng ảnh nhiệt”** là công trình nghiên cứu khoa học, sản phẩm do chính tôi thực hiện, dưới sự hướng dẫn của ThS. Phạm Xuân Hiên. Các kết quả nghiên cứu, số liệu và sản phẩm được trình bày trong khóa luận là trung thực, chính xác và chưa từng được sao chép từ bất kỳ nguồn nào khác. Tôi hoàn toàn chịu trách nhiệm về tính trung thực và nội dung của khóa luận này.

Sinh viên thực hiện

(Ký tên, ghi rõ họ tên)

Vũ Thạch Thanh Huy

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến Thầy Phạm Xuân Hiền. Thầy đã luôn dành thời gian quý báu để tận tình hướng dẫn, định hướng nghiên cứu và sát cánh hỗ trợ em trong suốt quá trình thực hiện đề tài khóa luận tốt nghiệp: "Thiết kế chế tạo quadcopter nhận dạng người bằng ảnh nhiệt". Những kiến thức chuyên môn sâu sắc và lời động viên của Thầy chính là nền tảng vững chắc giúp em vượt qua các thách thức kỹ thuật và hoàn thành tốt đề tài này.

Bên cạnh đó, em xin gửi lời tri ân sâu sắc đến quý Thầy Cô thuộc Bộ môn Vật lý Điện tử đã tận tâm giảng dạy, truyền đạt kiến thức và tạo môi trường học tập, nghiên cứu thuận lợi cho em trong suốt những năm tháng ngồi trên giảng đường đại học.

Cuối cùng, em xin dành lời cảm ơn chân thành đến nhóm các bạn sinh viên đồng hành cùng toàn thể gia đình, người thân. Mọi người đã luôn sát cánh, hỗ trợ, chia sẻ những khó khăn và là chỗ dựa tinh thần vô giá để em có thêm động lực hoàn thành tốt chặng đường học tập này.

Trong quá trình thực hiện đề tài, mặc dù đã có nhiều cố gắng nhưng chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý quý báu từ quý Thầy Cô để đề tài được hoàn thiện hơn.

Tp. Hồ Chí Minh, tháng 7 năm 2026

Vũ Thạch Thanh Huy

MỤC LỤC

LỜI CAM ĐOAN	2
LỜI CẢM ƠN.....	3
MỤC LỤC	4
DANH MỤC CÁC HÌNH ẢNH.....	7
DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT	8
LỜI MỞ ĐẦU	10
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	11
1.1 Đặt vấn đề.....	11
1.2 Tổng quan các nghiên cứu liên quan.....	11
1.2.1 Nghiên cứu trong nước	12
1.2.2 Nghiên cứu ngoài nước	13
1.2.3 Đánh giá chung và xác định khoảng trống nghiên cứu	16
CHƯƠNG 2: MỤC TIÊU ĐỀ TÀI VÀ YÊU CẦU KỸ THUẬT.....	18
2.1 Mục tiêu đề tài	18
2.1.1 Mục tiêu chung	18
2.1.2 Mục tiêu riêng.....	18
2.2 Đối tượng và phạm vi nghiên cứu	19
2.2.1 Đối tượng nghiên cứu	19
2.2.2 Phạm vi nghiên cứu	19
2.3 Yêu cầu kỹ thuật.....	19
CHƯƠNG 3: CƠ SỞ LÝ THUYẾT.....	21
3.1 Cơ sở vật lý của ảnh nhiệt và bức xạ hồng ngoại.....	21
3.1.1 Bản chất bức xạ nhiệt của vật đen tuyệt đối.....	21

3.1.2 Định luật dịch chuyển Wien và ứng dụng nhận diện người	21
3.1.3 Nguyên lý hoạt động của vi bức xạ kế và hệ màu trắng nóng.....	22
3.2 Mạng nơ-ron tích chập và sự trích xuất đặc trưng không gian.....	23
3.2.1 Phép toán tích chập ba chiều	23
3.2.2 Hàm kích hoạt phi tuyến.....	24
3.3 Kiến trúc lõi của mô hình RetinaNet.....	24
3.3.1 Mạng trích xuất đặc trưng.....	25
3.3.2 Mạng tháp đặc trưng FPN	26
3.3.3 Cơ sở toán học và cơ chế hoạt động của Anchor Box	27
3.3.4 Mạng con dự đoán.....	30
3.3.5 Nhánh mạng dự đoán và các hàm mất mát	31
3.3.6 Lý thuyết tinh chỉnh cho ảnh nhiệt	33
4.1 Sơ đồ tổng quan mô hình nhận dạng	35
4.2 Hệ thống phần cứng.....	36
4.2.1 Máy tính nhúng Jetson Orin Nano Developer Kit.....	36
4.2.2 Camera SIYI ZT6	37
4.2.3 Tay cầm điều khiển SIYI UniRC 7 Pro	38
4.2.4 Sơ đồ kết nối phần cứng	39
4.2.5 Thu thập dữ liệu.....	41
4.2.6 Tiền xử lý dữ liệu	41
4.3 Xây dựng và tối ưu thuật toán nhận dạng.....	42
4.3.1 Nền tảng phần cứng để huấn luyện mô hình	42
4.3.2 Thư viện TensorFlow và Keras	42
4.3.3 Xây dựng mô hình toán học nhận dạng.....	46
4.3.4 Thiết lập tham số và quá trình huấn luyện	48

4.3.5 Đánh giá kết quả huấn luyện mô hình.....	50
4.4 Chuyển đổi và triển khai mô hình trên máy tính nhúng.....	51
4.4.1 Xuất và chuyển đổi định dạng mô hình	51
4.4.2 Triển khai mô hình trên máy tính nhúng	52
CHƯƠNG 5: KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN	55
5.1 Kết quả lắp ráp hoàn chỉnh.....	55
5.2 Kết quả kiểm thử mô hình trên tập dữ liệu.....	56
5.2.1 Chi tiết bộ dữ liệu kiểm thử	56
5.2.2 Kết quả các độ đo hiệu năng	56
5.2.3 Trực quan hóa kết quả.....	58
5.3 Kết quả triển khai hệ thống thực tế.....	61
5.4 Thảo luận và đánh giá hạn chế	63
5.5 Kết luận chung.....	64
5.6 Hướng phát triển.....	65
TÀI LIỆU THAM KHẢO	66
PHỤ LỤC	69

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1 - Mạng kết hợp đặc trưng ảnh nhiệt và ảnh màu	12
Hình 1.2 - Các tư thế xâm nhập từ bộ dữ liệu	13
Hình 1.3 - Sơ đồ hệ thống sử dụng cho thuật toán Anchor Free	14
Hình 1.4 - Sơ đồ khối kiến trúc theo vết kết hợp bù trừ chuyển động	15
Hình 1.5 - Đồ thị so sánh thời gian suy luận và tiêu thụ năng lượng	16
Hình 3.1 - Ảnh nhiệt trắng nóng.....	23
Hình 3.2 - Một Residual Block.....	26
Hình 3.3 - Tọa độ của Anchor Free	29
Hình 3.4 - So sánh FL và CE.....	32
Hình 4.1 - Sơ đồ tổng quát quy trình hoạt động của thuật toán	35
Hình 4.2 - Máy tính nhúng Jetson Orin Nano	37
Hình 4.3 – Camera SIYI ZT6	38
Hình 4.4 – Tay cầm điều khiển SIYI UniRC 7 Pro.....	39
Hình 4.5 – Sơ đồ kết nối hệ thống	40
Hình 4.6 – Sơ đồ tổng quát kiến trúc mạng nhận dạng người bằng ảnh	47
Hình 4.7 - Biểu đồ hàm loss	51
Hình 4.8 - Sơ đồ khối thực thi trên máy tính nhúng.....	53
Hình 5.1 – Kết quả mô hình luận văn.....	55
Hình 5.2 - Chỉ số mAP@0.5.....	57
Hình 5.3 - Confusion matrix.....	58
Hình 5.4 - Kết quả nhận diện khi bay thấp.....	59
Hình 5.5 - Kết quả nhận dạng đối tượng ở xa	60
Hình 5.6 - Mô hình dự đoán với nền	61
Hình 5.7 - Thông số thực tế khi hoạt động trên máy tính nhúng	62

DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT

AI: trí tuệ nhân tạo.

IoU: giao điểm trên hợp nhất.

DIoU: khoảng cách giao điểm trên hợp nhất.

CNN: mạng nơ-ron tích chập.

FPN: mạng đặc trưng kim tự tháp.

CUDA: Kiến trúc thiết bị tính toán hợp nhất (Nền tảng tính toán song song do NVIDIA phát triển).

cuDNN: Thư viện mạng nơ-ron sâu trên nền tảng CUDA.

DIoU: Độ giao thoa trên hợp nhất dựa trên khoảng cách (Hàm mất mát tối ưu hóa tọa độ dựa trên khoảng cách).

FN: Dự đoán sai mẫu tiêu cực (Trường hợp bỏ sót đối tượng).

FP: Dự đoán sai mẫu tích cực (Trường hợp nhận diện nhầm / báo động giả).

FP16: Dấu phẩy động 16-bit (Định dạng dữ liệu bán độ chính xác).

GPU: Bộ xử lý đồ họa.

mAP: Độ chính xác trung bình.

NMS: Thuật toán triệt tiêu phi cực đại.

ONNX: Định dạng trao đổi mạng nơ-ron mở.

ReLU: Hàm kích hoạt chỉnh lưu tuyến tính.

ResNet: Mạng nơ-ron thặng dư.

SBUS: Giao thức truyền tin kỹ thuật số nối tiếp.

TensorRT: Công cụ tối ưu hóa đồ thị tính toán và tăng tốc suy luận mạng nơ-ron của NVIDIA.

TOPS: Nghìn tỷ phép tính mỗi giây (Đơn vị đo hiệu năng tính toán của thiết bị).

TP: Dự đoán đúng mẫu tích cực (Trường hợp nhận diện đúng đối tượng).

VRAM: Bộ nhớ truy cập ngẫu nhiên video (Bộ nhớ đồ họa).

LỜI MỞ ĐẦU

Đề tài khóa luận “Thiết kế chế tạo quadcopter nhận dạng người bằng ảnh nhiệt” là một dự án nghiên cứu phối hợp được chia thành hai mảng chuyên biệt. Trong đó, mảng thiết kế và chế tạo phần cứng quadcopter được thực hiện bởi sinh viên Lê Quốc Nguyên (MSSV: 22130122, lớp Cử nhân Tài năng Vật lý học, khóa 2022). Nội dung luận văn này tập trung vào phần thứ hai của tên đề tài, đó là nghiên cứu xây dựng thuật toán nhận dạng ảnh nhiệt cho quadcopter.

Mục tiêu trọng tâm của đề tài là phát triển hệ thống hỗ trợ nhận dạng người qua dữ liệu ảnh nhiệt thu thập từ quadcopter, ưu tiên đảm bảo độ chính xác cao khi vận hành trong các điều kiện môi trường thực tế phức tạp.

Chương 1 Tổng quan đề tài – Phân tích hạn chế của phương pháp tìm kiếm truyền thống, nêu bật vai trò của AI kết hợp quadcopter, tổng quan các nghiên cứu liên quan và xác định khoảng trống công nghệ.

Chương 2 Mục tiêu và yêu cầu kỹ thuật – Xác định mục tiêu xây dựng thuật toán nhận dạng người. Thiết lập yêu cầu về môi trường phần mềm (TensorFlow/Keras) và quy trình tối ưu hóa, chuyển đổi mô hình để triển khai trên thiết bị nhúng.

Chương 3 Cơ sở lý thuyết – Trình bày nền tảng vật lý ảnh nhiệt, nguyên lý cảm biến, lý thuyết mạng CNN, kiến trúc RetinaNet, Anchor Free và các hàm mất mát.

Chương 4 Xây dựng mô hình toán nhận dạng – Mô tả hệ thống phần cứng (Jetson Orin Nano, Siyi ZT6 và tay cầm SIYI UniRC 7 pro), quy trình thu thập dữ liệu, huấn luyện mô hình và các bước tối ưu hóa để triển khai thực thi độc lập.

Chương 5 Kết quả và hướng phát triển - Đánh giá hiệu năng mô hình thông qua tập dữ liệu kiểm thử với các chỉ số như mAP@0.5 và Confusion Matrix. Trình bày kết quả triển khai thực tế trên máy tính nhúng, thảo luận về các hạn chế và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh trí tuệ nhân tạo phát triển mạnh mẽ những năm gần đây, nhiều mô hình có khả năng xử lý thời gian thực đã được triển khai hiệu quả trong các lĩnh vực như y tế, giáo dục, xây dựng, sản xuất và giao thông. Bên cạnh đó, công tác tìm kiếm, cứu nạn và cứu hộ cũng là một lĩnh vực trọng điểm cần được đẩy mạnh ứng dụng các mô hình học sâu. Tuy nhiên, các phương pháp tìm kiếm truyền thống bằng mắt thường hoặc camera quang học thường gặp nhiều trở ngại do điều kiện ánh sáng yếu, sương mù dày đặc hoặc địa hình chia cắt phức tạp. Đặc biệt, việc phân biệt nạn nhân giữa môi trường có màu sắc tương đồng hoặc khi đối tượng bị che khuất bởi tán cây là một thách thức lớn, dễ dẫn đến sai sót do sự mệt mỏi và giới hạn về khả năng quan sát của con người.

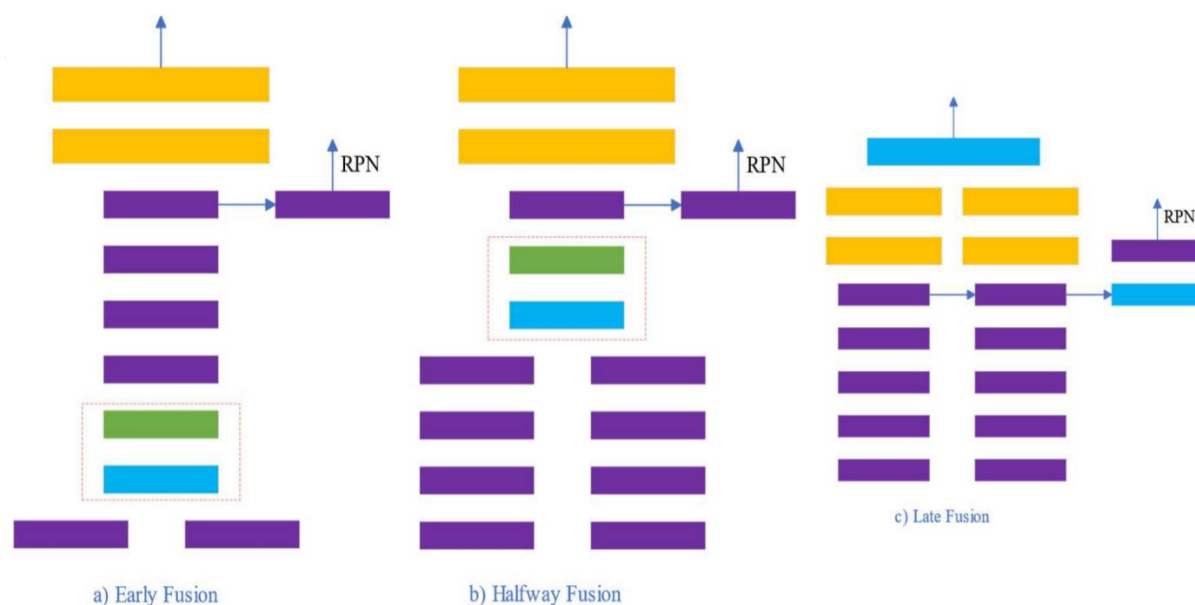
Nhằm khắc phục những hạn chế đó, việc ứng dụng mô hình AI kết hợp với quadcopter là điều hết sức cần thiết. Hệ thống này đóng vai trò như một trợ lý thông minh, giúp người điều khiển xác định nạn nhân thông qua đặc điểm bức xạ nhiệt, từ đó tối ưu hóa khả năng phát hiện và giảm thiểu rủi ro bỏ sót đối tượng. Do đó, đề tài tốt nghiệp tập trung vào bài toán xây dựng hệ thống nhận dạng người qua ảnh nhiệt từ quadcopter. Việc giải quyết bài toán này không chỉ mang giá trị về mặt nghiên cứu, mà còn đem lại lợi ích thiết thực trong công tác cứu hộ, cứu nạn nhằm đảm bảo an toàn tối đa cho cộng đồng.

1.2 Tổng quan các nghiên cứu liên quan

Sự phát triển vượt bậc của công nghệ thị giác máy tính và hệ thống tự hành đã thúc đẩy việc ứng dụng máy bay không người lái kết hợp với cảm biến hồng ngoại trong nhiều lĩnh vực quan trọng. Việc nghiên cứu tổng quan các giải pháp hiện tại cả trong và ngoài nước đóng vai trò then chốt giúp định hình rõ ràng ranh giới công nghệ, từ đó xác định khoảng trống nghiên cứu và thiết lập phương án tối ưu hóa kiến trúc thuật toán nhận dạng nhiệt tại biên.

1.2.1 Nghiên cứu trong nước

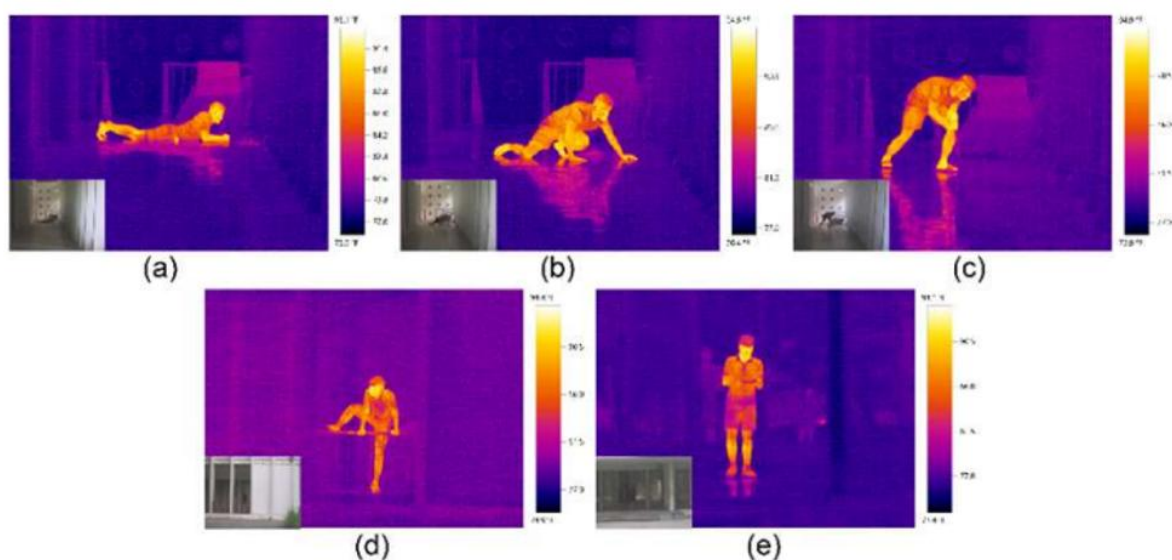
Tại Việt Nam, việc ứng dụng ảnh nhiệt vào các hệ thống an ninh và thiết bị tự hành đang nhận được sự quan tâm lớn từ các cơ sở nghiên cứu và đào tạo. Một trong những công trình nghiên cứu sâu rộng nhất về việc kết hợp đặc trưng đa nguồn ảnh có thể kể đến là luận án tiến sĩ về nghiên cứu giải pháp kết hợp ảnh nhiệt và ảnh màu trong bài toán phát hiện và theo vết đối tượng người của tác giả Đào Vũ Hiệp [1]. Chức năng cốt lõi của công trình này là thiết kế một kiến trúc mạng nơ-ron sâu có khả năng dung hợp các đặc trưng từ hai luồng ảnh màu và ảnh nhiệt, đồng thời đề xuất phương pháp xác định trọng số kết hợp tự động dựa trên phân tích độ nhiễu và cường độ nhiệt. Phạm vi ứng dụng chính của công trình là các hệ thống giám sát an ninh hoạt động xuyên suốt ngày và đêm, nơi có sự biến động lớn về ánh sáng. Ưu điểm nổi bật của giải pháp này là khả năng cải thiện đáng kể chỉ số tỷ lệ bỏ sót đối tượng so với các thuật toán đơn phổ thông thường, đặc biệt trong các điều kiện môi trường có độ che khuất cao. Tuy nhiên, nhược điểm kỹ thuật của hướng tiếp cận này nằm ở cấu trúc mạng nơ-ron tích chập chồng kênh, đòi hỏi năng lực tính toán cực lớn để duy trì song song hai luồng dữ liệu,



Hình 1.1 - Mạng kết hợp đặc trưng ảnh nhiệt và ảnh màu [1]

Một bài toán khác mang tính thực tiễn cao trong nước là sự thiếu hụt dữ liệu huấn luyện đặc thù cho ảnh nhiệt, vốn là rào cản lớn đối với việc huấn luyện các mô hình học sâu. Công trình nghiên cứu về bộ dữ liệu ảnh nhiệt phục vụ phát hiện người trong các hệ thống cảnh báo xâm nhập của nhóm tác giả Nguyễn Đức Thuận và cộng sự [2] đã mang

lại bước tiến quan trọng. Chức năng chính của nghiên cứu này là đề xuất phương pháp tổng hợp dữ liệu ảnh nhiệt nhân tạo bằng kỹ thuật chỉnh sửa ảnh Poisson để ghép nối các đối tượng người xâm nhập vào các bối cảnh nền khác nhau, tạo ra bộ dữ liệu gồm hàng ngàn bức ảnh nhiệt. Phạm vi ứng dụng của nghiên cứu hướng trực tiếp đến các hệ thống cảnh báo xâm nhập an ninh thông minh, nơi đối tượng xuất hiện với năm tư thế đặc thù bao gồm trườn, bò, khom lưng, leo trèo và đi bộ thông thường. Ưu điểm lớn nhất của công trình là khả năng giải quyết bài toán thiếu hụt hình ảnh huấn luyện đối với các tư thế xâm nhập bất thường, giúp các thuật toán phát hiện đối tượng phổ biến đạt được độ chính xác rất cao. Tuy nhiên, nhược điểm của phương pháp này nằm ở chỗ dữ liệu tổng hợp bằng nội suy đôi khi tạo ra những sai lệch vật lý ở vùng ranh giới ghép nối, khiến mô hình học sâu có nguy cơ học phải các đặc trưng nhiễu nhân tạo thay vì đặc trưng thật của đối tượng.

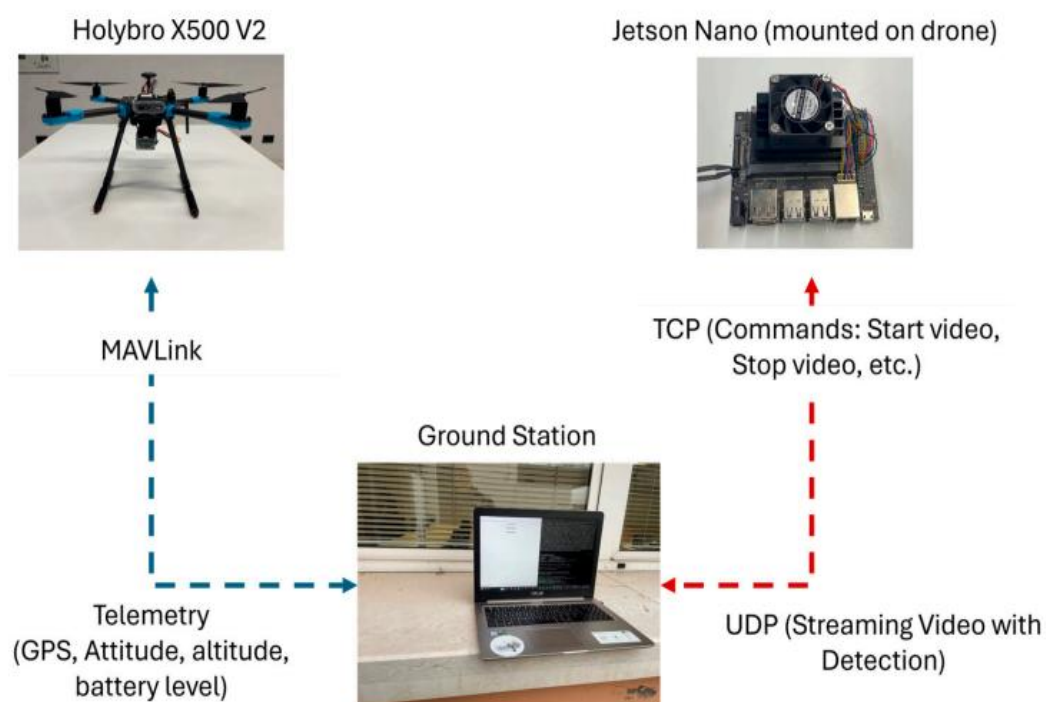


Hình 1.2 - Các tư thế xâm nhập từ bộ dữ liệu [2]

1.2.2 Nghiên cứu ngoài nước

Trên bình diện quốc tế, từ năm 2024 đến nay, các công bố khoa học tập trung mạnh mẽ vào việc tối ưu hóa cấu trúc mạng nơ-ron để giải quyết bài toán suy luận thời gian thực trên các thiết bị biên, đặc biệt là thiết bị bay không người lái. Nghiên cứu về mạng phát hiện đối tượng Anchor Free siêu nhẹ ứng dụng trong hình ảnh nhiệt từ thiết bị bay công bố trên tạp chí MDPI Drones đại diện cho hướng đi cắt giảm tham số mô hình [3]. Chức năng cốt lõi của nghiên cứu này là loại bỏ hoàn toàn cơ chế tạo hàng ngàn các Anchor box truyền thống vốn gây tiêu tốn bộ nhớ, thay vào đó sử dụng phương pháp dự đoán

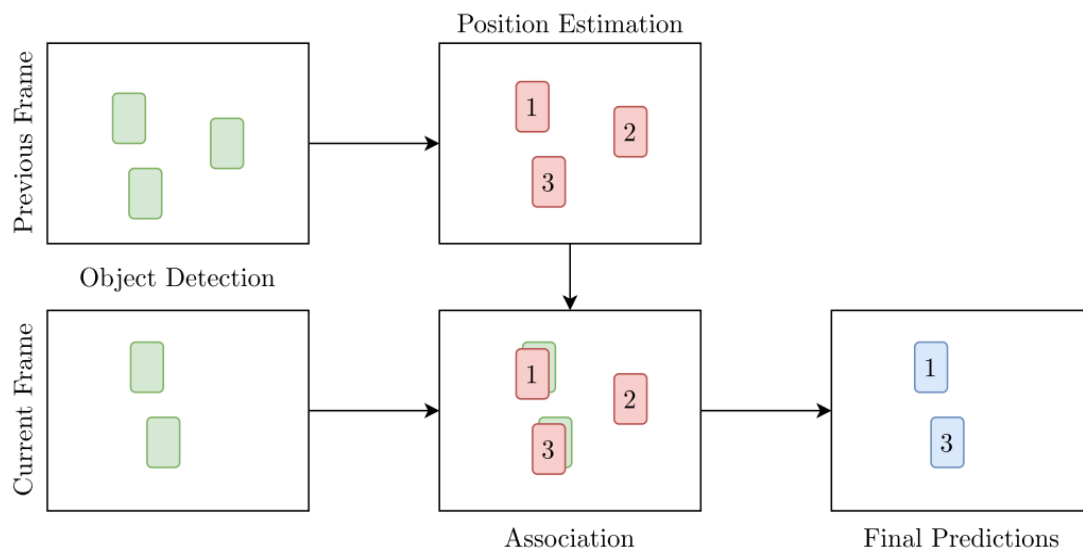
trực tiếp điểm trung tâm của đối tượng thông qua một mạng trích xuất đặc trưng đã được thu gọn các tầng tích chập sâu. Phạm vi ứng dụng của công trình hướng đến các thiết bị bay trinh sát cỡ nhỏ thực hiện nhiệm vụ giám sát giao thông và an ninh khu vực vào ban đêm. Ưu điểm của giải pháp này là giảm thiểu đến mức tối đa các phép toán dư thừa, giúp thuật toán đạt tốc độ xử lý hàng chục khung hình mỗi giây ngay cả trên các bo mạch tính toán giới hạn. Nhược điểm của thiết kế này là khi cắt giảm quá sâu các tầng thặng dư, mô hình dễ bị mất mát các đặc trưng ngữ nghĩa phức tạp, dẫn đến hiện tượng nhận diện nhầm các nguồn nhiệt nhỏ của môi trường thành đối tượng mục tiêu khi bay ở độ cao lớn.



Hình 1.3 - Sơ đồ hệ thống sử dụng cho thuật toán Anchor Free [3]

Trong lĩnh vực ứng dụng cứu hộ khẩn cấp, công trình nghiên cứu về hệ thống tìm kiếm nạn nhân ban đêm sử dụng thị giác máy tính ảnh nhiệt công bố trên tạp chí MDPI Remote Sensing đã mang lại nhiều cải tiến về mặt thuật toán theo vết [4]. Chức năng của nghiên cứu là tích hợp một mô-đun bù trừ chuyển động dựa trên tư thế máy bay vào một kiến trúc theo vết đối tượng một giai đoạn nhằm ép buộc mô hình phải tập trung học hỏi các vết nhiệt mờ nhạt nhất của con người bất chấp sự rung lắc của thiết bị bay. Phạm vi ứng dụng tập trung chuyên sâu vào công tác tìm kiếm cứu nạn tại các khu vực rừng núi, nơi thân nhiệt nạn nhân thường bị suy giảm và bị che khuất một phần bởi tán

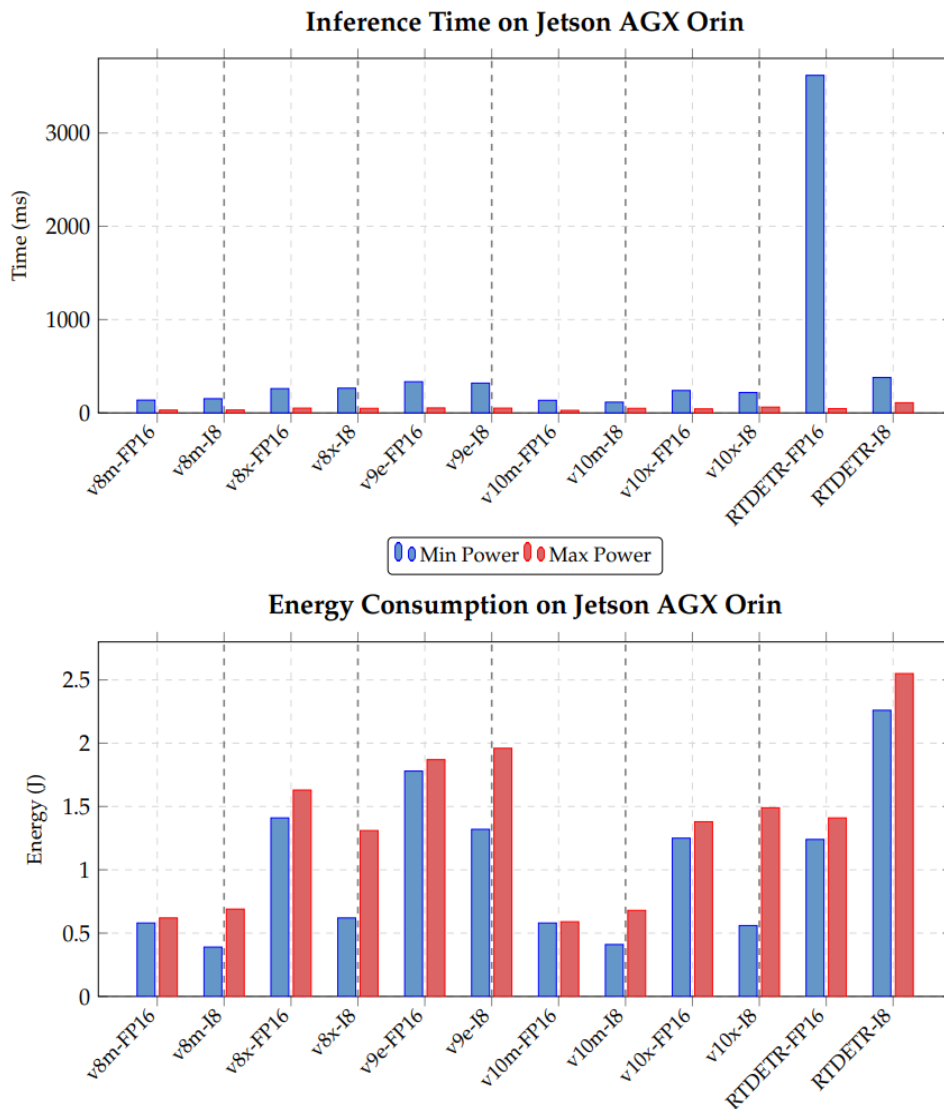
cây. Ưu điểm nổi bật của công trình là nhờ mô-đun bù trừ chuyển động dựa trên dữ liệu từ thiết bị bay, đường viền dự đoán của mạng nơ-ron hội tụ cực kỳ nhanh và ôm sát lấy hình bóng nạn nhân bất chấp sự biến dạng của đám nhiệt, giúp tỷ lệ bao phủ mục tiêu được nâng lên mức tối đa. Tuy nhiên, nhược điểm kỹ thuật được ghi nhận là kiến trúc mạng lõi phía sau thuật toán này vẫn còn khá nặng nề, đòi hỏi dung lượng bộ nhớ đồ họa lớn khiến việc tích hợp toàn bộ hệ thống lên các nền tảng thiết bị bay dân dụng giá rẻ gặp rất nhiều rào cản về mặt vật lý và chi phí.



Hình 1.4 - Sơ đồ khối kiến trúc theo vết kết hợp bù trừ chuyển động [4]

Cuối cùng, hướng nghiên cứu về tối ưu hóa hiệu năng trên máy tính nhúng đã nhận được sự quan tâm đặc biệt. Công trình đánh giá các mô hình học sâu phục vụ phát hiện người trong ảnh nhiệt trên nền tảng Jetson AGX Orin công bố trên tạp chí Journal of Imaging năm 2025 đã cung cấp một quy trình triển khai phần mềm khép kín [5]. Chức năng của nghiên cứu là so sánh hiệu năng của các kiến trúc phát hiện đối tượng và phân đoạn ngữ nghĩa, thực hiện chuyển đổi mô hình sang định dạng tối ưu để tận dụng các lõi gia tốc phần cứng chuyên dụng. Phạm vi ứng dụng của công trình nhắm đến việc chế tạo các cụm camera nhiệt thông minh có khả năng tự động bám bắt mục tiêu độc lập mà không cần truyền tải video về trạm điều khiển mặt đất. Ưu điểm của phương pháp tối ưu hóa đồ thị tính toán này là giúp hệ thống giảm độ trễ suy luận xuống mức vài mili-giây, đáp ứng hoàn hảo yêu cầu xuất tín hiệu điều khiển bay mượt mà theo thời gian thực. Mặc dù vậy, nhược điểm chí mạng của hệ thống là khi hoạt động liên tục ở tần số cao, máy tính nhúng biên sinh ra lượng nhiệt rất lớn, nếu không có cơ chế tản

nhệt khí động học phù hợp sẽ dẫn đến hiện tượng giảm xung nhịp đột ngột, làm hệ thống tự hành mất tính ổn định trong các nhiệm vụ kéo dài.



Hình 1.5 - Đồ thị so sánh thời gian suy luận và tiêu thụ năng lượng [5]

1.2.3 Đánh giá chung và xác định khoảng trống nghiên cứu

Qua việc khảo sát và phân tích toàn diện các công trình nghiên cứu khoa học trong và ngoài nước trong giai đoạn từ năm 2024 đến nay, có thể nhận thấy một sự phân hóa rõ rệt trong các phương pháp tiếp cận đối với bài toán nhận dạng đối tượng bằng ảnh nhiệt trên thiết bị bay không người lái. Các công bố hiện nay đang tập trung vào hai hướng chính, bao gồm nhóm các giải pháp gia tăng độ chính xác thông qua việc sử dụng các mô hình nơ-ron sâu phức tạp, cấu trúc biến áp hoặc mạng siêu phân giải, nhưng lại vấp phải rào cản chí mạng về khối lượng tính toán, dẫn đến việc không thể vận hành độc

lập trên các thiết bị nhúng di động. Nhóm thứ hai lại tập trung tối ưu hóa các mô hình nhẹ để đạt tốc độ thời gian thực trên bo mạch nhúng, tuy nhiên, sự cắt giảm cấu trúc quá mức thường dẫn đến tình trạng bỏ sót mục tiêu hoặc nhận diện nhầm.

Từ những đánh giá trên, có thể xác định hai khoảng trống công nghệ then chốt mà đề tài khóa luận này sẽ tập trung giải quyết. Thứ nhất là bài toán cân bằng giữa độ chính xác và khả năng triển khai thực tế trên máy tính nhúng. Các giải pháp hiện tại thường hoặc là quá công kênh, gây quá tải cho bộ xử lý. Luận văn sẽ tập trung xây dựng và tối ưu mạng nơ-ron Anchor Free dựa trên nền tảng RetinaNet để vừa duy trì khả năng nhận dạng mục tiêu nhỏ ở khoảng cách xa, vừa tinh gọn khối lượng tính toán nhằm tối ưu hóa cho nền tảng Jetson Orin Nano. Thứ hai là khoảng trống về quy trình đóng gói và biên dịch phần mềm từ môi trường nghiên cứu sang thiết bị biên cũng là một rào cản cần vượt qua. Đề tài không chỉ dừng lại ở việc xây dựng thuật toán, mà còn đi sâu vào tối ưu hóa biên dịch qua công cụ TensorRT để tận dụng phần cứng gia tốc đồ họa trên Jetson Orin Nano, đảm bảo hệ thống có thể chạy độc lập, tự chủ hoàn toàn trên không trung mà không cần phụ thuộc vào băng thông truyền tải về trạm điều khiển mặt đất. Đây chính là cơ sở khoa học và động lực thực tiễn quan trọng nhất để luận văn quyết định thực hiện đề tài này, nhằm cung cấp một giải pháp khả thi, đồng bộ và hiệu quả cho các nhiệm vụ cứu nạn, cứu hộ trong tương lai.

CHƯƠNG 2: MỤC TIÊU ĐỀ TÀI VÀ YÊU CẦU KỸ THUẬT

2.1 Mục tiêu đề tài

2.1.2 Mục tiêu chung

Mục tiêu chung của đề tài là thiết kế, chế tạo một nguyên mẫu thiết bị bay không người lái quadcopter vận hành ổn định, đóng vai trò làm nền tảng cơ động tích hợp hệ thống camera ảnh nhiệt nhằm phát hiện và nhận dạng nguồn thân nhiệt người phục vụ các công tác giám sát, tìm kiếm. Tuy nhiên, do khối lượng nghiên cứu của đề tài quá lớn và cần nhiều thời gian để hoàn thành nghiên cứu nên toàn bộ nội dung nghiên cứu được chia làm hai nội dung:

- Phần một là nghiên cứu chế tạo thiết kế phần cứng, mô hình hóa cơ khí và phát triển thuật toán điều khiển bay.
- Phần hai là nghiên cứu xây dựng thuật toán để phân tích, nhận dạng đối tượng từ luồng dữ liệu ảnh nhiệt thu thập từ camera.

2.1.2 Mục tiêu riêng

Mục tiêu cốt lõi của đề tài là nghiên cứu, xây dựng và tối ưu hóa mô hình trí tuệ nhân tạo nhận dạng người (bao gồm cả các đối tượng sống và người đi lạc) dựa trên dữ liệu ảnh nhiệt định dạng white-hot. Bên cạnh việc tinh chỉnh thuật toán để đạt độ chính xác cao trong môi trường thực tế, đề tài còn hướng tới mục tiêu triển khai thực tiễn thông qua việc chuyển đổi định dạng trọng số mô hình, nhằm tích hợp và vận hành trơn tru trên nền tảng máy tính nhúng biên Jetson Orin Nano phiên bản 8GB. Hệ thống sau khi tối ưu được kỳ vọng sẽ hoạt động ổn định trên thiết bị bay không người lái với tầm quan sát và phát hiện hiệu quả ở độ cao từ 50 mét trở lên so với mặt đất, phục vụ đắc lực cho công tác tìm kiếm, cứu nạn.

2.2 Đối tượng và phạm vi nghiên cứu

2.2.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu trọng tâm của đề tài là mô hình thuật toán trí tuệ nhân tạo nhận dạng người (bao gồm cả các đối tượng sống và người đi lạc) dựa trên việc phân tích đặc trưng của dữ liệu ảnh nhiệt định dạng white-hot.

2.2.2 Phạm vi nghiên cứu

Để đảm bảo tính tập trung, đề tài xác định phạm vi nghiên cứu của mô hình thuật toán giới hạn trong các điều kiện vận hành và kỹ thuật cụ thể. Về mặt đối tượng, mô hình tập trung chuyên sâu vào nhiệm vụ nhận dạng người đi lạc thông qua phân tích dữ liệu ảnh nhiệt. Về quy chuẩn dữ liệu, thông số đầu vào của mạng nơ-ron được thiết lập ở độ phân giải 512x512 pixel nhằm tối ưu hóa quá trình tính toán. Về khả năng vận hành thực tế, hệ thống được thiết kế để đảm bảo năng lực phát hiện mục tiêu hiệu quả ở phạm vi hoạt động từ 50 mét trở lên so với mặt đất. Cuối cùng, về mặt phần cứng triển khai, toàn bộ trọng số mô hình sau khi huấn luyện được giới hạn tối ưu hóa để đảm bảo khả năng thực thi trơn tru và đạt hiệu suất tốt nhất trên nền tảng bo mạch máy tính nhúng Jetson Orin Nano phiên bản 8GB.

2.3 Yêu cầu kỹ thuật

Để triển khai thành công một hệ thống nhận dạng đối tượng dựa trên học sâu ứng dụng cho thiết bị bay không người lái, quá trình phát triển thuật toán cần đáp ứng các yêu cầu kỹ thuật khắt khe nhằm đảm bảo sự thống nhất giữa môi trường huấn luyện trên máy tính trạm và môi trường triển khai thực tế trên máy tính nhúng. Các yêu cầu này được chia thành ba trụ cột chính, đóng vai trò nền tảng cho sự ổn định và hiệu năng của toàn bộ hệ thống.

Yêu cầu đầu tiên và mang tính tiền đề chính là việc thiết lập môi trường phần mềm hỗ trợ tính toán song song, cụ thể là cài đặt bộ thư viện CUDA và cuDNN. Do các thuật toán học sâu hiện đại đòi hỏi tốc độ xử lý ma trận cực lớn, việc khai thác sức mạnh phần cứng từ GPU là yêu cầu bắt buộc thay vì sử dụng CPU. Việc cài đặt cần đảm bảo tính tương thích tuyệt đối giữa phiên bản hệ điều hành, driver, bộ công cụ CUDA toolkit và thư viện cuDNN chuyên sâu cho mạng nơ-ron. Sự đồng bộ này không chỉ giúp tối ưu

hóa thời gian huấn luyện mà còn là điều kiện cần để đảm bảo các tiến trình lan truyền tiến và lan truyền ngược diễn ra chính xác, tránh các lỗi xung đột phần cứng hoặc sai số trong quá trình tính toán đạo hàm.

Yêu cầu thứ hai liên quan đến việc xây dựng mô hình thuật toán trên nền tảng TensorFlow. Hệ thống yêu cầu việc sử dụng TensorFlow và giao diện lập trình cấp cao Keras để định nghĩa kiến trúc mạng nơ-ron một cách linh hoạt và khoa học. Môi trường lập trình cần hỗ trợ tốt cho việc tùy biến các lớp mạng đặc thù, bao gồm các cấu trúc Anchor Free, nhánh hồi quy và các hàm mất mát tối ưu hóa hình học. Việc xây dựng mô hình không chỉ dừng lại ở việc lắp ghép các lớp tích chập có sẵn, mà còn yêu cầu khả năng điều khiển luồng dữ liệu thông qua các phép toán tensor, quản lý bộ nhớ đồ họa hiệu quả và hỗ trợ kỹ thuật huấn luyện với độ chính xác hỗn hợp để đạt hiệu năng tính toán tối đa mà vẫn giữ được độ hội tụ cần thiết của mô hình.

Yêu cầu thứ ba, cũng là bước hiện thực hóa quan trọng nhất, là quá trình chuyển đổi và biên dịch mô hình sang định dạng tương thích với thiết bị nhúng. Sau khi quá trình huấn luyện kết thúc với file trọng số ở định dạng chuẩn của framework, mô hình cần được chuyển đổi sang định dạng ONNX để làm lớp trung gian. Từ file ONNX này, yêu cầu kỹ thuật đặt ra là phải sử dụng công cụ biên dịch chuyên dụng TensorRT để chuyển đổi thành file engine. Quá trình biên dịch này yêu cầu người lập trình phải am hiểu về các cơ chế tối ưu hóa đồ thị tính toán như gộp các lớp tích chập, ép kiểu dữ liệu sang bán độ chính xác (FP16), và tự động dò tìm cấu hình nhân tính toán tối ưu cho bộ xử lý đồ họa trên Jetson Orin Nano. Sự tối ưu hóa này là nhân tố quyết định để giảm độ trễ suy luận xuống mức tối thiểu, biến một mô hình học sâu phức tạp trở thành một chương trình chạy mượt mà, đáp ứng yêu cầu nhận dạng đối tượng theo thời gian thực trên không trung.

Đáp ứng đầy đủ ba yêu cầu kỹ thuật trên sẽ đảm bảo hệ thống nhận dạng không chỉ vận hành ổn định trong pha huấn luyện ngoại tuyến mà còn đạt được sự linh hoạt và hiệu suất cao nhất khi triển khai độc lập tại biên, đáp ứng đúng mục tiêu ứng dụng của đề tài.

CHƯƠNG 3: CƠ SỞ LÝ THUYẾT

3.1 Cơ sở vật lý của ảnh nhiệt và bức xạ hồng ngoại

3.1.1 Bản chất bức xạ nhiệt của vật đen tuyệt đối

Để hiểu rõ cách thức hệ thống máy tính có thể nhận diện đối tượng người trong điều kiện hoàn toàn không có ánh sáng, chúng ta cần đi sâu vào bản chất vật lý của bức xạ điện từ. Mọi vật thể trong vũ trụ có nhiệt độ lớn hơn độ không tuyệt đối hay không Kelvin đều không ngừng phát ra các sóng điện từ vào không gian xung quanh. Quá trình này được gọi là bức xạ nhiệt [6]. Cơ sở toán học nền tảng nhất để mô tả quá trình này là định luật bức xạ Planck. Định luật này mô tả sự phân bố năng lượng bức xạ của một vật đen tuyệt đối theo bước sóng tại một nhiệt độ nhiệt động lực học nhất định [7]. Công thức Planck được biểu diễn dưới dạng mật độ phổ bức xạ như sau:

$$M(\lambda, T) = \frac{2\pi hc^2}{\lambda^5} \frac{1}{e^{\frac{hc}{\lambda kT}} - 1} \quad (3.1)$$

Trong phương trình trên, hằng số Planck h và tốc độ ánh sáng c trong chân không là các hằng số vật lý phổ quát. Biến số bước sóng được ký hiệu là λ và biến số nhiệt độ tuyệt đối của vật thể được ký hiệu là T . Hằng số Boltzmann k đóng vai trò liên kết giữa nhiệt độ và năng lượng lượng tử. Phương trình này chỉ ra rằng năng lượng bức xạ không phân bố đều trên mọi bước sóng mà đạt cực đại tại một dải bước sóng cụ thể phụ thuộc vào nhiệt độ của vật phát.

3.1.2 Định luật dịch chuyển Wien và ứng dụng nhận diện người

Từ định luật Planck, việc lấy đạo hàm hàm mật độ phổ theo bước sóng và cho bằng không sẽ dẫn đến định luật dịch chuyển Wien [6]. Định luật này xác định mối liên hệ nghịch đảo giữa nhiệt độ của một vật thể và bước sóng mà tại đó vật thể phát ra năng lượng bức xạ mạnh nhất. Công thức của định luật Wien được viết gọn lại như sau:

$$\lambda_{max} = \frac{b}{T} \quad (3.2)$$

Với b là hằng số dịch chuyển Wien xấp xỉ bằng 2898 micromet Kelvin. Khi áp dụng phương trình này vào sinh lý học cơ thể người, với nhiệt độ bề mặt trung bình của da người dao động quanh mức 34 đến 37 độ C tương đương với khoảng 310 Kelvin, chúng ta có thể tính toán được bước sóng bức xạ cực đại rơi vào khoảng 9.3 đến 9.5 micromet. Dải bước sóng này nằm hoàn toàn trong vùng hồng ngoại xa hay còn gọi là bức xạ nhiệt của phổ điện từ. Đây là cơ sở vật lý cốt lõi cho phép các thiết bị cảm biến ảnh nhiệt được thiết kế chuyên biệt để hoạt động trong dải sóng từ 8 đến 14 micromet nhằm thu nhận tối đa bức xạ từ cơ thể người, tách biệt hoàn toàn với vùng ánh sáng khả kiến [8].

3.1.3 Nguyên lý hoạt động của vi bức xạ kế và hệ màu trắng nóng

Các camera nhiệt hiện đại sử dụng công nghệ vi bức xạ kế không làm mát để chuyển đổi năng lượng hồng ngoại vô hình thành tín hiệu điện tử [8]. Bề mặt cảm biến được cấu tạo từ một mảng lưới gồm hàng trăm ngàn phần tử điểm ảnh siêu nhỏ, thường làm từ vật liệu oxit vanadi có điện trở phụ thuộc rất mạnh vào nhiệt độ. Khi bức xạ hồng ngoại từ môi trường đi qua hệ thống thấu kính gecmani và hội tụ lên mảng vi bức xạ kế, năng lượng nhiệt làm thay đổi trực tiếp nhiệt độ của các phần tử cảm biến, từ đó làm thay đổi điện trở của chúng. Mạch điện tử tích hợp sẽ đo lường sự thay đổi dòng điện đi qua các điện trở này và số hóa thành dạng ma trận dữ liệu hai chiều.

Hệ thống biến đổi tín hiệu này không ghi nhận màu sắc quang phổ như camera thông thường mà chỉ ghi nhận cường độ năng lượng. Do đó, hình ảnh nội suy đầu ra tồn tại ở dạng đơn sắc. Trong hệ quy chiếu màu trắng nóng, thuật toán ánh xạ tuyến tính được áp dụng để chuyển đổi ma trận nhiệt độ thành ma trận điểm ảnh với giá trị độ sáng từ 0 đến 255. Phương trình ánh xạ cơ bản như sau:

$$I(x,y) = 255 \times \frac{T(x,y) - T_{\min}}{T_{\max} - T_{\min}} \quad (3.3)$$

Trong đó I là giá trị độ sáng tại tọa độ không gian x, y , T là nhiệt độ đo được tại điểm đó. Các giới hạn nhiệt độ thấp nhất và cao nhất trong khung hình được nội suy liên tục. Nhờ phép biến đổi này, các đối tượng có thân nhiệt cao như con người sẽ được gán cho các giá trị điểm ảnh tiến gần đến 255 tạo ra sắc trắng rực rỡ, trong khi các mảng môi trường lạnh lẽo xung quanh sẽ có giá trị tiến về 0 tạo ra sắc đen. Sự chênh lệch biên độ

cực lớn này cung cấp một đường viền hình học sắc nét tuyệt đối, tạo điều kiện thuận lợi nhất cho các thuật toán thị giác máy tính trích xuất đặc trưng [8].



Hình 3.1 - Ảnh nhiệt trắng nóng [9]

3.2 Mạng nơ-ron tích chập và sự trích xuất đặc trưng không gian

3.2.1 Phép toán tích chập ba chiều

Thị giác máy tính hiện đại dựa phần lớn vào kiến trúc mạng nơ-ron tích chập nhằm mô phỏng cách thức vỏ não thị giác của con người nhận diện hình ảnh [10]. Khác với tích chập hai chiều thông thường, phép toán tích chập ba chiều mở rộng khả năng trích xuất đặc trưng không chỉ theo hai chiều không gian (chiều rộng và chiều cao) mà còn theo chiều sâu (chiều thứ ba, ví dụ như kênh độ sâu không gian hoặc chuỗi các khung hình liên tiếp theo thời gian) [11]. Hoạt động cốt lõi của mạng nơ-ron này là sử dụng các bộ lọc kỹ thuật số ba chiều trượt qua toàn bộ khối ma trận dữ liệu để phát hiện các mẫu hình học từ đơn giản đến phức tạp. Xét một khối dữ liệu đầu vào I và một bộ lọc ma trận K có kích thước m nhân n nhân d , phép tích chập tạo ra một khối bản đồ đặc trưng S được định nghĩa bằng tổng hợp các tích chập toán học trong không gian ba chiều:

$$S(i, j, k) = \sum_m \sum_n \sum_d I(i + m, j + n, k + d) K(m, n, d) \quad (3.4)$$

Trong quá trình huấn luyện, các giá trị phần tử của bộ lọc K không được cài đặt thủ công mà là các trọng số được hệ thống tự động cập nhật liên tục thông qua thuật toán lan truyền ngược. Ở các tầng mạng nông, các bộ lọc khối này tự động học cách nhận diện các cạnh thẳng, góc nhọn và sự thay đổi cường độ sáng đột ngột không chỉ trên bề mặt ảnh mà còn xuyên suốt theo chiều sâu của dữ liệu. Càng đi sâu vào các tầng mạng tiếp theo, thông tin từ các khối bản đồ đặc trưng trước đó được kết hợp lại, cho phép các bộ lọc học được những cấu trúc ngữ nghĩa phức tạp hơn như hình bóng tổng thể hay tư thế di chuyển đa chiều của con người [10], [11].

3.2.2 Hàm kích hoạt phi tuyến

Sau mỗi phép toán tích chập, để mạng nơ-ron có khả năng phân tách các đường biên ranh giới quyết định phức tạp trong không gian đa chiều, một hàm kích hoạt phi tuyến bắt buộc phải được áp dụng lên từng phần tử của bản đồ đặc trưng. Hàm chỉnh lưu tuyến tính tương đương viết tắt là ReLU được sử dụng phổ biến nhất nhờ đặc tính tính toán nhanh và hạn chế được vấn đề triệt tiêu đạo hàm. Công thức của hàm được biểu diễn rất đơn giản:

$$f(x) = \max(0, x) \quad (3.5)$$

Hàm này loại bỏ toàn bộ các giá trị điểm ảnh âm bằng cách đưa chúng về 0 và giữ nguyên các giá trị dương. Việc này không chỉ tạo ra tính phi tuyến mà còn mang lại đặc tính thưa thớt cho tín hiệu, giúp mạng tập trung vào các đặc trưng quan trọng nhất của đối tượng nhiệt.

3.3 Kiến trúc lõi của mô hình RetinaNet

Trong lĩnh vực phát hiện đối tượng dựa trên học sâu, các kiến trúc mạng được chia thành hai trường phái chính là mô hình hai giai đoạn và mô hình một giai đoạn. Các mô hình hai giai đoạn thực hiện tuần tự quá trình trích xuất các vùng không gian tiềm năng chứa vật thể rồi mới đưa vào mạng tinh chỉnh để phân loại và dự đoán tọa độ [12]. Phương pháp này mang lại độ chính xác rất cao nhưng lại tiêu tốn quá nhiều tài nguyên tính toán khiến tốc độ xử lý bị suy giảm nghiêm trọng, không thể đáp ứng yêu cầu truyền dữ liệu thời gian thực của thiết bị bay. Ngược lại, các mô hình một giai đoạn bỏ qua

hoàn toàn quá trình đề xuất vùng tiềm năng, thực hiện tính toán xác suất phân loại và hồi quy tọa độ song song trực tiếp trên toàn bộ mạng lưới không gian của bức ảnh.

RetinaNet ra đời như một mô hình một giai đoạn mang tính đột phá, được thiết kế để giải quyết điểm yếu chí mạng của các mô hình cùng loại thời bấy giờ là sự mất cân bằng giữa cực kỳ nhiều mảng hậu cảnh trống trải và một số lượng rất ít đối tượng cần phát hiện [13]. Kiến trúc của RetinaNet được kết nối từ ba mạng nơ-ron phụ trợ đóng vai trò độc lập gồm mạng trích xuất đặc trưng hình học cơ sở, mạng xây dựng tháp đa độ phân giải và mạng con dự đoán phân loại kết hợp tọa độ dựa trên cơ chế Anchor Box hoặc Anchor Free.

3.3.1 Mạng trích xuất đặc trưng

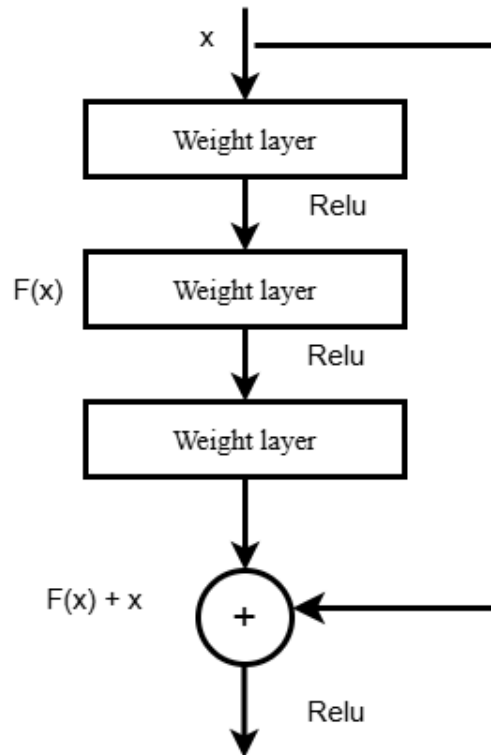
Thành phần đầu tiên làm nhiệm vụ phân tích ảnh nhiệt đầu vào là một mạng nơ-ron tích chập rất sâu được xây dựng dựa trên kiến trúc mạng thặng dư. Lịch sử phát triển của học sâu đã chỉ ra rằng khi tăng số lượng các tầng tích chập, mô hình sẽ học được các đặc trưng càng phức tạp. Tuy nhiên, khi mạng trở nên quá sâu, tín hiệu đạo hàm trong quá trình lan truyền ngược bị nhân với hàng loạt các trọng số nhỏ bé, dẫn đến hiện tượng đạo hàm tiến dần về 0 khi truyền đến các tầng đầu tiên. Điều này khiến các tầng mạng nông hoàn toàn không thể học thêm được kiến thức mới, gây ra sự suy giảm chất lượng toàn hệ thống.

Để phá vỡ giới hạn vật lý toán học này, mạng thặng dư đề xuất một cấu trúc đặc biệt mang tên khối thặng dư. Ý tưởng cốt lõi là thay vì ép buộc một chuỗi các lớp tích chập phải học trực tiếp một ánh xạ mục tiêu phức tạp, mạng sẽ chỉ yêu cầu các lớp này học một ánh xạ phần dư, đồng thời tạo ra một luồng kết nối tắt đi vòng qua các lớp tính toán để cộng trực tiếp dữ liệu đầu vào nguyên bản vào dữ liệu đầu ra. Phương trình biểu diễn một khối thặng dư được định nghĩa như sau:

$$y = F(x, \{W_i\}) + x \quad (3.6)$$

Trong phương trình học máy này, biến đầu vào của khối cấu trúc được ký hiệu là x và đầu ra cuối cùng là y . Hàm đại diện cho phần dư mà các lớp tích chập bên trong cần phải học để biến đổi dữ liệu được biểu diễn thông qua ma trận trọng số. Phép cộng ở cuối công thức cho phép luồng thông tin không gian gốc và quan trọng nhất là luồng

đạo hàm toán học được truyền tải xuyên suốt một cách trực tiếp từ cuối mạng lên đầu mạng mà không bị suy hao. Cơ chế này đảm bảo mạng có thể được xếp chồng lên tới hàng chục thậm chí hàng trăm lớp mà vẫn duy trì được độ hội tụ ổn định, giúp bóc tách hoàn hảo các mức độ sáng nhạt khác nhau của bức xạ nhiệt sinh học so với môi trường xung quanh [10], một khối Residual Block được thể hiện qua hình sau:



Hình 3.2 - Một Residual Block

3.3.2 Mạng tháp đặc trưng FPN

Một trong những thách thức to lớn nhất khi phân tích hình ảnh từ thiết bị bay là sự biến động cực độ về tỷ lệ kích thước của con người dựa trên độ cao bay. Một người đứng ngay dưới ống kính có thể chiếm hàng ngàn điểm ảnh trong khi một người ở khoảng cách xa hàng trăm mét chỉ hiện diện như một đốm sáng cấu thành từ vài chục điểm ảnh. Nếu chỉ sử dụng một bản đồ đặc trưng ở tầng cuối cùng của mạng trích xuất, hệ thống sẽ hoàn toàn mù lòa trước các đối tượng nhỏ do quá trình gộp cụm liên tục đã làm mất mát thông tin không gian chi tiết.

Để giải quyết triệt để rào cản đa tỷ lệ này, kiến trúc mạng tháp đặc trưng được thiết kế để kết hợp sức mạnh phân giải vị trí sắc nét ở các tầng nông với sức mạnh ngữ nghĩa phân loại sâu sắc ở các tầng sâu [15]. Quá trình này được thực hiện thông qua một cơ chế truyền thông tin ngược từ trên xuống kết hợp với các đường dẫn kết nối theo phương ngang. Về mặt toán học, bản đồ đặc trưng tại một cấp độ nhất định trong tháp được tính toán dựa trên sự hòa trộn dữ liệu từ cấp độ liền kề phía trên nó. Cụ thể, bản đồ đặc trưng mang ngữ nghĩa sâu ở tầng trên sẽ được tăng gấp đôi kích thước không gian thông qua thuật toán nội suy láng giềng gần nhất. Sau đó, ma trận vừa được phóng to này sẽ được cộng trực tiếp với bản đồ đặc trưng tương ứng từ luồng mạng trích xuất đi lên thông qua một phép tích chập đồng bộ kênh [15]. Phương trình hòa trộn được định nghĩa:

$$P_i = \text{Conv}_{3 \times 3}(\text{Upsample}(P_{i+1}) + \text{Conv}_{1 \times 1}(C_i)) \quad (3.7)$$

Thành phần bản đồ đặc trưng nguyên gốc được lấy từ lớp tích chập tương ứng. Phép tính phóng to ma trận giúp đồng nhất số lượng phần tử không gian. Phép tích chập một nhân một được sử dụng chỉ để thay đổi số lượng kênh dữ liệu cho khớp nhau trước khi thực hiện phép cộng ma trận. Cuối cùng, một phép tích chập ba nhân ba được áp dụng lên kết quả cộng để làm mịn các hiệu ứng nhiễu răng cưa do thuật toán nội suy sinh ra, tạo ra một bản đồ hoàn chỉnh. Cấu trúc kim tự tháp này cho phép hệ thống tạo ra các dự đoán nhận diện độc lập trên nhiều cấp độ khác nhau, trong đó các cấp độ lớn phụ trách tìm kiếm người ở xa và các cấp độ nhỏ phụ trách tìm kiếm người ở gần.

3.3.3 Cơ sở toán học và cơ chế hoạt động của Anchor Box

3.3.3.1 Khái niệm và phương pháp sinh Anchor Box

Trong các thuật toán phát hiện đối tượng tiêu chuẩn, việc dự đoán trực tiếp bốn tọa độ ngẫu nhiên của một vật thể từ các đặc trưng điểm ảnh là một bài toán hàm lỗi vô cùng khó khăn, dẫn đến quá trình hội tụ chậm và thiếu chính xác. Để biến đổi bài toán khó này thành một quy trình dễ giải quyết hơn, kiến trúc RetinaNet nguyên bản sử dụng một khái niệm hình học được gọi là Anchor Box [12].

Anchor Box bản chất là các hình chữ nhật tham chiếu được định nghĩa trước về mặt kích thước và tỷ lệ, được trải đều một cách dày đặc trên toàn bộ lưới không gian của hình ảnh. Thay vì yêu cầu mạng nơ-ron tự đoán mò tọa độ một cách tuyệt đối từ không

gian trống, hệ thống sử dụng các Anchor Box này làm các bộ phóng giả định. Mạng nơ-ron chỉ cần làm nhiệm vụ trả lời hai câu hỏi đơn giản hơn rất nhiều: Anchor Box này có chứa đối tượng hay không và cần dịch chuyển trung tâm hoặc thu phóng các cạnh của Anchor Box này bao nhiêu đơn vị để nó ôm khít lấy đối tượng thực tế.

Để bao quát mọi hình dáng và kích thước tư thế của con người trong ảnh nhiệt, từ tư thế đứng thẳng đến nằm trườn, hệ thống sinh Anchor Box dựa trên các tổ hợp toán học về tỷ lệ diện tích và tỷ lệ khung hình. Tại mỗi vị trí không gian trên các bản đồ tháp đặc trưng, hệ thống sẽ khởi tạo nhiều Anchor Box với các diện tích cơ sở tăng dần theo cấp số nhân nhằm tìm kiếm các đối tượng lớn dần. Đồng thời, tại mỗi diện tích cơ sở đó, ba tỷ lệ khung hình chuẩn là một chia một, một chia hai và hai chia một tương ứng với hình vuông, hình chữ nhật đứng dọc và hình chữ nhật nằm ngang được áp dụng. Việc sử dụng tổ hợp tham số này giúp sinh ra một lượng lớn các hộp bao phủ đa dạng, đảm bảo dù người xuất hiện với tư thế nào thì cũng luôn tồn tại ít nhất một Anchor Box có hình dáng gần giống với họ nhất để mạng nơ-ron tiến hành tinh chỉnh.

3.3.3.2 Khái niệm và phương pháp sinh Anchor Free

Việc sử dụng Anchor Box gây ảnh hưởng nặng nề về mặt tính toán cho mô hình vì cơ chế rải hàng ngàn Anchor Box trên một tầng ma trận đặc trưng. Để khắc phục những hạn chế về mặt tính toán của Anchor Box, các nghiên cứu hiện đại đã chuyển hướng sang phương pháp tiếp cận Anchor Free[14]. Thay vì tính toán độ lệch so với các hộp định sẵn, phương pháp này dự đoán trực tiếp tọa độ của vật thể.

Toán học của cơ chế dự đoán Anchor Free : giả sử một điểm ảnh trên bản đồ đặc trưng có tọa độ không gian là (x, y) và tháp đặc trưng tại tầng đó có hệ số thu phóng là s tọa độ tương ứng của điểm ảnh này khi ánh xạ ngược về không gian của bức ảnh gốc, ký hiệu là (c_x, c_y) được thiết lập bằng công thức:

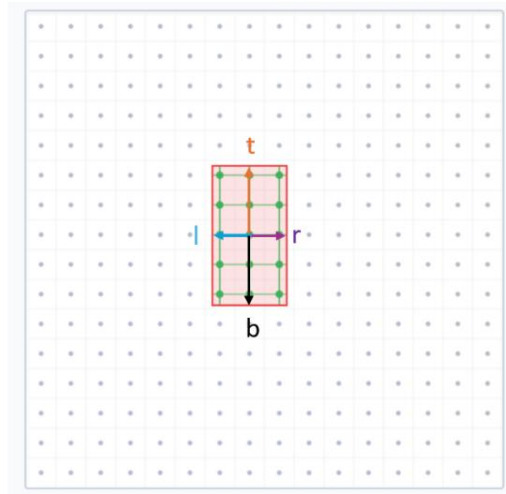
$$\begin{cases} c_x = \left\lfloor \frac{s}{2} \right\rfloor + x \cdot s \\ c_y = \left\lfloor \frac{s}{2} \right\rfloor + y \cdot s \end{cases} \quad (3.8)$$

Nếu điểm tâm (c_x, c_y) được nhánh phân loại xác định là thuộc về lớp đối tượng người, nhánh hồi quy sẽ xuất ra một véc-tơ gồm bốn giá trị thực (l, t, r, b) . Bốn đại lượng này

lần lượt đại diện cho khoảng cách từ tâm đến cạnh trái, cạnh trên, cạnh phải và cạnh dưới của hình chữ nhật bao quanh đối tượng. Tọa độ hình học tuyệt đối của bounding box dự đoán $x_{min}, y_{min}, x_{max}, y_{max}$ được khôi phục thông qua hệ phương trình tuyến tính như sau:

$$\begin{cases} x_{min} = c_x - l, & y_{min} = c_y - t \\ x_{max} = c_x + r, & y_{max} = c_y + b \end{cases} \quad (3.9)$$

Hình 3.3 minh họa trực quan cơ chế này: khi nhánh phân loại xác định một điểm tâm thuộc về đối tượng người, hệ thống sẽ sinh ra bốn véc-tơ khoảng cách l, t, r, b hướng từ điểm đó đến các cạnh tương ứng để định hình bounding box.



Hình 3.3 - Tọa độ của Anchor Free

3.3.3.3 Thuật toán đo lường độ giao thoa (IoU)

Để có thể huấn luyện mạng nơ-ron cập nhật trọng số, hệ thống cần phải xác định xem trong hàng chục ngàn Anchor Box được sinh ra, Anchor Box nào được coi là một dự đoán tích cực chứa người và Anchor Box nào bị coi là một dự đoán tiêu cực chứa nền trống. Toán học xác định sự phân loại này dựa trên thuật toán tính diện tích giao thoa trên hợp nhất.

Giả sử chúng ta có một Anchor Box dự đoán và một bounding box chứa thực thể người được dán nhãn thủ công từ trước. Độ giao thoa là tỷ lệ phần trăm được tính bằng cách lấy diện tích của phần không gian mà hai hình chữ nhật này cắt nhau chia cho tổng diện

tích không gian của cả hai hình chữ nhật hợp lại trừ đi phần cắt nhau để tránh tính lặp. Phương trình toán học biểu diễn phép tính này như sau:

$$IoU = \frac{\text{Area (Anchor Box} \cap \text{Ground Truth)}}{\text{Area (Anchor Box} \cup \text{Ground Truth)}} \quad (3.10)$$

Giá trị của độ giao thoa luôn dao động từ 0 đến 1. Nếu hai hình chữ nhật hoàn toàn không chạm nhau, tử số bằng 0 dẫn đến kết quả bằng 0. Nếu hai hình chữ nhật chồng khít lên nhau hoàn toàn, tử số và mẫu số bằng nhau dẫn đến kết quả bằng 1. Dựa trên phép đo lường hình học chặt chẽ này, hệ thống thiết lập các ngưỡng gán nhãn cứng nhắc trong quá trình học. Một Anchor Box sẽ được gán nhãn là tích cực nếu giá trị giao thoa với khung thực thể người vượt qua ngưỡng 0.5. Ngược lại, nếu giá trị này nằm dưới ngưỡng 0.4, Anchor Box đó sẽ bị ép buộc gán nhãn là nền tiêu cực. Phần khoảng trống ở giữa từ 0.4 đến 0.5 được hệ thống chủ động bỏ qua và không đưa vào tính toán đạo hàm để tránh việc cung cấp các tín hiệu gây nhầm lẫn cho mạng nơ-ron.

3.3.4 Mạng con dự đoán

Tại mỗi cấp độ của tháp đặc trưng P_i , mô hình rẽ nhánh thành hai mạng con song song, hoạt động dưới dạng các mạng nơ-ron tích chập thuần túy để thực hiện hai nhiệm vụ toán học độc lập:

- Nhánh phân loại có nhiệm vụ dự đoán xác suất một Anchor Box chứa đối tượng người hoặc chỉ là hậu cảnh. Đầu ra của nhánh này sử dụng hàm kích hoạt Sigmoid để tính toán xác suất độc lập cho từng lớp:

$$p = \sigma(x) = \frac{1}{1 + e^{-x}} \quad (3.11)$$

- Với đầu ra có kích thước không gian là $W \times H \times (K \times A)$, trong đó K là số lớp đối tượng và A là số lượng Anchor Box tại mỗi vị trí không gian.

- Nhánh hồi quy khung không dự đoán trực tiếp tọa độ tuyệt đối, mà tính toán các tham số độ lệch giữa khung dự đoán và Anchor Box ban đầu nhằm khớp chặt vào hình dáng người. Bốn tham số hồi quy bao gồm khoảng cách tâm t_x, t_y và tỷ lệ kích thước t_w, t_h được tính toán bằng các công thức:

$$\begin{cases} t_x = \frac{x - x_0}{w_a}, & t_y = \frac{y - y_0}{h_a} \\ t_w = \log\left(\frac{w}{w_a}\right), & t_h = \log\left(\frac{h}{h_a}\right) \end{cases} \quad (3.12)$$

Trong đó, x, y, w, h là tọa độ tâm và kích thước của bounding box dự đoán, còn x_a, y_a, w_a, h_a là thông số tương ứng của Anchor Box mặc định.

3.3.5 Nhánh mạng dự đoán và các hàm mất mát

3.3.5.1 Giới hạn của hàm Entropy chéo trong bài toán mất cân bằng

Tại mỗi cấp độ của mạng tháp đặc trưng đa độ phân giải, mô hình tách ra làm hai nhánh mạng nơ-ron tích chập nhỏ hoạt động độc lập nhằm giải quyết hai bài toán đã nêu trên. Nhánh đầu tiên làm nhiệm vụ phân loại xác suất sự hiện diện của đối tượng. Theo lý thuyết học sâu kinh điển, hàm mất mát chuẩn mực cho bài toán phân loại nhị phân này là hàm Entropy chéo [8]. Gọi p là xác suất do mạng dự đoán cho lớp nhãn thực sự có mặt, phương trình của hàm mất mát này được viết đơn giản là:

$$CE(p) = -\log(p) \quad (3.13)$$

Đặc điểm của hàm số logarit này là khi xác suất dự đoán tiến gần đến 1 có nghĩa là mạng rất tự tin và đoán đúng, giá trị sai số sẽ giảm về một con số rất nhỏ gần bằng 0. Tuy nhiên, khi áp dụng vào không gian Anchor Box của ảnh nhiệt, một bức ảnh duy nhất có thể sinh ra tới một trăm ngàn Anchor Box trải đều trên mọi cấp độ tháp nhưng số lượng Anchor Box thực sự chứa người thường chỉ đếm trên đầu ngón tay. Sự chênh lệch tỷ lệ một trên hàng chục ngàn này dẫn đến một thảm họa toán học. Mặc dù các mảng hậu cảnh trông trải như nền đất hay mái nhà rất dễ nhận diện và mỗi Anchor Box nền chỉ sinh ra một sai số cực nhỏ, nhưng khi cộng dồn hàng trăm ngàn sai số nhỏ bé này lại, chúng tạo ra một lực kéo đạo hàm khổng lồ áp đảo hoàn toàn tín hiệu học tập quý giá phát ra từ một vài Anchor Box chứa người. Kết quả là mạng nơ-ron bị mù hướng và chỉ cố gắng học cách dự đoán mọi thứ là nền để tối thiểu hóa sai số một cách máy móc.

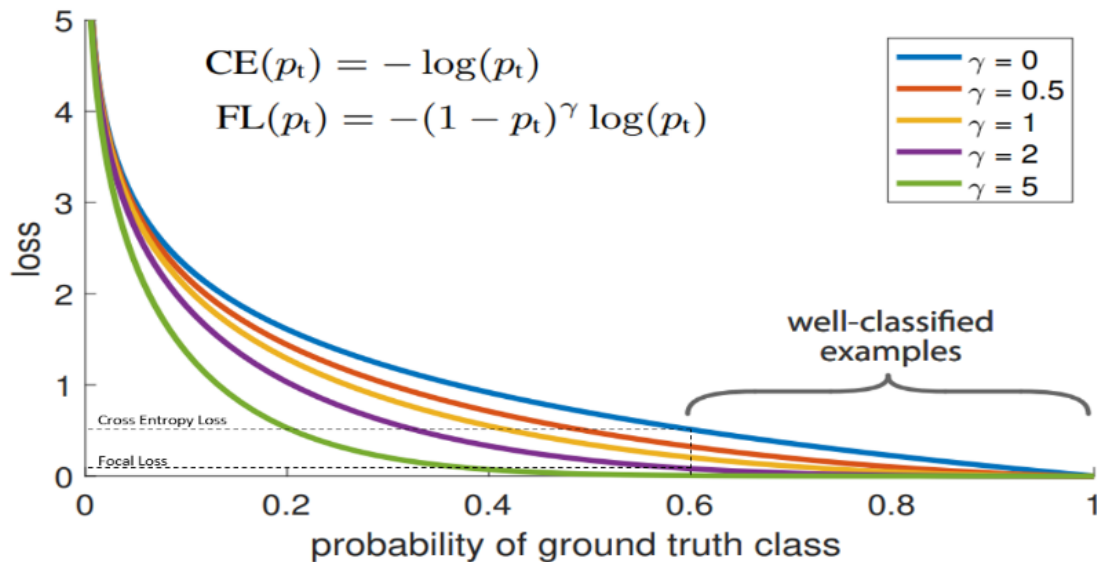
3.3.5.2 Hàm Focal Loss

Để cứu vãn tình trạng mất cân bằng cực đoan này, các nhà khoa học đã thiết kế một hàm toán học động lực mới có khả năng tự động điều chỉnh lực phạt mang tên hàm Focal Loss [12]. Ý tưởng nền tảng là thay vì trừng phạt mọi sai số một cách tuyến tính, hàm sẽ chủ động làm suy giảm trọng lượng của các mẫu hậu cảnh dễ nhận diện và dồn toàn bộ nguồn lực để sửa sai cho các mẫu đối tượng khó nhận diện hoặc đang bị dự đoán sai nghiêm trọng. Phương trình toán học đầy đủ của hàm Focal Loss được cấu thành từ hàm gốc bằng cách nhân thêm hai hệ số điều chỉnh:

$$FL = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (3.14)$$

Trong đó, α_t là tham số trọng số nhằm cân bằng tỷ lệ giữa các lớp, γ (thường đặt bằng 2) là tham số tập trung. Thành phần $(1 - p_t)^\gamma$ đóng vai trò là hệ số điều chỉnh: khi mô hình dự đoán một mẫu nền dễ với độ tự tin cao ($p_t \rightarrow 1$), hệ số này tiến về 0, triệt tiêu gần như hoàn toàn đóng góp của mẫu đó vào tổng sai số, qua đó giải quyết triệt để vấn đề mất cân bằng lớp học.

Hình 3.3 so sánh giá trị hàm loss giữa CE và FL tại cùng một điểm xác suất, từ đó ta thấy tại điểm xác suất 0.6 giá trị loss của hàm FL đã hội tụ gần 0 nhanh hơn.



Hình 3.4 - So sánh FL và CE

3.3.5.3 Hàm DIoU Loss

Trong cơ chế dự đoán Anchor Free, nhánh hồi quy đảm nhận nhiệm vụ phức tạp là dự đoán trực tiếp bốn giá trị khoảng cách từ một điểm tâm tiềm năng đến bốn cạnh của đối tượng mục tiêu. Tuy nhiên, việc sử dụng các hàm mất mát tính toán khoảng cách thông thường như Smooth L1 (hoặc L2) tồn tại một hạn chế mang tính bản chất: chúng xem xét từng điểm tọa độ một cách độc lập thay vì đánh giá bounding box như một khối hình học hoàn chỉnh.

Để cải thiện vấn đề này, hàm mất mát IoU Loss đã được ứng dụng nhờ khả năng đánh giá trực tiếp diện tích chồng lấn giữa khung dự đoán và khung thực tế [19]. Dù vậy, IoU Loss lại bộc lộ điểm yếu chí mạng khi khung dự đoán và khung thực tế hoàn toàn không giao nhau ($IoU = 0$). Tại trạng thái này, hàm mất mát trở thành hằng số ($Loss = 1$), khiến đạo hàm bằng 0 và quá trình lan truyền ngược bị tê liệt hoàn toàn, mạng nơ-ron sẽ không nhận được bất kỳ định hướng nào để tinh chỉnh tọa độ ở chu kỳ học tiếp theo.

Nhằm giải quyết triệt để rào cản này, hệ thống áp dụng hàm mất mát DIoU (Distance Intersection over Union) do Zheng và các cộng sự đề xuất [19]. Hàm DIoU kế thừa ưu điểm đánh giá hình học của IoU, đồng thời bổ sung thêm một hệ số phạt dựa trên khoảng cách không gian giữa điểm tâm của hai khung bao. Công thức hàm DIoU được định nghĩa như sau:

$$L_{DIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} \quad (3.15)$$

Trong đó, $\rho(\cdot)$ đại diện cho khoảng cách theo hình học Euclid b và b^{gt} lần lượt là tọa độ tâm của bounding box dự đoán và bounding box thực tế. Đại lượng c là chiều dài đường chéo của hộp đóng kín nhỏ nhất bao trọn cả hai khung này.

3.3.6 Lý thuyết tinh chỉnh cho ảnh nhiệt

Để ứng dụng hiệu quả kiến trúc RetinaNet từ miền ảnh màu khả kiến (RGB) sang miền ảnh nhiệt, hệ thống đòi hỏi phải điều chỉnh hai khía cạnh lý thuyết cốt lõi về dữ liệu. Đầu tiên là không gian đầu vào, trong khi ảnh RGB mang thông tin trên ba kênh màu độc lập, ảnh nhiệt thô thường chỉ tồn tại dưới dạng một kênh cường độ sáng đơn sắc. Dựa trên cơ sở lý thuyết của học chuyển giao (Transfer Learning), để tận dụng triệt để

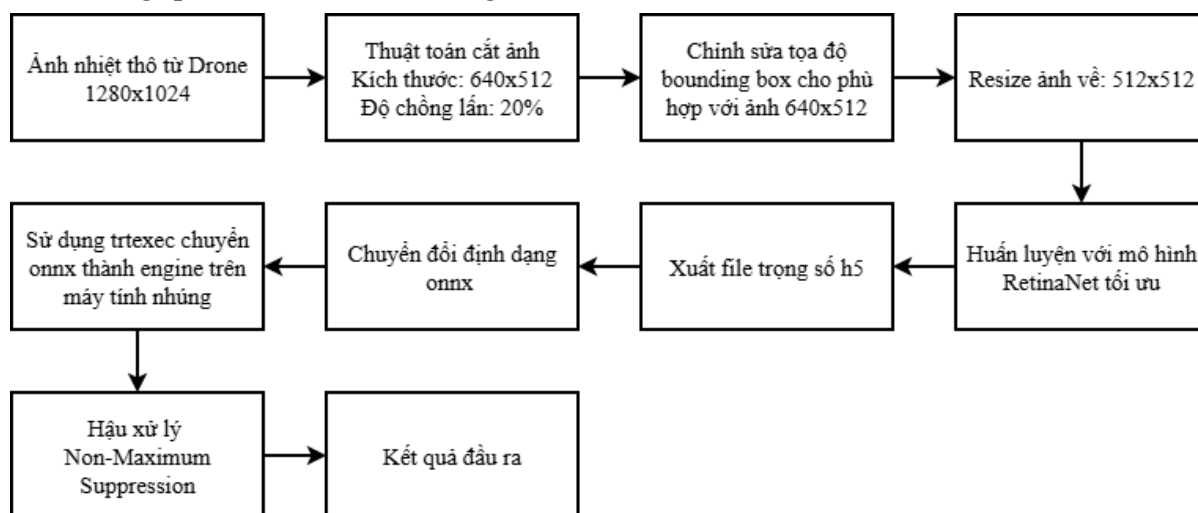
bộ trọng số đã được huấn luyện rất tốt trên các tập dữ liệu lớn (như COCO) [15], giải pháp tối ưu là sao chép kênh ảnh nhiệt lên ba lần nhằm tương thích hoàn toàn với cấu trúc đầu vào của mạng trích xuất ResNet. Thứ hai, đối với kỹ thuật tăng cường dữ liệu trong quá trình huấn luyện, do bản chất ảnh nhiệt không chứa thông tin quang phổ màu, các phép biến đổi liên quan đến màu sắc hoàn toàn không mang lại tác dụng.

CHƯƠNG 4: XÂY DỰNG MÔ HÌNH TOÁN NHẬN DẠNG NGƯỜI BẰNG ẢNH NHIỆT VÀ TRIỂN KHAI

4.1 Sơ đồ tổng quan mô hình nhận dạng

Chương 4 sẽ tập trung vào quá trình hiện thực hóa hệ thống nhận dạng đối tượng người trong thực tiễn. Việc ứng dụng ảnh nhiệt vào bài toán thị giác máy tính trên thiết bị bay không người lái đặt ra những thách thức lớn, không chỉ nằm ở khâu xử lý đặc thù của dải dữ liệu thiếu hụt thông tin màu sắc và kết cấu bề mặt, mà còn ở rào cản về tài nguyên phần cứng khi hệ thống phải duy trì tốc độ suy luận theo thời gian thực. Để giải quyết các vấn đề này, mục tiêu trọng tâm của chương là trình bày quy trình xây dựng toàn diện mô hình toán học nhận dạng, bắt đầu từ bước thiết lập phần cứng, thu thập và tiền xử lý cơ sở dữ liệu nhiệt, cho đến khâu tinh chỉnh và huấn luyện mạng nơ-ron. Đồng thời, chương cũng sẽ làm rõ phương pháp tối ưu hóa và chuyển đổi định dạng trọng số mô hình nhằm triển khai luồng suy luận độc lập trên máy tính nhúng biên Jetson Orin Nano, qua đó hoàn thiện chu trình trích xuất tọa độ mục tiêu để phục vụ công tác điều khiển thiết bị bay.

Sơ đồ tổng quát mô hình nhận dạng được mô tả như hình 4.1.



Hình 4.1 - Sơ đồ tổng quát quy trình hoạt động của thuật toán

Sơ đồ khối hệ thống thể hiện toàn bộ chu trình khép kín từ khâu thu thập, xử lý dữ liệu cho đến khi triển khai thực thi trực tiếp trên thiết bị biên. Ở giai đoạn chuẩn bị dữ liệu, ảnh nhiệt thô được thu thập từ thiết bị bay không người lái với độ phân giải cao

1280x1024 pixel. Do yêu cầu phát hiện mục tiêu nhỏ và giới hạn của mạng nơ-ron, luồng ảnh này đi qua thuật toán cắt ảnh để chia nhỏ thành các khung hình 640x512 pixel với độ chồng lấn được thiết lập ở mức 20%, đảm bảo các đối tượng ở vùng biên không bị cắt đứt. Tọa độ của các bounding box tương ứng sau đó được hệ thống nội suy và chỉnh sửa lại cho khớp với hệ quy chiếu của ảnh cắt mới, trước khi tiếp tục được resize về kích thước chuẩn 512x512. Tập dữ liệu hoàn thiện này được đưa vào quá trình học sâu với mô hình RetinaNet đã được tối ưu và kết xuất dưới định dạng file h5. Bước sang giai đoạn triển khai thực tế, file trọng số này trước tiên được chuyển đổi sang chuẩn mở ONNX để tăng tính linh hoạt. Điểm mấu chốt của hệ thống nằm ở khâu sử dụng công cụ trtexec trực tiếp trên máy tính nhúng để biên dịch file ONNX thành định dạng engine (TensorRT), qua đó tận dụng tối đa năng lực tăng tốc phần cứng của kiến trúc GPU trên thiết bị Jetson Orin Nano nhằm đạt tốc độ suy luận thời gian thực. Cuối cùng, luồng dữ liệu dự đoán từ mô hình sẽ đi qua bước hậu xử lý sử dụng thuật toán triệt tiêu phi cực đại (Non-Maximum Suppression) để lọc bỏ các dự đoán chồng chéo, từ đó đưa ra kết quả đầu ra là tọa độ duy nhất và chính xác nhất của mục tiêu nhằm phục vụ công tác điều khiển bám sát.

4.2 Hệ thống phần cứng

Để tối ưu hóa khả năng thu thập và xử lý hình ảnh nhiệt theo thời gian thực ngay trên thiết bị bay, hệ thống phần cứng của đề tài được thiết kế theo kiến trúc phân tán, bao gồm cụm thiết bị tính toán tại biên gắn trên qua và trạm điều khiển mặt đất. Việc lựa chọn thiết bị được tính toán kỹ lưỡng dựa trên yêu cầu khắt khe về tải trọng, công suất tiêu thụ và năng lực tính toán song song.

4.2.1 Máy tính nhúng Jetson Orin Nano Developer Kit

NVIDIA Jetson Orin Nano Super Developer Kit phiên bản 8GB đóng vai trò là bộ não xử lý trí tuệ nhân tạo tại biên trên thiết bị bay. Đây là thiết bị có kiến trúc GPU NVIDIA Ampere với 1024 lõi CUDA và 32 lõi Tensor, cung cấp hiệu năng lên đến 67 TOPS, cho phép thực thi các thuật toán học sâu phức tạp như RetinaNet với tốc độ suy luận thời gian thực. Máy tính này đảm nhận nhiệm vụ tiếp nhận luồng video thô từ cảm biến, thực hiện các bước tiền xử lý ảnh và chạy mô hình suy luận để xuất tín hiệu tọa độ đối tượng mục tiêu phục vụ công tác giám sát trên màn hình.



Hình 4.2 - Máy tính nhúng Jetson Orin Nano

Thông số kỹ thuật của Jetson Orin Nano Developer Kit như sau[16]:

- GPU: NVIDIA Ampere, 1024 lõi CUDA, 32 lõi Tensor.
- Hiệu năng AI: 67 TOPS.
- RAM: 8GB 128-bit LPDDR5 (Băng thông 102 GB/s).
- Hệ điều hành: Ubuntu Linux (Hỗ trợ JetPack SDK 6.2 trở lên).

4.2.2 Camera SIYI ZT6

Camera SIYI ZT6 được kết nối với Jetson Orin Nano bằng Ethernet thông qua air unit. Thiết bị này có nhiệm vụ truyền tải ảnh nhiệt sang cho máy tính nhúng. Được trang bị vi bức xạ kế VOx không làm mát với độ phân giải 640x512 pixel, hoạt động ở tần số quét 30 khung hình trên giây. Cụm camera được trang bị gimbal ba trục tích hợp, giúp duy trì sự ổn định của hình ảnh ngay cả khi thiết bị bay chịu tác động từ gió hoặc các rung động cơ học trong quá trình di chuyển. Việc lựa chọn camera này giúp hệ thống thu được hình ảnh nhiệt ổn định và sắc nét, tạo điều kiện thuận lợi cho mạng nơ-ron nhận dạng chính xác đối tượng.



Hình 4.3 – Camera SIYI ZT6

Thông số kỹ thuật của Camera SIYI ZT6 như sau:

- Cảm biến nhiệt: vi bức xạ kế VOx không làm mát (Uncooled VOx Microbolometer).
- Độ phân giải ảnh nhiệt: 640 x 512 pixel.
- Tần số quét: 30 fps.
- Ống kính (Tiêu cự / Khẩu độ): 13mm / F1.0 (Lấy nét cố định).
- Cơ chế ổn định: gimbal 3 trục (Yaw, Pitch, Roll) độ chính xác cao.
- Giao tiếp dữ liệu: ethernet, CVBS, Micro-HDMI, S.Bus, UART.

4.2.3 Tay cầm điều khiển SIYI UniRC 7 Pro

Công tác giám sát và điều khiển toàn diện hệ thống được thực hiện thông qua trạm điều khiển mặt đất SIYI UniRC 7 Pro, một thiết bị tích hợp sở hữu màn hình cảm ứng 7-inch có độ sáng cao, vận hành trên nền tảng hệ điều hành Android tùy biến. Thiết bị này không chỉ đóng vai trò là giao diện hiển thị thông tin mà còn là trung tâm điều khiển từ xa, cho phép người vận hành quan sát luồng video truyền về theo thời gian thực từ thiết bị bay không người lái và can thiệp điều khiển thủ công khi cần thiết. Về cấu trúc kết nối phần cứng trên thiết bị bay, cụm camera nhiệt kép SIYI ZT6 được kết nối trực tiếp với bộ truyền tải tín hiệu Air Unit thông qua cổng Ethernet, đảm bảo luồng video thô được truyền tải liên tục và ổn định với băng thông đủ lớn để duy trì chất lượng hình ảnh sắc nét. Air Unit đóng vai trò là cầu nối liên lạc trung tâm, được kết nối đồng bộ với mạch điều khiển bay thông qua giao thức SBUS, một giao thức truyền tin kỹ thuật số

tốc độ cao, cho phép truyền dẫn các tín hiệu điều khiển từ mạch bay đến các cơ cấu chấp hành và ngược lại một cách chính xác và trễ thấp.



Hình 4.4 – Tay cầm điều khiển SIYI UniRC 7 Pro

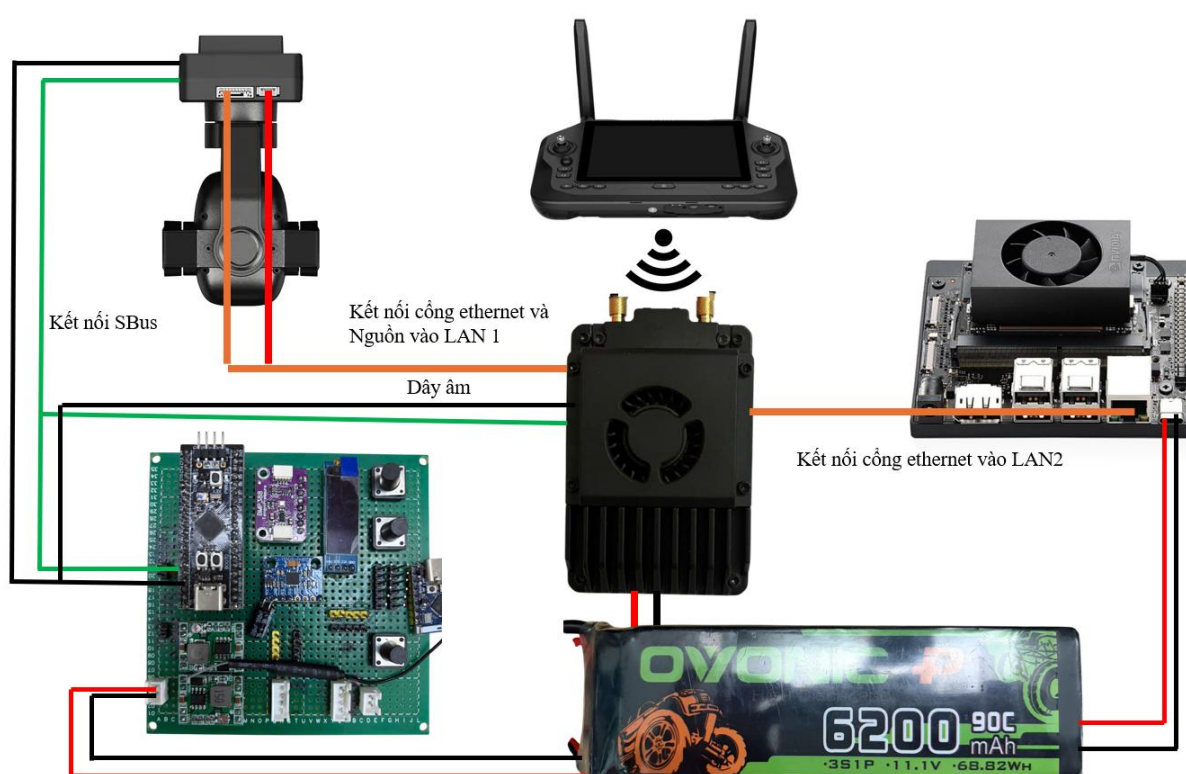
Thông số kỹ thuật tay cầm điều khiển SIYI UniRC 7 Pro như sau:

- Hệ điều hành / Vi xử lý: Android 13 / Qualcomm Snapdragon 8-core CPU.
- Tần số truyền dữ liệu: băng tần kép 2.4 GHz và 5 GHz.
- Tầm hoạt động tối đa: lên đến 40 km.
- Màn hình hiển thị: cảm ứng 7-inch HD (1920x1200), độ sáng cao 1600-nit.
- Tính năng điều khiển phụ: tích hợp sub-joystick và các nút tùy chỉnh để can thiệp khóa mục tiêu.

4.2.4 Sơ đồ kết nối phần cứng

Sơ đồ kết nối tổng thể của hệ thống được mô tả tại Hình 4.5, thể hiện sự phối hợp đồng bộ và phân bổ luồng tín hiệu giữa các thành phần phần cứng quan trọng trên thiết bị bay

không người lái. Hệ thống được thiết kế dựa trên kiến trúc tập trung vào thiết bị truyền dẫn trung tâm là Air Unit, đi kèm với trạm điều khiển mặt đất SIYI UniRC 7 Pro, nhằm tối ưu hóa quá trình truyền tải dữ liệu hình ảnh và tín hiệu điều khiển. Camera nhiệt SIYI ZT6 đóng vai trò là nguồn thu dữ liệu hình ảnh chính, thực hiện truyền luồng video thô về Air Unit thông qua cáp kết nối Ethernet chuẩn cắm vào cổng LAN 1, được đại diện bằng đường dây màu cam trên sơ đồ. Để thực hiện các tác vụ xử lý trí tuệ nhân tạo tại biên, máy tính nhúng NVIDIA Jetson Orin Nano được kết nối vào cổng LAN 2 của Air Unit, cho phép thiết bị nhận luồng dữ liệu hình ảnh ổn định từ camera mà không làm ảnh hưởng đến băng thông truyền dẫn tín hiệu điều khiển từ xa.



Hình 4.5 – Sơ đồ kết nối hệ thống

Bên cạnh luồng dữ liệu hình ảnh, hệ thống điều khiển được thiết lập thông qua kết nối giữa camera nhiệt, Air Unit và mạch điều khiển bay trung tâm sử dụng giao thức truyền tin kỹ thuật số SBUS. Việc cấu hình kết nối này có vai trò then chốt trong việc cho phép người vận hành sử dụng tay cầm điều khiển SIYI UniRC 7 Pro để can thiệp trực tiếp vào các góc quay của gimbal camera thông qua các kênh tín hiệu đã được lập trình sẵn trên mạch bay. Giao thức SBUS đảm bảo độ trễ thấp và độ tin cậy cao, cho phép các thao tác điều khiển thủ công được phản hồi tức thời lên cơ cấu chấp hành gimbal, giúp việc bám bắt đối tượng trong môi trường thực địa trở nên chủ động và linh hoạt hơn.

Năng lượng vận hành cho toàn bộ hệ thống, từ máy tính nhúng Jetson Orin Nano, hệ thống truyền dẫn video Air Unit cho đến cụm camera nhiệt và mạch bay, đều được cung cấp bởi một bộ pin Lithium 3S có thông số kỹ thuật 11.1V, 90C với dung lượng năng lượng tổng đạt 68.82Wh. Sự tích hợp chặt chẽ giữa các khối kết nối, từ luồng dữ liệu hình ảnh, các kênh điều khiển đến hệ thống phân phối nguồn điện ổn định, cho phép thiết bị vận hành tự chủ hoàn toàn trên không trung, đồng thời giảm thiểu tối đa độ trễ trong quá trình xử lý và phản hồi dữ liệu trong các tình huống cứu nạn khẩn cấp.

4.2.5 Thu thập dữ liệu

Để đảm bảo tính đa dạng và nâng cao khả năng tổng quát hóa của mạng nơ-ron, cơ sở dữ liệu huấn luyện được xây dựng dựa trên chiến lược kết hợp giữa nguồn ảnh tự ghi nhận thực tế và các bộ dữ liệu tiêu chuẩn có sẵn. Đối với nguồn dữ liệu thực tiễn, quá trình thu thập được tiến hành trực tiếp tại tầng 11, Tòa nhà I thuộc Trường Đại học Khoa học Tự nhiên (cơ sở Nguyễn Văn Cừ), vị trí này được lựa chọn có chủ đích nhằm mô phỏng chính xác góc máy chúi xuống từ trên không (top-down view) tương đương với cao độ hoạt động của thiết bị bay, đồng thời sử dụng trực tiếp cụm camera nhiệt kép SIYI ZT6 để ghi hình nhằm đảm bảo sự đồng nhất về đặc tả phần cứng. Nhằm bổ sung số lượng mẫu và làm phong phú sự biến thiên của hậu cảnh, hệ thống tiếp tục tích hợp thêm các tập dữ liệu ảnh nhiệt uy tín từ các nền tảng mở như Roboflow [17], WiSARD [26] và NII-CU [27]. Tổng thể kịch bản dữ liệu bao trùm nhiều loại hình môi trường khác nhau, trải dài từ các khu vực đô thị phức tạp có nhiều nguồn nhiễu nhiệt (bê tông, mái tôn) đến các địa hình cánh đồng trống trải. Toàn bộ cơ sở dữ liệu được thu thập trong điều kiện thời tiết bình thường, tập trung tối đa vào việc tái tạo góc nhìn từ trên cao, qua đó cung cấp một kho mẫu chuẩn xác giúp mô hình RetinaNet tập trung học hỏi các đặc trưng hình học của đối tượng người đi lạc thông qua dải bức xạ nhiệt.

4.2.6 Tiền xử lý dữ liệu

Quá trình tiền xử lý dữ liệu đóng vai trò quyết định đến độ chính xác và khả năng hội tụ của mô hình mạng nơ-ron. Đối với các tập dữ liệu khai thác từ nền tảng mở, luận văn đã tiến hành rà soát, loại bỏ các khung hình lỗi và tận dụng trực tiếp bộ nhãn đã được chuẩn hóa từ trước. Tuy nhiên, để xử lý sự chênh lệch về độ phân giải và tối ưu hóa khả năng phát hiện các đối tượng người có kích thước rất nhỏ từ góc nhìn thiết bị bay không

người lái, hệ thống đã áp dụng kỹ thuật cắt ảnh [18] . Cụ thể, các khung hình gốc được chia cắt thành các vùng ảnh nhỏ hơn với kích thước 640x512 pixel, đi kèm với đó, tọa độ của các bounding box gốc cũng được thuật toán toán học nội suy và tinh chỉnh lại một cách chuẩn xác để khớp hoàn toàn với không gian tọa độ của từng bức ảnh cắt tương ứng. Sau khi hoàn thiện việc ánh xạ tọa độ nhãn, toàn bộ cơ sở dữ liệu tiếp tục trải qua phép biến đổi hình học để thu phóng đồng loạt về kích thước chuẩn 512x512 pixel nhằm đáp ứng nghiêm ngặt yêu cầu cấu trúc đầu vào của mạng trích xuất RetinaNet. Sau đó, dữ liệu sẽ được chia theo tỉ lệ 70% train, 20% validation và 10% test (20780, 5937, 2969) ảnh. Trải qua chuỗi quy trình làm sạch, chuẩn hóa tọa độ và đồng bộ kích thước kể trên, bộ dữ liệu đã đạt được tính nhất quán cao nhất và hoàn toàn sẵn sàng để đưa vào pha huấn luyện chuyên sâu.

4.3 Xây dựng và tối ưu thuật toán nhận dạng

4.3.1 Nền tảng phần cứng để huấn luyện mô hình

Quá trình xây dựng và huấn luyện mô hình mạng nơ-ron sâu đòi hỏi khối lượng tính toán ma trận khổng lồ. Do đó, thay vì thực thi trực tiếp trên máy tính nhúng hạn chế tài nguyên, toàn bộ quá trình này được tiến hành ngoại tuyến trên máy tính trạm chuyên dụng nhằm rút ngắn thời gian phát triển. Cụ thể, hệ thống huấn luyện của đề tài được triển khai trên nền tảng máy tính ROG Strix G16 phiên bản 2023, vận hành trên hệ điều hành mã nguồn mở Ubuntu. Cấu hình phần cứng bao gồm bộ vi xử lý trung tâm Intel Core i7-13650HX, bộ nhớ trong 16GB, và bộ xử lý đồ họa NVIDIA GeForce RTX 4060 với 8GB VRAM. Môi trường phát triển được thiết lập dựa trên ngôn ngữ lập trình Python, kết hợp chặt chẽ với nền tảng tính toán song song CUDA và thư viện lõi cuDNN nhằm khai thác tối đa sức mạnh của bộ xử lý đồ họa.

4.3.2 Thư viện TensorFlow và Keras

Để hiện thực hóa kiến trúc toán học của mạng RetinaNet, đề tài lựa chọn sử dụng nền tảng học máy TensorFlow kết hợp với giao diện lập trình cấp cao Keras. Sự lựa chọn này xuất phát từ bốn ưu điểm kỹ thuật vượt trội. Thứ nhất, cơ chế tự động tính đạo hàm của hệ thống giúp khai thác tối đa sức mạnh phần cứng, đẩy nhanh tốc độ hội tụ. Thứ hai, giao diện lập trình linh hoạt cho phép khai báo mạng nơ-ron theo mô hình hướng

đối tượng, giúp chia nhỏ các thành phần phức tạp thành các lớp độc lập để dễ dàng quản lý.

4.3.2.1 Nhóm cấu trúc mô hình

Nhóm thư viện này đóng vai trò nền tảng trong việc kiến tạo kiến trúc vật lý của mạng nơ-ron. Các hàm và lớp đối tượng thuộc nhóm này được sử dụng trực tiếp để định nghĩa cấu trúc đồ thị tính toán, cấu thành nên mạng trích xuất đặc trưng và mạng FPN.

Thư viện nhóm cấu trúc mô hình gồm các hàm như sau:

- `tf.keras.Model` và `tf.keras.layers` là các lớp cơ sở dùng để định nghĩa và khởi tạo các thành phần cấu trúc cốt lõi của mạng nơ-ron.
- `tf.keras.layers.Conv2D` là lớp thực hiện phép tính tích chập, đóng vai trò chủ đạo trong việc trích xuất đặc trưng hình học từ dữ liệu ảnh.
- `tf.keras.layers.BatchNormalization` là lớp thực thi quá trình chuẩn hóa phân phối dữ liệu sau lớp tích chập, giúp ổn định đạo hàm và gia tăng tốc độ hội tụ.
- `tf.keras.layers.ReLU` là hàm kích hoạt phi tuyến tính, hỗ trợ mạng nơ-ron phân tách các đường ranh giới quyết định phức tạp.
- `tf.keras.layers.Add` thực hiện phép cộng ma trận trực tiếp, ứng dụng để tạo các kết nối tắt trong kiến trúc khối thặng dư.
- `tf.keras.layers.UpSampling2D` là lớp phóng to kích thước bản đồ đặc trưng, ứng dụng chủ yếu trong mạng tháp đặc trưng nhằm hòa trộn thông tin đa độ phân giải.

4.3.2.2 Nhóm tính toán toán học và xử lý dữ liệu đa chiều

Để hiện thực hóa cơ chế dự đoán Anchor Free và xử lý luồng dữ liệu, hệ thống yêu cầu các công cụ can thiệp sâu vào cấu trúc ma trận. Nhóm hàm này cung cấp các phương thức thao tác hình học, định hình lại không gian dữ liệu và thực thi các phép toán ma

trận phức tạp, phục vụ trực tiếp cho việc tính toán khoảng cách giới hạn và ánh xạ tọa độ đối tượng.

Thư viện nhóm tính toán toán học và xử lý dữ liệu đa chiều gồm các hàm sau:

- `tf.concat` dùng để ghép nối các ma trận đa chiều lại với nhau theo một trục không gian xác định.
- `tf.reshape`, `tf.expand_dims`, `tf.squeeze` là các hàm biến đổi hình học dùng để tái cấu trúc hình dáng và số chiều của ma trận nhằm chuẩn bị cho các phép tính sai số.
- `tf.split` dùng để tách một ma trận lớn thành các ma trận thành phần độc lập (ví dụ: tách các tọa độ điểm ảnh của bounding box).
- `tf.where`, `tf.equal`, `tf.greater` là các phép toán logic nhằm tạo mặt nạ phân tích, phục vụ việc phân loại các Anchor Box thành nhóm đối tượng hoặc nhóm nền.
- `tf.math.exp`, `tf.square`, `tf.maximum`, `tf.minimum` là các phép tính toán học chuyên biệt dùng để tính toán diện tích, khoảng cách tâm và độ lệch đường chéo trong thuật toán tối ưu hóa tọa độ.

4.3.2.3 Nhóm tính toán hàm mất mát

Hàm mất mát là thước đo cốt lõi để mạng nơ-ron tự đánh giá và hiệu chỉnh trọng số thông qua thuật toán lan truyền ngược. Nhóm thư viện này cung cấp khung nền (framework) để tùy biến các hàm sai số chuyên biệt, đồng thời hỗ trợ các phép mã hóa nhãn nhằm chuẩn bị định dạng đầu ra cho việc so sánh sai số.

Thư viện nhóm tính toán các hàm loss dùng các hàm như sau:

- `tf.losses.Loss` là lớp trừu tượng định hướng đối tượng, cho phép kế thừa để xây dựng các thuật toán tính toán sai số tùy chỉnh.
- `tf.one_hot` dùng cho chuyển đổi dữ liệu nhãn đối tượng dạng số nguyên sang định dạng ma trận mã hóa phân loại.

4.3.2.4 Nhóm luồng xử lý dữ liệu tự động

Việc nạp dữ liệu ảnh nhiệt vào GPU đòi hỏi một pipeline xử lý tối ưu để tránh hiện tượng nghẽn cổ chai. Nhóm thư viện này được sử dụng nhằm tự động hóa toàn bộ vòng đời của dữ liệu đầu vào: từ khâu đọc dữ liệu thô, thực thi phép biến đổi cho đến việc đóng gói thành các tensor đa chiều.

Thư viện cho nhóm luồng xử lý dữ liệu tự động dùng các hàm sau:

- `tf.data.Dataset` là cấu trúc dữ liệu chuyên biệt dùng để quản lý toàn diện vòng đời và luồng chảy của dữ liệu đầu vào.
- `map` là hàm ánh xạ thực thi các phép biến đổi dữ liệu song song bằng cơ chế đa luồng, kết hợp tham số tự động điều phối để tối ưu tài nguyên xử lý trung tâm.
- `padded_batch` là hàm gom cụm dữ liệu tự động tích hợp tính năng đệm kích thước, đảm bảo sự đồng nhất về mặt ma trận cho các gói dữ liệu.
- `prefetch` là kỹ thuật nạp trước dữ liệu lên bộ nhớ hệ thống, triệt tiêu thời gian gián đoạn chờ đợi của bộ xử lý đồ họa.

4.3.2.5 Nhóm tối ưu hóa và điều khiển quy trình học

Để đảm bảo quá trình đào tạo mạng nơ-ron diễn ra ổn định, hội tụ nhanh và giới hạn trong tài nguyên phần cứng cho phép, hệ thống tích hợp nhóm các thuật toán tối ưu và cơ chế giám sát tự động (Callbacks). Nhóm thư viện này đóng vai trò như một bộ điều khiển trung tâm, tự động can thiệp vào tiến trình học và cấp phát bộ nhớ.

Thư viện cho nhóm tối ưu hóa và điều khiển quy trình học dùng các hàm sau:

- `tf.keras.optimizers.Adam` là bộ thuật toán tối ưu hóa cập nhật trọng số, được tinh chỉnh thêm tham số chặn ngưỡng nhằm ngăn ngừa hiện tượng bùng trực đạo hàm.
- `mixed_precision.Policy` giúp cấu hình hệ thống để thực thi tính toán với độ chính xác hỗn hợp dựa trên định dạng số thực 16-bit, giúp tối ưu hóa dung lượng bộ nhớ.
- `tf.keras.callbacks.ModelCheckpoint` là cơ chế gọi lại tự động nhằm giám sát độ chính xác kiểm định và lưu trữ phiên bản trọng số hoàn thiện nhất.

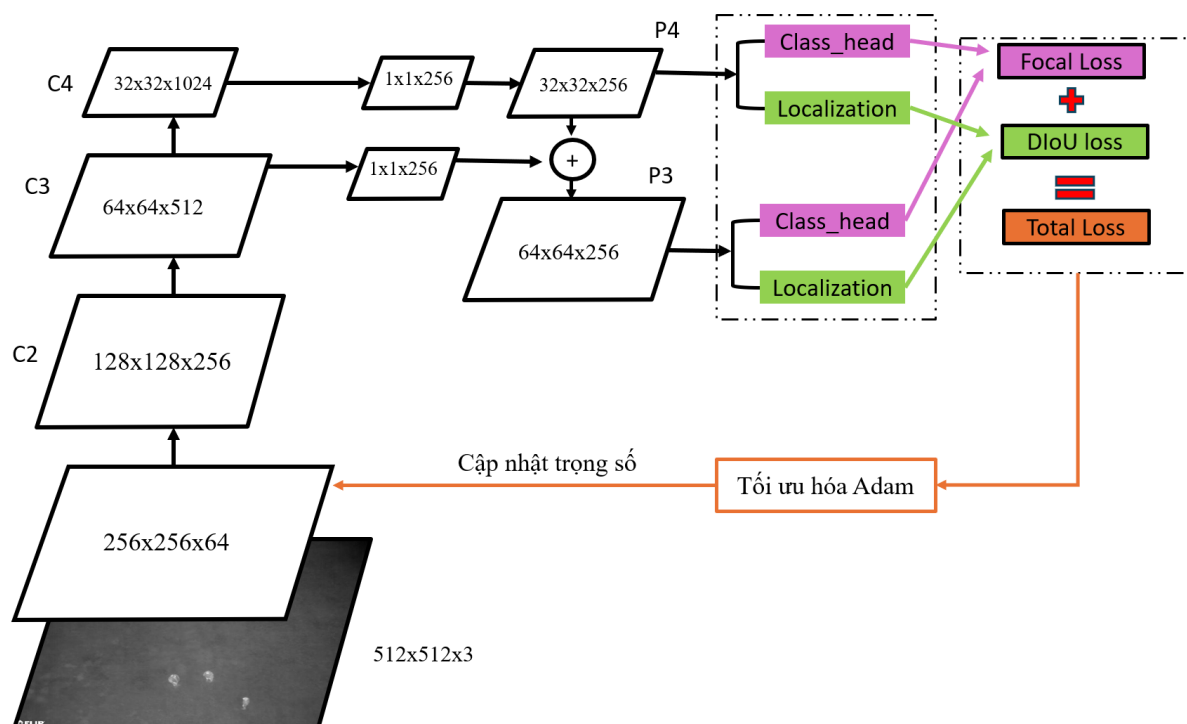
- `tf.keras.callbacks.TerminateOnNaN` là hàm bảo vệ hệ thống, tự động chấm dứt quá trình huấn luyện nếu sai số toán học rơi vào trạng thái vô định.
- `tf.keras.callbacks.ReduceLROnPlateau` là cơ chế tự động điều chỉnh và làm suy giảm tốc độ học khi đồ thị sai số có dấu hiệu hội tụ đi ngang.
- `tf.keras.callbacks.EarlyStopping` là hàm giám sát thông minh nhằm dừng sớm chu trình học nếu kết quả không cải thiện, triệt tiêu rủi ro xảy ra hiện tượng quá khớp.
- `tf.config.experimental.set_memory_growth` chỉ lệnh cấu hình hệ thống yêu cầu quản lý và cấp phát linh hoạt tài nguyên bộ nhớ đồ họa dựa trên nhu cầu thực tế.

4.3.3 Xây dựng mô hình toán học nhận dạng

Để đáp ứng yêu cầu khắt khe về tốc độ xử lý thời gian thực trên thiết bị máy tính nhúng và giải quyết triệt để bài toán nhận dạng người bằng ảnh nhiệt cho thiết bị bay không người lái quadcopter hoạt động ở độ cao từ 50 mét trở lên so với mặt đất, luận văn đã chủ động thiết kế và xây dựng một kiến trúc mạng nhận dạng hoàn toàn tùy chỉnh. Điểm cải tiến cốt lõi nằm ở việc thay thế phương pháp dự đoán Anchor Box truyền thống bằng cơ chế Anchor Free, kết hợp cùng các hàm mất mát tối ưu hóa hình học. Đặc biệt, luận văn đã tự thiết kế mạng trích xuất đặc trưng ResNet-40 riêng biệt, việc chủ đích tính toán và tinh giản số lượng khối thịnh dư giúp mô hình tránh được hiện tượng học vẹt đối với dải dữ liệu ảnh nhiệt vốn thiếu hụt thông tin kết cấu, đồng thời rút ngắn đáng kể thời gian lan truyền tiến.

Như được minh họa chi tiết tại Hình 4.6, luồng dữ liệu bắt đầu với ảnh nhiệt đầu vào kích thước 512x512x3. Khi đi qua mạng trích xuất nông C2, dữ liệu được biến đổi thành bản đồ đặc trưng có kích thước 128x128x256 nhằm tập trung thu thập các chi tiết không gian sắc nét như ranh giới và đường viền của luồng nhiệt. Đi sâu hơn, các tầng C3 (64x64x512) và C4 (32x32x1024) được thiết kế để bóc tách các đặc trưng ngữ nghĩa phức tạp về hình dáng và mức độ bức xạ của con người. Tiếp nối mạng Backbone, luận văn đã tự xây dựng một mạng tháp đặc trưng FPN tối giản. Dựa trên những phân tích thực nghiệm về kích thước đối tượng người đi lạc cực kỳ nhỏ từ góc nhìn thiết bị bay không người lái quadcopter trên 50 mét, luận văn quyết định loại bỏ hoàn toàn các bản đồ đặc trưng có độ phân giải cực thấp và chỉ duy trì duy nhất hai tầng P3 và P4. Tại mạng tháp này, đặc trưng từ C4 được giảm kênh qua tích chập 1x1 xuống còn

32x32x256 để tạo thành P4. Tầng P4 này tiếp tục được phóng to và cộng gộp trực tiếp với đặc trưng từ C3 để tạo ra bản đồ P3 kích thước 64x64x256. Hai tầng P3 và P4 sau đó trở thành đầu vào độc lập cho hai nhánh dự đoán (Class_head và Localization). Toàn bộ chuỗi thiết kế kiến trúc mang tính chuyên biệt này đã giúp hệ thống tiết kiệm một lượng lớn các phép toán tích chập, mang lại hiệu năng xử lý vượt trội mà không làm suy giảm năng lực nhận dạng.



Hình 4.6 – Sơ đồ tổng quát kiến trúc mạng nhận dạng người bằng ảnh

Để đảm bảo quá trình tối ưu hóa trọng số diễn ra hiệu quả trong cơ chế dự đoán Anchor Free, hệ thống được xây dựng để xử lý độc lập sai số thông qua hai hàm Focal loss và DIoU loss nhằm giải quyết triệt để những thách thức đặc thù của dữ liệu ảnh nhiệt. Ở nhánh phân loại (Class_head), do phần lớn các điểm ảnh trên bản đồ đặc trưng đều thuộc về nhóm hậu cảnh, mô hình phải đối mặt với sự mất cân bằng lớp cực đoan. Việc lựa chọn hàm Focal Loss nhằm mục đích tự động suy giảm mức độ đóng góp sai số của các mẫu nền dễ nhận diện, từ đó ép buộc mạng nơ-ron phải tập trung dồn nỗ lực cập nhật đạo hàm vào các đối tượng người có bức xạ nhiệt mờ nhạt vốn rất khó phát hiện từ trên cao. Song song đó, tại nhánh hồi quy (Localization), việc dự đoán trực tiếp khoảng cách từ điểm tâm đến bốn cạnh giới hạn đòi hỏi một thước đo có khả năng đánh giá tổng thể hình học định vị thay vì chỉ quan tâm đến diện tích. Do đó, luận văn sử

dùng hàm sai số khoảng cách giao thức DIoU (Distance Intersection over Union) nhằm khắc phục điểm yếu của hàm Smooth L1 truyền thống. Bằng cách bổ sung thêm một đại lượng phạt trực tiếp dựa trên khoảng cách giữa điểm tâm của khung dự đoán và khung nhãn thực tế, DIoU giúp các đường viền tọa độ hội tụ nhanh và chính xác hơn ngay cả trong trường hợp hai khung hình hoàn toàn không có sự chồng lấn.

Cuối cùng, để hệ thống có thể tối ưu hóa đồng thời cả hai nhiệm vụ phân loại và định vị, tổng sai số (Total Loss) của toàn mạng được tính bằng tổng tuyến tính của hai hàm mất mát thành phần được tính bằng công thức sau:

$$L_{Total} = L_{Focal} + L_{DIOU} \quad (4.1)$$

Dựa trên tín hiệu tổng sai số này, thuật toán tối ưu hóa Adam được đề cập trên hình 4.6 sẽ tính toán đạo hàm và tiến hành truyền ngược (backpropagation) để cập nhật liên tục bộ trọng số cho toàn bộ đồ thị mạng, giúp mô hình tiệm cận đến điểm hội tụ lý tưởng.

Mạng con dự đoán và cơ chế Anchor Free [14]: đây là cải tiến cốt lõi nhất của hệ thống. Thay vì rải hàng nghìn Anchor Box mặc định lên bản đồ đặc trưng và tính toán độ lệch, kiến trúc Anchor Free xem mỗi điểm ảnh trên bản đồ đặc trưng P3 và P4 là một tâm dự đoán tiềm năng. Nhánh phân loại sẽ dự đoán xem điểm ảnh đó có rơi vào vùng chứa đối tượng người hay không. Nhánh hồi quy sẽ trực tiếp dự đoán khoảng cách từ điểm ảnh tâm đó đến bốn cạnh giới hạn (trái, trên, phải, dưới) của đối tượng. Cơ chế này loại bỏ hoàn toàn các siêu tham số phức tạp liên quan đến việc thiết kế Anchor Box, giúp mạng nhẹ hơn và suy luận sắc bén hơn.

Để quá trình tối ưu hóa trọng số diễn ra hiệu quả, hệ thống dựa trên phương trình ánh xạ tọa độ của Anchor Free và xử lý độc lập sai số bằng hai hàm mất mát tiên tiến.

4.3.4 Thiết lập tham số và quá trình huấn luyện

Trong quá trình đào tạo kiến trúc Anchor Free cho bài toán phát hiện đối tượng trên ảnh nhiệt, hệ thống siêu tham số được tinh chỉnh và thiết lập một cách cẩn trọng nhằm khắc phục triệt để các hạn chế vật lý của dữ liệu.

4.3.4.1 Kỹ thuật thiết lập tham số cơ bản

Khác với phương pháp học chuyển giao (Transfer Learning) thông thường, mạng nơ-ron được cấu hình để thực thi quá trình huấn luyện hoàn toàn từ đầu mà không phụ thuộc vào bất kỳ bộ trọng số nào được đào tạo trước đó. Tại mạng trích xuất đặc trưng và mạng tháp đặc trưng, hệ thống sử dụng thuật toán phân phối đồng đều Glorot để khởi tạo trọng số một cách ngẫu nhiên, kỹ thuật này đóng vai trò quan trọng trong việc bảo toàn tín hiệu và phương sai của các vật sáng nhiệt không bị biến dạng khi lan truyền qua các tầng sâu. Đối với nhánh mạng dự đoán, để giải quyết sự mất cân bằng dữ liệu cực đoan giữa lượng lớn điểm ảnh thuộc lớp nền và số lượng rất nhỏ các điểm ảnh thuộc lớp đối tượng, thuật toán áp dụng kỹ thuật khởi tạo xác suất tiên nghiệm với hệ số 0.01 cho lớp phân loại, kết hợp cùng phân phối chuẩn ngẫu nhiên cho ma trận nhân tích chập. Sự can thiệp toán học này ngăn chặn việc sai số tập trung bị cộng dồn thành một giá trị khổng lồ ở chu kỳ đầu tiên, qua đó bảo vệ mạng khỏi trạng thái vô định.

Về mặt dữ liệu học, ảnh nhiệt sau khi được cắt nhỏ sẽ được nội suy về độ phân giải chuẩn 512x512 điểm ảnh và phân nhóm thành các lô dữ liệu có kích thước bằng 8. Kích thước lô được mở rộng nhằm tận dụng tối đa dung lượng bộ nhớ của bộ xử lý đồ họa, mang lại sự ổn định và mượt mà trong quá trình ước lượng đạo hàm cập nhật. Vòng đời huấn luyện được đặt giới hạn trần là 200 chu kỳ, tuy nhiên, hệ thống được bảo vệ bởi một cơ chế dừng sớm thông minh [20]: nếu sai số trên tập kiểm định không cho thấy dấu hiệu cải thiện sau năm chu kỳ liên tiếp, quá trình học sẽ tự động chấm dứt nhằm triệt tiêu rủi ro xảy ra hiện tượng quá khớp. Tốc độ học khởi điểm của hệ thống được chỉ định ở mức 10^{-4} .

4.3.4.2 Kỹ thuật tối ưu hóa sự hội tụ trên ảnh nhiệt

Đặc thù của bức xạ hồng ngoại là sự vắng bóng hoàn toàn của thông tin kết cấu bề mặt và quang phổ màu, mục tiêu thường chỉ hiện diện dưới dạng các đốm sáng nhạt nhòa, dễ bị hòa lẫn vào môi trường nền xung quanh. Sự thiếu hụt đặc trưng này khiến không gian hàm mất mát trở nên vô cùng gồ ghề và chứa đựng nhiều điểm cực tiểu cục bộ. Để giải quyết vấn đề, hệ thống loại bỏ phương pháp suy giảm đạo hàm thông thường và áp dụng thuật toán tối ưu hóa Adam [21]. Nhờ cơ chế tự động điều chỉnh tốc độ học độc lập cho từng trọng số dựa trên động lượng bậc nhất và đạo hàm bậc hai, mô hình có

năng lực tự bứt phá qua các vùng nhiều nhiệt và định hướng đường đi thẳng đến điểm cực tiểu toàn cục.

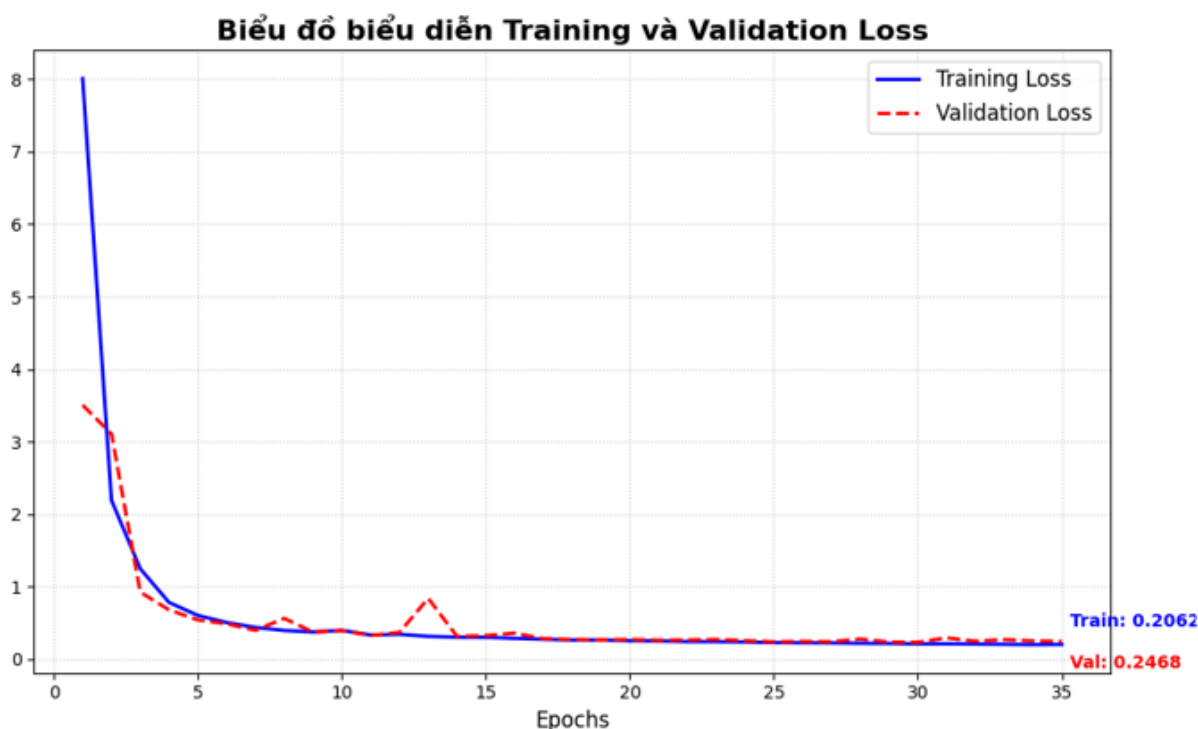
Song song đó, việc ứng dụng hàm sai số khoảng cách giao thức trong cơ chế Anchor Free dễ gây ra sự bùng nổ giá trị đạo hàm trong quá trình lan truyền ngược, nhất là khi mạng dự đoán chệch hướng hoàn toàn một mục tiêu có kích thước nhỏ. Để đảm bảo an toàn, bộ tối ưu hóa Adam được cấu hình thêm thông số cắt xén đạo hàm với ngưỡng cố định là 10.0. Kỹ thuật đóng vai trò như một bộ lọc tín hiệu, tự động tước bỏ các giá trị cập nhật vượt ngưỡng, giữ cho biên độ học luôn nằm trong tầm kiểm soát.

Cuối cùng, nhằm hỗ trợ mạng nơ-ron đạt được độ sắc bén cao nhất, một kỹ thuật lập lịch tự động suy giảm tốc độ học được lồng ghép vào hệ thống. Mặc dù tốc độ học ban đầu là đủ lớn để thuật toán tiến những bước dài, nhưng khi đường cong sai số chững lại trong ba chu kỳ liên tiếp, tốc độ này sẽ tự động giảm đi một nửa và giới hạn chốt chặn ở mức 10^{-7} đặc tính giảm tốc này ép buộc mô hình phải dò dẫm bằng những bước cập nhật vi mô ở giai đoạn cuối, qua đó hội tụ chính xác vào tâm của hình chữ nhật bao quanh vệt nhiệt mà không gây ra hiện tượng vượt đích.

4.3.5 Đánh giá kết quả huấn luyện mô hình

Sau khi hoàn tất quá trình huấn luyện theo các thông số đã thiết lập, việc đánh giá độ hội tụ và hiệu năng của mô hình là bước bắt buộc để xác nhận tính khả thi trước khi tiến hành chuyển đổi sang thiết bị phân cứng. Hai yếu tố cốt lõi được xem xét là trạng thái của hàm mất mát và các chỉ số độ chính xác trung bình (mAP).

Đồ thị hàm mất mát (tổng sai số phân loại Focal Loss và sai số tọa độ DIoU) là công cụ trực quan nhất để phản ánh quá trình học tập của mạng nơ-ron qua từng chu kỳ. Dựa trên biểu đồ biểu diễn Training và Validation Loss, đồ thị cho thấy sự sụt giảm rất mạnh và đồng pha của cả hai đường sai số trong 5 chu kỳ đầu tiên, minh chứng cho việc kỹ thuật khởi tạo trọng số và thuật toán tối ưu hóa Adam đã điều hướng đạo hàm chính xác.



Hình 4.7 - Biểu đồ hàm loss

Bắt đầu từ chu kỳ thứ 10 trở đi, tốc độ giảm của hàm mất mát bắt đầu chững lại và tiến vào trạng thái bão hòa. Đặc biệt, tại thời điểm chu kỳ thứ 35, sai số trên tập huấn luyện đạt mức 0.2062, trong khi sai số trên tập kiểm định cũng duy trì ở mức rất thấp là 0.2468. Quan trọng nhất, khoảng cách giữa hai đường đồ thị này là vô cùng hẹp và đường Validation Loss tuyệt đối không có dấu hiệu ngóc đầu đi lên ở cuối chu trình. Hiện tượng đồ thị đi ngang và bám sát nhau này là minh chứng toán học rõ ràng nhất khẳng định mô hình đã hội tụ hoàn toàn tại điểm cực tiểu toàn cục, không hề xảy ra hiện tượng Overfitting hay Underfitting.

4.4 Chuyển đổi và triển khai mô hình trên máy tính nhúng

4.4.1 Xuất và chuyển đổi định dạng mô hình

Quá trình chuyển đổi định dạng mô hình được thực hiện qua hai giai đoạn chính nhằm tối ưu hóa tính tương thích và hiệu suất phần cứng:

Giai đoạn 1 chuyển đổi từ định dạng trọng số gốc sang định dạng trung gian ONNX, mô hình sau khi hoàn tất quá trình huấn luyện được lưu trữ dưới định dạng file trọng số tiêu chuẩn (h5). Tuy nhiên, để tách biệt mô hình khỏi sự phụ thuộc vào môi trường

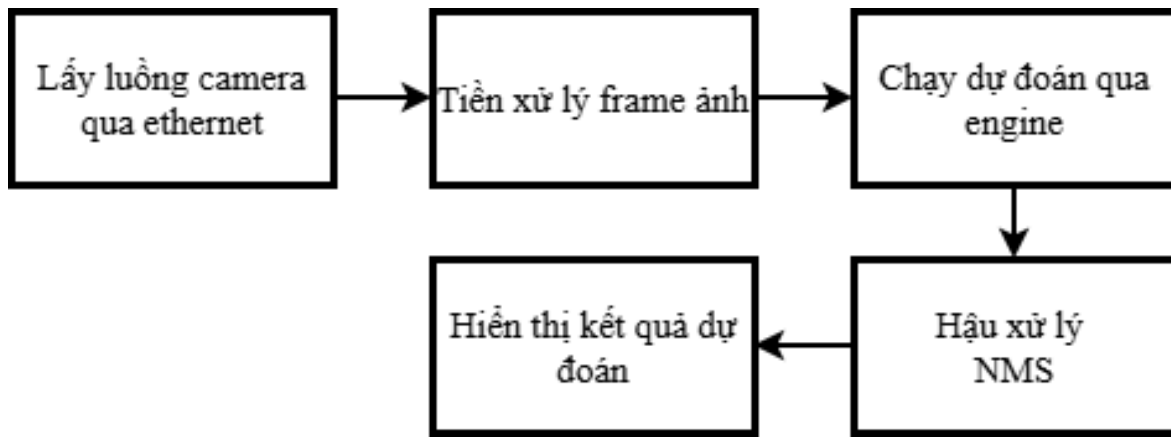
framework huấn luyện ban đầu và tạo tiền đề cho việc tăng tốc phần cứng, file h5 được xuất và chuyển đổi sang định dạng ONNX (Open Neural Network Exchange). Quá trình này được thực hiện thông qua một kịch bản lập trình Python. ONNX đóng vai trò là một định dạng biểu diễn trung gian mã nguồn mở, giúp chuẩn hóa cấu trúc mạng nơ-ron và cho phép mô hình dễ dàng tương thích với nhiều bộ công cụ nội suy (inference) khác nhau.

Giai đoạn 2 tối ưu hóa mô hình với TensorRT trên thiết bị nhúng. Sau khi thu được file mô hình ở định dạng ONNX, bước tiếp theo là biên dịch và tối ưu hóa mô hình để khai thác tối đa năng lực tính toán của kiến trúc GPU trên máy tính nhúng Jetson Orin Nano. Quá trình này được thực hiện bằng cách sử dụng công cụ dòng lệnh trtexec thuộc bộ thư viện TensorRT do NVIDIA phát triển [22].

Công cụ trtexec sẽ phân tích đồ thị tính toán của file ONNX và thực hiện biên dịch trực tiếp trên môi trường của Jetson Orin Nano để tạo ra file engine (TensorRT Engine). Trong quá trình chuyển đổi này, TensorRT áp dụng nhiều kỹ thuật tối ưu hóa phần cứng chuyên sâu như: gộp các lớp mạng, tối ưu hóa đường dẫn thực thi hạt nhân, và hiệu chỉnh độ chính xác của trọng số. Kết quả là file engine thu được có khả năng giảm thiểu đáng kể độ trễ và tăng tốc độ suy luận lên nhiều lần, đáp ứng trực tiếp yêu cầu xử lý khắt khe của hệ thống nhúng trong môi trường thực tế.

4.4.2 Triển khai mô hình trên máy tính nhúng

Để mô hình có thể hoạt động đồng bộ, khép kín và đáp ứng nghiêm ngặt các yêu cầu về tốc độ xử lý thời gian thực trên nền tảng máy tính nhúng, một luồng thực thi liên tục đã được thiết kế và xây dựng. Luồng thực thi này đóng vai trò là xương sống của hệ thống AI tại biên, điều phối luồng dữ liệu từ khâu tiếp nhận đầu vào cho đến khi đưa ra kết quả dự đoán cuối cùng. Cấu trúc của quy trình này được mô tả khái quát thông qua sơ đồ khối tại hình 4.8, trong đó mỗi giai đoạn thực hiện một nhiệm vụ chuyên biệt nhằm tối ưu hóa hiệu năng tổng thể.



Hình 4.8 - Sơ đồ khối thực thi trên máy tính nhúng

Quá trình bắt đầu với việc thu nhận dữ liệu từ camera thông qua giao thức truyền dẫn Ethernet tốc độ cao. Việc sử dụng kết nối Ethernet thay vì các giao thức không dây thông thường giúp đảm bảo băng thông rộng, tính ổn định của luồng video và quan trọng nhất là giảm thiểu tối đa độ trễ tín hiệu ngay từ khâu đầu vào. Các khung hình thô sau khi được tiếp nhận sẽ đi vào giai đoạn tiền xử lý, tại đây dữ liệu được thực hiện các phép biến đổi về kích thước, chuyển đổi không gian màu và chuẩn hóa thông số sao cho tương thích tuyệt đối với cấu trúc đầu vào mà mạng nơ-ron đã được huấn luyện. Bước tiền xử lý này đóng vai trò quyết định trong việc bảo toàn đặc trưng của các đối tượng nhiệt nhỏ, tránh tình trạng mất mát thông tin không gian trước khi đưa vào mô hình học sâu.

Sau bước chuẩn bị, khung hình đã được tiền xử lý sẽ được đẩy vào khối thực thi dự đoán, nơi mô hình RetinaNet đã biên dịch thành định dạng TensorRT (engine) vận hành trực tiếp trên nhân đồ họa GPU của Jetson Orin Nano. Nhờ vào việc áp dụng các kỹ thuật tối ưu hóa phần cứng chuyên sâu như gộp lớp và ép kiểu dữ liệu bán độ chính xác (FP16), khối này có khả năng thực hiện suy luận trên toàn bộ ảnh với tốc độ cực nhanh, đảm bảo đáp ứng yêu cầu xử lý thời gian thực cho hệ thống bay tự hành.

Tuy nhiên, đầu ra thô từ mạng nơ-ron thường tồn tại tình trạng nhiều khung bao dự đoán chồng chéo lên nhau tại cùng một đối tượng. Do đó, giai đoạn hậu xử lý sử dụng thuật toán Triệt tiêu phi cực đại (Non-Maximum Suppression - NMS) [23] được áp dụng để loại bỏ các kết quả dư thừa, từ đó lọc ra tọa độ bounding box duy nhất có độ tin cậy cao nhất. Kết quả cuối cùng là thông tin tọa độ của mục tiêu sẽ được hiển thị trên giao diện người dùng và truyền trực tiếp xuống mạch điều khiển bay để thực hiện các thao

tác bám bắt. Sự phối hợp nhịp nhàng giữa các khối chức năng này tạo nên một hệ thống tự hành khép kín, cho phép thiết bị bay không chỉ quan sát mà còn có khả năng nhận diện và phản hồi tức thời đối với các mục tiêu trong nhiệm vụ tìm kiếm cứu nạn.

Cuối cùng, hệ thống sẽ tiến hành hiển thị kết quả dự đoán. Các thông tin độ tin cậy (confidence score) của mục tiêu sẽ được vẽ đè trực tiếp lên luồng video gốc một cách trực quan trên giao diện người dùng. Việc thực hiện toàn bộ quy trình thị giác máy tính này ngay tại biên (Edge Computing) giúp hệ thống hoạt động hoàn toàn độc lập, giải quyết triệt để bài toán độ trễ đường truyền và giảm tải băng thông do không cần gửi dữ liệu video thô đi xử lý ở nơi khác. Sự phối hợp nhịp nhàng giữa các khối chức năng tạo ra một mô hình giám sát thông minh, cung cấp dữ liệu đầu ra chính xác, trực quan, hỗ trợ đắc lực cho công tác quan sát, phân tích tình huống và tìm kiếm cứu nạn.

CHƯƠNG 5: KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả lắp ráp hoàn chỉnh

Sau khi chọn lọc, cân nhắc về các phần cứng và hoàn thiện mạch của phần thiết kế và chế tạo thiết bị bay không người lái quadcopter của bạn Lê Quốc Nguyên, luận văn đã hoàn thiện cơ bản được quadcopter để bay. Hình 5.1 là kết quả lắp ráp mạch bay, Jetson Orin Nano, Camera ảnh nhiệt SIYI ZT6, SIYI Air Unit, ESC, 4 động cơ, Pin và định vị GPS lên quadcopter.



Hình 5.1 – Kết quả mô hình luận văn

5.2 Kết quả kiểm thử mô hình trên tập dữ liệu

5.2.1 Chi tiết bộ dữ liệu kiểm thử

Để có cái nhìn chính xác và toàn diện về hiệu năng của mạng nơ-ron sau quá trình huấn luyện, mô hình được tiến hành đánh giá ngoại tuyến trên một bộ dữ liệu kiểm thử tiêu chuẩn. Việc đánh giá này giúp xác định khả năng tổng quát hóa của mô hình trước khi đưa vào triển khai trên thiết bị nhúng thực tế.

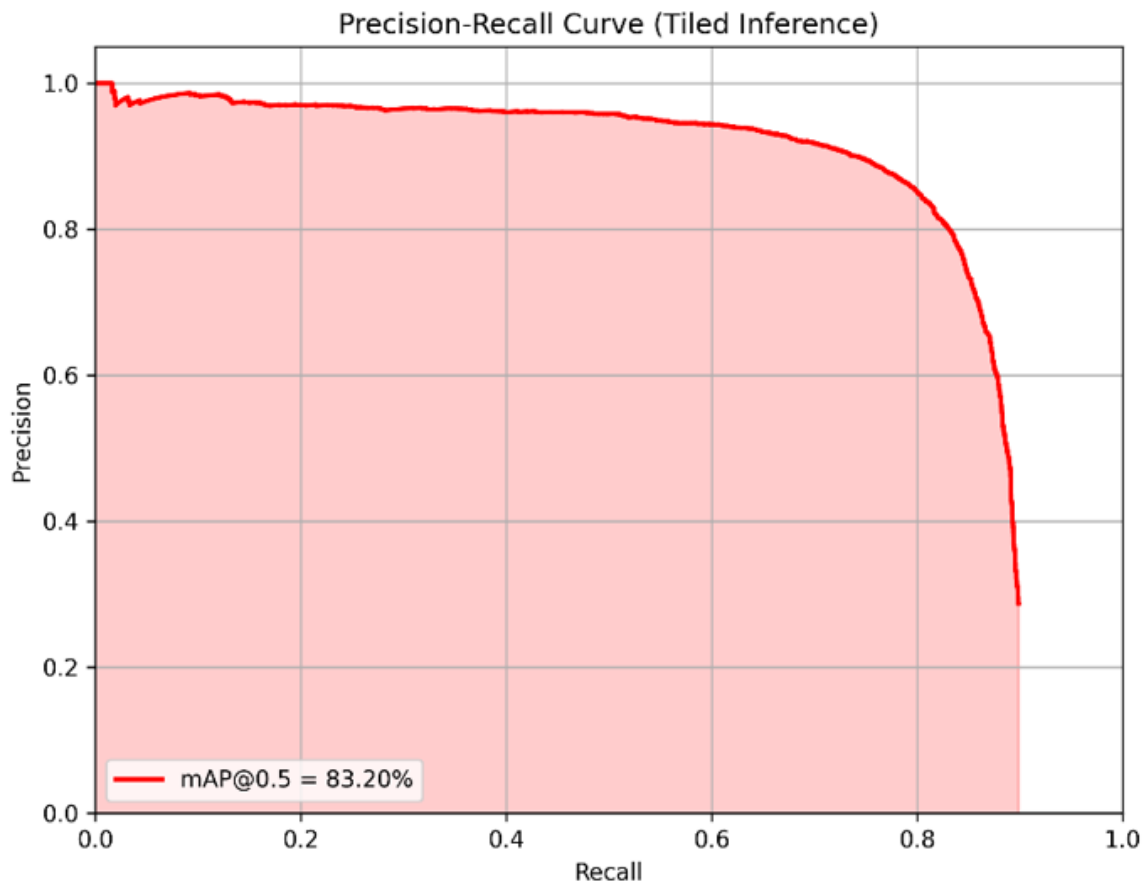
Quá trình kiểm thử được thực hiện trên một tập dữ liệu bao gồm 2969 hình ảnh. Tập dữ liệu này được phân chia từ cơ sở dữ liệu ban đầu theo tỷ lệ chuẩn 70% Train - 20% Valid - 10% Test nhằm đảm bảo có đủ số lượng mẫu để đánh giá một cách thống kê và đáng tin cậy.

Đặc điểm quan trọng nhất của tập dữ liệu kiểm thử này là tính độc lập và khách quan. Toàn bộ 2969 bức ảnh đều là những dữ liệu mới mà mô hình hoàn toàn chưa từng "nhìn thấy" trong suốt quá trình huấn luyện và xác thực trước đó. Việc sử dụng dữ liệu mới hoàn toàn giúp:

- Đảm bảo tính khách quan, ngăn chặn và phát hiện hiện tượng học vẹt – tình trạng mô hình chỉ hoạt động tốt trên dữ liệu huấn luyện nhưng lại nhận dạng kém trên dữ liệu thực tế.
- Đánh giá khả năng tổng quát hóa, tập test được lựa chọn bao gồm sự đa dạng về góc chụp, điều kiện ánh sáng (sáng/tối), độ nhiễu và nền phức tạp.

5.2.2 Kết quả các độ đo hiệu năng

Hiệu năng của mô hình trên tập dữ liệu kiểm thử được đánh giá thông qua các chỉ số thống kê và ma trận nhầm lẫn. Cụ thể, biểu đồ Precision-Recall cho thấy chỉ số mAP@0.5 (mean Average Precision tại ngưỡng IoU 0.5) đạt mức 83.20%. Chỉ số này phản ánh trung bình độ chính xác của mô hình trên toàn bộ tập dữ liệu, giá trị càng tiệm cận về 1 (hoặc 100%) thì mô hình càng thể hiện độ tin cậy cao. Trong đó, Precision (Độ chính xác) đánh giá tỷ lệ các đối tượng được mô hình nhận diện đúng là người trên tổng số các dự đoán đưa ra, còn Recall (Độ phủ) thể hiện khả năng mô hình tìm kiếm và phát hiện được bao nhiêu người thực tế có trong toàn bộ dữ liệu. Mức mAP 83.20% chứng tỏ mô hình có khả năng cân bằng khá tốt giữa việc dự đoán chính xác và không bỏ sót đối tượng.

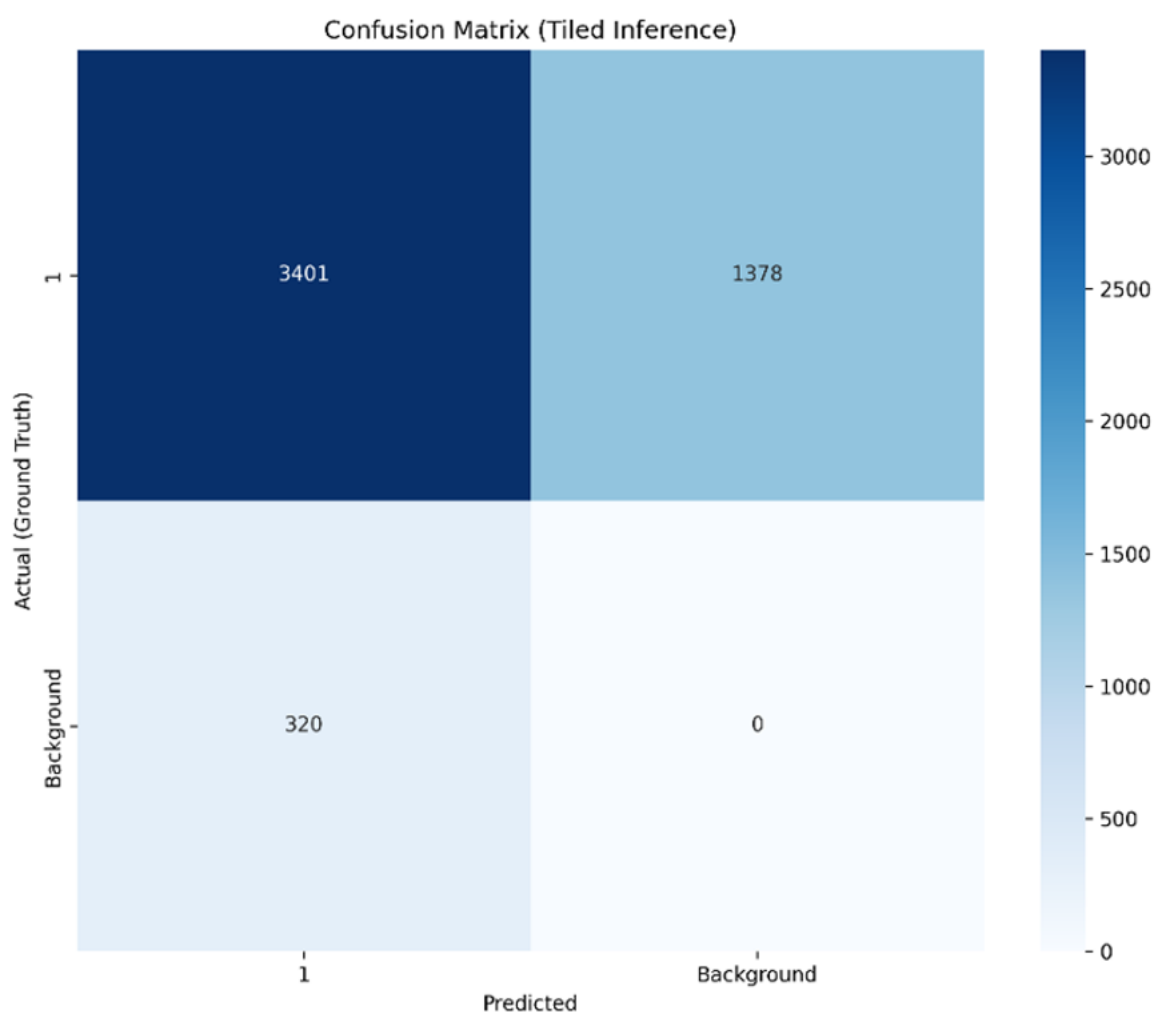


Hình 5.2 - Chỉ số mAP@0.5

Từ các số liệu định lượng chi tiết của Confusion Matrix, hiệu suất của mô hình được thể hiện rõ nét qua các chỉ số đánh giá tiêu chuẩn. Cụ thể, việc khoanh vùng chính xác 3401 vật thể đại diện cho số lượng True Positive (TP), khẳng định khả năng trích xuất đặc trưng tốt. Tuy nhiên, vẫn còn 1378 trường hợp bị bỏ sót (False Negative - FN), dẫn đến chỉ số Recall (Độ nhạy) của tập kiểm thử dừng lại ở mức 71.17%. Điều này cho thấy mô hình đôi khi vẫn gặp khó khăn khi bối cảnh phức tạp hoặc vật thể không hiện diện rõ ràng.

Ngược lại, đối với các vùng nền, chỉ có 320 trường hợp mô hình nhận diện nhầm thành đối tượng (False Positive - FP). Tỷ lệ FP thấp giúp chỉ số Precision (Độ chính xác) đạt mức ấn tượng là 91.40%. Sự chênh lệch giữa hai chỉ số này chỉ ra rằng mô hình có xu hướng dự đoán thận trọng: kiểm soát rất tốt các cảnh báo giả nhưng bù lại bị sót một lượng đối tượng nhất định. Tổng hợp toàn diện, đường cong Precision-Recall ghi nhận chỉ số trung bình mAP@0.5 đạt 83.20%. Đây là một kết quả khả quan, khẳng định mô hình hoạt động ổn định và duy trì được sự cân bằng tổng thể tốt khi xét tại ngưỡng giao

thoá (IoU) 0.5. Dù vậy, các chiến lược tinh chỉnh tiếp theo nên tập trung vào việc gia tăng Recall để hạn chế tối đa số lượng vật thể bị bỏ sót.



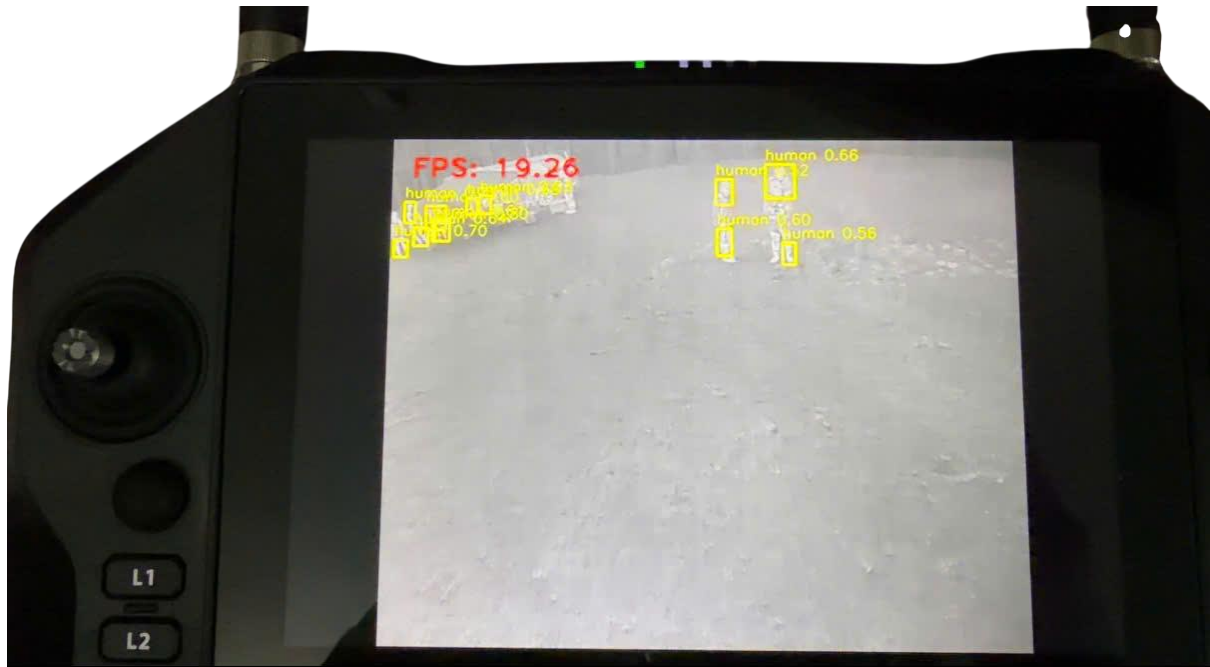
Hình 5.3 - Confusion matrix

5.2.3 Trực quan hóa kết quả

Để minh chứng rõ nét hơn cho các số liệu thống kê về chỉ số mAP và ma trận nhầm lẫn, mô hình đã được thực hiện suy luận thực tế trên tập dữ liệu kiểm thử độc lập. Kết quả dự đoán được trích xuất và trực quan hóa thông qua việc vẽ các bounding box cùng chỉ số độ tin cậy tương ứng lên ảnh nhiệt gốc. Những hình ảnh này cung cấp cái nhìn trực quan về năng lực nhận dạng của mô hình RetinaNet trong các điều kiện môi trường, tư thế đối tượng và khoảng cách khác nhau.

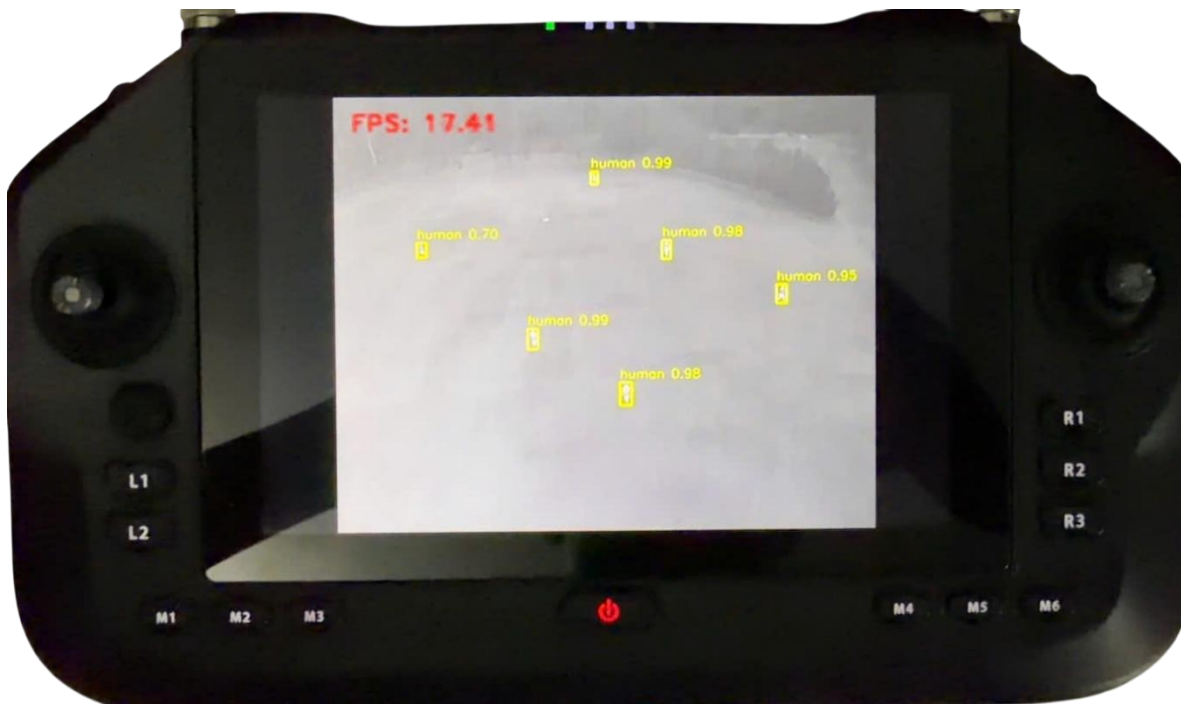
Kết quả dự đoán thực tế tại hình 5.4 minh họa khả năng nhận diện của mô hình khi quadcopter bay thấp, từ đó ta thấy được hiện tượng sự phân mảnh bounding box. Thay

vì khởi tạo một khung dự đoán duy nhất bao trọn toàn bộ cơ thể, mô hình lại xuất ra nhiều bounding box có kích thước nhỏ, bám vào từng cụm nhiệt cục bộ của mục tiêu (như phần đầu, vai, hoặc thân dưới) với các mức độ tin cậy dao động từ 0.56 đến 0.70. Kết quả này chứng minh việc mô hình chỉ áp dụng khi quadcopter bay cao.



Hình 5.4 - Kết quả nhận diện khi bay thấp

Khả năng nhận dạng đối tượng khi quadcopter bay ở độ cao lớn được thể hiện tại hình 5.5. Dù kích thước đối tượng trong ảnh bị thu nhỏ đáng kể do khoảng cách, kiến trúc Anchor Free vẫn thể hiện năng lực vượt trội trong việc khoanh vùng chính xác. Các hộp dự đoán vẫn được mô hình vẽ ra bao trọn mục tiêu với độ tin cậy ổn định. Ngoài ra, việc xuất hiện một vật thể phát ra nhiệt nhưng mô hình không đóng khung bounding box nó cũng cho thấy mô hình không bị nhầm lẫn các vật thể khác thành người mặc dù có 320 kết quả báo hiệu giả trong ma trận nhầm lẫn.



Hình 5.5 - Kết quả nhận dạng đối tượng ở xa

Khả năng phân biệt đối tượng mục tiêu và năng lực kháng nhiễu trong các điều kiện môi trường khắc nghiệt được minh chứng rõ nét qua kết quả kiểm thử thực tế tại hình 5.6. Trong khung hình này, hệ thống phải đối mặt với một thách thức vô cùng lớn của ảnh nhiệt: hiện tượng bão hòa nhiệt diện rộng. Toàn bộ bề mặt nền hấp thụ bức xạ mặt trời cường độ cao, tạo ra một mảng không gian trắng-xóa đồng nhất, làm suy giảm nghiêm trọng độ tương phản và triệt tiêu gần như hoàn toàn các đường viền cấu trúc hình học.

Mặc dù mức phát xạ nhiệt của môi trường lúc này tương đương, thậm chí vượt ngưỡng thân nhiệt người, mô hình mạng nơ-ron vẫn không khởi tạo bất kỳ bounding box sai lệch nào. Việc duy trì kết quả "trống" trong một điều kiện bất lợi như vậy chứng minh sự ưu việt của việc áp dụng hàm mất mát Focal Loss ở Chương 4. Hệ thống đã học được cách từ chối các vùng nhiệt độ cao nhưng thiếu hụt đặc trưng ngữ nghĩa về hình thái cơ thể người (như tỷ lệ các chi, đường nét khuôn dạng).

Nhờ khả năng học được bối cảnh tổng thể xung quanh đám nhiệt, ở nhiều khung hình, mô hình có thể tự tin loại trừ các tín hiệu nhiệt giả mạo. Minh chứng trực quan tại hình 5.6 phần nào giải thích lý do tỷ lệ nhận diện nhầm nền thành người (False Positive) được kiểm soát ở mức tương đối thấp so với tổng số dự đoán (giúp độ chính xác

Precision đạt 91.40%). Mặc dù trên toàn bộ tập kiểm thử, mô hình vẫn còn tạo ra 320 cảnh báo giả tại các vùng nền có nhiều quá phức tạp hoặc hình thái quá giống đối tượng, nhưng nhìn chung, sự nhạy bén của hàm mất mát tiêu điểm đã giúp hạn chế đáng kể các dự đoán sai lầm này. Điều này đóng vai trò quan trọng trong các ứng dụng thực tế, nơi việc cân bằng giữa khả năng phát hiện và việc giảm thiểu cảnh báo giả từ môi trường là yếu tố tiên quyết.



Hình 5.6 - Mô hình dự đoán với nền

5.3 Kết quả triển khai hệ thống thực tế

Quá trình triển khai và tối ưu hóa mô hình mạng nơ-ron trên thiết bị máy tính nhúng đã được thực hiện thông qua bộ công cụ TensorRT (lệnh trtexec). Để tối đa hóa hiệu năng trên giới hạn phần cứng, đồ thị mạng nơ-ron định dạng ONNX ban đầu đã được cấu hình ép kiểu dữ liệu trọng số xuống định dạng bán độ chính xác (FP16). Đồng thời, quá trình biên dịch sang file engine được thiết lập mức độ tối ưu hóa cực đại - Builder Optimization Level 5 cùng không gian bộ nhớ làm việc được cấp phát lên tới 4096 MB. Việc thiết lập các tham số này giúp bộ biên dịch của TensorRT có đủ không gian và quyền hạn để tiến hành gộp các lớp mạng và tự động tinh chỉnh, lựa chọn các hạt nhân tính toán tương thích và chạy nhanh nhất với kiến trúc GPU của thiết bị.

Kết quả đánh giá hiệu năng thu được từ quá trình chạy thử nghiệm cho thấy sự vượt trội trong khả năng xử lý thời gian thực. Cụ thể, Throughput đạt mức 78.6473 khung hình/giây. Phân tích chi tiết về mặt thời gian, độ trễ suy luận trung bình chỉ mất vốn vẹn 12.84 mili-giây cho mỗi khung hình, với thời gian chờ xếp hàng trung bình là 2.09 mili-giây. Việc đi sâu vào phân rã độ trễ cho thấy sự phân bổ tối ưu tài nguyên của hệ thống: thời gian truyền tải dữ liệu đầu vào từ bộ nhớ CPU sang VRAM của GPU (H2D Latency mean) diễn ra cực kỳ nhanh chóng với mức trung bình chỉ 0.167879 ms, chứng tỏ băng thông truyền tải nội bộ đã được khai thác hiệu quả.

Tiếp theo chu trình, giai đoạn tiêu tốn nhiều tài nguyên nhất là quá trình tính toán đồ thị học sâu thuần túy trên các khối xử lý của GPU (GPU Compute Time mean), nhưng cũng chỉ chiếm trung bình 12.66 ms. Cuối cùng, việc đẩy dữ liệu đầu ra là các tọa độ dự đoán từ GPU trở lại CPU (D2H Latency mean) để phục vụ cho thuật toán hậu xử lý NMS diễn ra gần như tức thời với mức trung bình 0.0177481 ms. Những thông số kỹ thuật chi tiết này minh chứng rõ ràng rằng các rào cản về mặt tính toán phần cứng nhúng đã được giải quyết hiệu quả. Tốc độ khung hình cao cùng với độ trễ truyền tải dữ liệu và tính toán cực thấp đảm bảo hệ thống có khả năng xuất tín hiệu điều khiển bám bắt mục tiêu một cách tức thời, đáp ứng trọn vẹn các tiêu chuẩn khắt khe nhất khi ứng dụng thiết bị bay không người lái quadcopter.

```
[07/12/2026-02:51:12] [I] === Performance summary ===
[07/12/2026-02:51:12] [I] Throughput: 78.6473 qps
[07/12/2026-02:51:12] [I] Latency: min = 12.6245 ms, max = 14.1766 ms, mean = 12.8412 ms, median = 12.6853 ms, percentile(90%) = 13.8963 ms, percentile(95%) = 14.059 ms, percentile(99%) = 14.1407 ms
[07/12/2026-02:51:12] [I] Enqueue Time: min = 1.3562 ms, max = 4.37866 ms, mean = 2.09367 ms, median = 2.14233 ms, percentile(90%) = 2.8125 ms, percentile(95%) = 3.09265 ms, percentile(99%) = 3.4588 ms
[07/12/2026-02:51:12] [I] H2D Latency: min = 0.142822 ms, max = 0.391327 ms, mean = 0.167879 ms, median = 0.151489 ms, percentile(90%) = 0.205261 ms, percentile(95%) = 0.244446 ms, percentile(99%) = 0.347351 ms
[07/12/2026-02:51:12] [I] GPU Compute Time: min = 12.4705 ms, max = 13.924 ms, mean = 12.6556 ms, median = 12.5117 ms, percentile(90%) = 13.6707 ms, percentile(95%) = 13.7387 ms, percentile(99%) = 13.867 ms
[07/12/2026-02:51:12] [I] D2H Latency: min = 0.00756836 ms, max = 0.0439453 ms, mean = 0.0177481 ms, median = 0.0155029 ms, percentile(90%) = 0.0292664 ms, percentile(95%) = 0.0312805 ms, percentile(99%) = 0.0380554 ms
[07/12/2026-02:51:12] [I] Total Host Walltime: 3.03889 s
[07/12/2026-02:51:12] [I] Total GPU Compute Time: 3.02468 s
[07/12/2026-02:51:12] [W] * GPU compute time is unstable, with coefficient of variance = 3.11861%.
[07/12/2026-02:51:12] [W] If not already in use, locking GPU clock frequency or adding --useSpinWait may improve the stability.
[07/12/2026-02:51:12] [I] Explanations of the performance metrics are printed in the verbose logs.
[07/12/2026-02:51:12] [I]
&&& PASSED TensorRT.trtexec [TensorRT v100300] # /usr/src/tensorrt/bin/trtexec --onnx=/home/huy/Documents/onnx/retinanet_anchorfree_DIoU.onnx --saveEngine=/home/huy/Documents/onnx/retinanet_anchorfree_DIoU.engine --fp16 --memPoolSize=workspace:4096 --builderOptimizationLevel=5 --timingCacheFile=timing.cache
huy@ubuntu:~$
```

Hình 5.7 - Thông số thực tế khi hoạt động trên máy tính nhúng

5.4 Thảo luận và đánh giá hạn chế

Mặc dù mô hình đã đạt được mức hiệu năng tương đối khả quan với chỉ số mAP@0.5 đạt 83.20%, việc triển khai hệ thống nhận diện ảnh nhiệt trong môi trường thực tế, đặc biệt là trên các thiết bị di động như thiết bị bay không người lái quadcopter, vẫn còn tồn tại một số thách thức nhất định. Dựa trên việc phân tích sâu Confusion Matrix và đặc thù vật lý của thiết bị, các hạn chế chính của hệ thống được tóm tắt như sau:

- Kết quả báo động giả từ ma trận nhầm lẫn, hệ thống ghi nhận 320 trường hợp dự đoán nhầm vùng nền thành đối tượng người. Nguyên nhân cốt lõi xuất phát từ nguyên lý hoạt động của camera hồng ngoại: cảm biến thu nhận bức xạ nhiệt độ bề mặt thay vì quang phổ ánh sáng khả kiến. Do đó, trong môi trường phức tạp, các vật thể có mức phát xạ nhiệt tương đương với nhiệt độ cơ thể người (khoảng 34°C - 37°C) sẽ xuất hiện đặc trưng hình ảnh rất giống với mục tiêu. Các nguồn gây nhiễu điển hình bao gồm: động cơ xe máy vừa tắt, nắp cống hoặc mặt đường nhựa hấp thụ nhiệt mặt trời, và các loài động vật lớn. Sự tương đồng về vùng nhiệt độ này khiến mạng nơ-ron bị đánh lừa, dẫn đến việc khoanh vùng sai đối tượng.

- Bỏ sót đối tượng là thách thức lớn nhất của hệ thống khi có tới 1378 trường hợp (chiếm phần lớn sai số) mô hình không thể phát hiện ra người. Nguyên nhân chủ yếu gây ra tình trạng này là do hiện tượng "hòa trộn nhiệt" [24]. Hiện tượng này thường xảy ra vào thời điểm ban ngày hoặc mùa hè, khi nhiệt độ môi trường ngoài trời tăng cao và tiệm cận với nhiệt độ cơ thể người. Lúc này, độ tương phản nhiệt giữa đối tượng và cảnh nền bị suy giảm nghiêm trọng hoặc gần như bằng không. Hậu quả là hình dáng của người bị "tàng hình" hoặc hòa lẫn hoàn toàn vào môi trường trong ảnh nhiệt, khiến mô hình mất đi các đặc trưng đường biên quan trọng để nhận dạng.

- Giới hạn về thiết bị phần cứng và rào cản vật lý trên thiết bị bay không người lái quadcopter cũng là một thách thức lớn ảnh hưởng trực tiếp đến quá trình triển khai thực tế của hệ thống. Thứ nhất, về mặt quang học, cảm biến nhiệt của camera SIYI ZT6 được trang bị chỉ có độ phân giải ở mức 640x512 pixel. Khi thiết bị bay không người lái quadcopter nâng cao tầm quan sát (hoạt động ở độ cao từ 50 mét trở lên), kích thước của đối tượng trong khung hình bị thu nhỏ lại đáng kể. Mặc dù mô hình Anchor Free hoạt động tốt, nhưng ở khoảng cách quá xa, hiện tượng bỏ sót mục tiêu vẫn xảy ra do

giới hạn phân giải của thiết bị thu hình. Thứ hai, về năng lực xử lý và tản nhiệt, máy tính nhúng Jetson Orin Nano khi phải liên tục chạy mô hình học sâu ở tần số cao sẽ sinh ra một lượng nhiệt lượng rất lớn. Nếu hệ thống không được trang bị cơ chế tản nhiệt khí động học chuyên dụng phù hợp, nhiệt độ cao sẽ dẫn đến hiện tượng hệ thống tự động giảm xung nhịp đột ngột. Điều này làm hệ thống tự hành mất đi tính ổn định trong các nhiệm vụ bay kéo dài. Nhìn chung, những hạn chế cốt lõi về độ phân giải thấp của cảm biến nhiệt và khả năng tản nhiệt của phần cứng máy tính nhúng trên không trung đang là những bài toán vật lý lớn cần được tiếp tục giải quyết và tối ưu trước khi đưa hệ thống vào ứng dụng thương mại với quy mô lớn.

5.5 Kết luận chung

Luận văn đã hoàn thành mục tiêu nghiên cứu, thiết kế và triển khai thành công hệ thống nhận diện đối tượng người qua ảnh nhiệt trên nền tảng máy tính nhúng chuyên dụng cho thiết bị bay không người lái quadcopter. Qua quá trình thực hiện từ khâu xây dựng dữ liệu, huấn luyện đến triển khai thực tế, luận văn rút ra những kết luận trọng tâm sau:

- Về mặt thiết kế và triển khai hệ thống, luận văn đã xây dựng được một luồng thực thi khép kín, xử lý trực tiếp luồng dữ liệu video theo thời gian thực từ camera nhiệt qua giao thức Ethernet. Việc ứng dụng bộ công cụ TensorRT để chuyển đổi và tối ưu hóa mô hình sang định dạng engine đã giải quyết thành công bài toán thắt nút cổ chai về hiệu năng trên máy tính nhúng Jetson Orin Nano, đảm bảo hệ thống có thể xuất tín hiệu điều khiển với độ trễ thấp để phục vụ mục đích bám bắt mục tiêu.

- Về mặt hiệu năng nhận diện mô hình học sâu được huấn luyện đã thể hiện sự ổn định và khả năng tổng quát hóa tốt khi đạt chỉ số mAP@0.5 ở mức 83.20% trên tập dữ liệu kiểm thử hoàn toàn mới (2969 ảnh). Hệ thống có khả năng nhận diện và khoanh vùng chính xác đối tượng người trong nhiều điều kiện thiếu sáng hoặc ban đêm – điều mà các camera quang học thông thường không thể thực hiện được.

- Về tính thực tiễn và các giới hạn cốt lõi dù đạt được các kết quả khả quan, quá trình thử nghiệm thực tế cũng đã chỉ ra những rào cản vật lý đặc thù của công nghệ ảnh nhiệt. Hiện tượng hòa trộn nhiệt khi nhiệt độ môi trường tiệm cận thân nhiệt người là nguyên nhân chính dẫn đến việc bỏ sót đối tượng (1378 trường hợp False Negative). Ngoài ra,

những hạn chế về độ phân giải của cảm biến nhiệt và khả năng tản nhiệt của phần cứng trên thiết bị bay không người lái quadcopter cũng đặt ra những bài toán cần tiếp tục giải quyết để đưa hệ thống vào ứng dụng thương mại quy mô lớn.

Nhìn chung, luận văn đã chứng minh được tính khả thi của việc tích hợp trí tuệ nhân tạo lên các thiết bị nhúng Edge AI để xử lý ảnh nhiệt. Kết quả của nghiên cứu này hoàn toàn có thể làm nền tảng cho các ứng dụng thực tiễn như tìm kiếm cứu nạn, tuần tra an ninh ban đêm hoặc giám sát biên giới bằng thiết bị bay không người lái quadcopter.

5.6 Hướng phát triển

Dung hợp cảm biến: kết hợp dữ liệu từ camera quang học (RGB) và camera nhiệt để khắc phục hiện tượng hòa trộn nhiệt vào ban ngày.

Thu thập thêm dữ liệu ngoại lệ: bổ sung các mẫu dữ liệu về động vật, máy móc phát nhiệt vào tập huấn luyện để giảm thiểu tỷ lệ báo động giả.

Tối ưu hóa phần cứng: thiết kế các bộ tản nhiệt khí động học chuyên dụng cho máy tính nhúng khi treo trên thiết bị bay không người lái quadcopter để duy trì hiệu năng ổn định trong thời gian bay dài.

Thuật toán tracking: kết hợp phần tracking theo đối tượng để cho thiết bị bay không người lái quadcopter bám theo đối tượng.

TÀI LIỆU THAM KHẢO

- [1] D. V. Hiep, "Nghiên cứu giải pháp kết hợp ảnh nhiệt và ảnh màu trong bài toán phát hiện và theo vết đối tượng người," Luận án Tiến sĩ, Đại học Bách khoa Hà Nội, Hà Nội, Việt Nam, 2023.
- [2] N. D. Thuan, H. S. Hong, and L. H. Anh, "PDIWS: Thermal imaging dataset for person detection in intrusion warning systems," in Proc. Int. Conf. Control Autom. Inf. Sci. (ICCAIS), 2023, pp. 1-6.
- [3] F. Ciccone and A. Ceruti, "Real-time search and rescue with Quadcopters: A deep learning approach for small-object detection based on YOLO," Quadcopters, vol. 9, no. 8, p. 514, 2025.
- [4] P. Fraternali, L. Morandini, and R. Motta, "Enhancing search and rescue missions with QUADCOPTER thermal video tracking," Remote Sens., vol. 17, no. 17, p. 3032, 2025.
- [5] A. A. Ulmămei, T. D'Adamo, C. E. Vasile, and R. Hobincu, "Human detection in QUADCOPTER thermal imagery: Dataset extension and comparative evaluation on embedded platforms," J. Imaging, vol. 11, no. 12, p. 436, 2025.
- [6] M. Planck, "On the law of distribution of energy in the normal spectrum," Ann. Phys., vol. 4, no. 3, pp. 553-563, 1901.
- [7] J. R. Howell, M. P. Menguc, and R. Siegel, Thermal Radiation Heat Transfer, 6th ed. Boca Raton, FL, USA: CRC Press, 2015.
- [8] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
- [9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436-444, 2015.

- [10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770-778.
- [11] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 2117-2125.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 2980-2988.
- [13] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137-1149, Jun. 2017.
- [14] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 9627-9636.
- [15] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740-755.
- [16] NVIDIA Corporation. "Jetson Orin Nano Series Module Data Sheet." NVIDIA. <https://developer.nvidia.com/embedded/downloads>
- [17] Roboflow. "Roboflow Universe: Quadcopter Thermal Datasets." Roboflow. <https://universe.roboflow.com/>
- [18] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2022, pp. 966-970.
- [19] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in Proc. AAAI Conf. Artif. Intell., vol. 34, no. 7, 2020, pp. 11251-11258.

- [20] L. Prechelt, "Early stopping - but when?," in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 55-69.
- [21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [22] NVIDIA Corporation. "NVIDIA TensorRT Documentation." NVIDIA. <https://docs.nvidia.com/deeplearning/tensorrt/>
- [23] A. Neubeck and L. Van Gool, "Efficient non-maximum suppression," in *Proc. 18th Int. Conf. Pattern Recognit. (ICPR)*, vol. 3, 2006, pp. 850-855.
- [24] R. Gade and T. B. Moeslund, "Thermal cameras and applications: A survey," *Mach. Vis. Appl.*, vol. 25, no. 1, pp. 245-262, Jan. 2014.
- [25] J. Delmerico et al., "The current state and future outlook of rescue robotics," *J. Field Robot.*, vol. 36, no. 7, pp. 1171-1191, 2019.
- [26] WiSARD Dataset. "WiSARD: Wildfire Search and Rescue Dataset." University of Washington. <https://sites.google.com/uw.edu/wisard/>
- [27] S. Speth, A. Gonçalves, B. Rigault, S. Suzuki, M. Bouazizi, Y. Matsuo, and H. Prendinger, "Deep Learning with RGB and Thermal Images onboard a Quadcopter for Monitoring Operations," *J. Field Robot.*, pp. 1–29, 2022, doi: 10.1002/rob.22082.

PHỤ LỤC

Phân tách file xml và file ảnh thành hai thư mục:

```
import os,glob,shutil

if __name__ == "__main__":

    data_dir = '/home/huy/Documents/de_tai_tot_nghiep/Quadcopter Thermal.v4i.voc'

    #--- lay dia chi folder ---

    train_data = os.path.join(data_dir,'train') #lay dia chi cua folder train
    valid_data = os.path.join(data_dir,'valid') #lay dia chi cua folder valid
    test_data = os.path.join(data_dir,'test') #lay dia chi cua folder test
    anno_dir = os.path.join(data_dir,'anno') #lay dia chi cua folder anno
    img_dir = os.path.join(data_dir,'imgs') #lay dia chi cua folder imgs

    #--- lay cac file xml va jpg

    """

    can di chuyen cac file trong folder train,test va valid sang folder anno va imgs
    thay doi folder bang cach thay doi ten dia chi folder o 2 dong code phia duoi

    """

    xml_file = glob.glob(os.path.join(train_data,'*.xml'))
    jpg_file = glob.glob(os.path.join(train_data,'*.jpg'))

    #--- di chuyen file sang folder khac

    #lay file name cua xml va jpg bang lenh os.path.basename

    for xml_path in xml_file:

        #co the xoa dau # o dong bien duoi de biet no lay gia tri nhu the nao

        #print(xml_path)

        xml_file_name = os.path.basename(xml_path)

        #print(xml_file_name)

        new_path = os.path.join(anno_dir,xml_file_name)

        #print(new_path)
```

```

#di chuyen file
shutil.move(xml_path,new_path)

print(f"da di chuyen file xml thanh cong sang {new_path}")

for jpg_path in jpg_file:
    jpg_file_name = os.path.basename(jpg_path)
    #print(jpg_file_name)
    new_path = os.path.join(img_dir,jpg_file_name)
    #print(new_path)
    #di chuyen file
    shutil.move(jpg_path, new_path)

    print(f"da di chuyen file jpg thanh cong sang {new_path}")

```

Phân cắt ảnh và chuyển dữ liệu thành file CSV

Phân cắt ảnh:

"""

Tiling utilities — dùng chung cho preprocessing (main_step_123.py) và inference (predict.py).

Cung cấp các hàm:

- generate_tile_coords : tính tọa độ các tile trên ảnh gốc
- get_bboxes_for_tile : lọc và chuyển bbox sang hệ tọa độ tile
- crop_and_save_tile : crop và lưu tile ra đĩa
- apply_global_nms : NMS trên toàn ảnh sau khi gộp boxes từ tất cả tiles

"""

```
import os
```

```
import numpy as np
```

```
from PIL import Image
```

```
def generate_tile_coords(img_w, img_h, tile_w, tile_h, overlap=0.2):
```

"""

Tạo danh sách tọa độ (x_offset, y_offset, tile_w, tile_h) cho từng tile.

Tiles cuối hàng / cuối cột được đẩy sát mép ảnh để đảm bảo toàn bộ ảnh đều được cover, dù step không chia hết.

Args:

img_w, img_h : kích thước ảnh gốc (pixel)

tile_w, tile_h : kích thước mỗi tile (pixel)

overlap : tỉ lệ chồng lấp giữa 2 tile liền kề (0.0 – 1.0)

Returns:

list of (x_offset, y_offset, tile_w, tile_h)

"""

```
step_x = max(1, int(tile_w * (1 - overlap)))
```

```
step_y = max(1, int(tile_h * (1 - overlap)))
```

```
#---Quét vùng bắt đầu cho các tile---
```

```
# --- Trục X ---
```

```
xs = list(range(0, img_w - tile_w + 1, step_x))
```

```
# Tile cuối chạm sát cạnh phải nếu chưa cover hết
```

```
if not xs or xs[-1] + tile_w < img_w:
```

```
    xs.append(max(0, img_w - tile_w))
```

```
xs = list(dict.fromkeys(xs)) # loại trùng, giữ thứ tự
```

```
# --- Trục Y ---
```

```
ys = list(range(0, img_h - tile_h + 1, step_y))
```

```
# Tile cuối chạm sát cạnh dưới nếu chưa cover hết
```

```
if not ys or ys[-1] + tile_h < img_h:
```

```
    ys.append(max(0, img_h - tile_h))
```

```
ys = list(dict.fromkeys(ys)) # loại trùng, giữ thứ tự
```

```
return [(x, y, tile_w, tile_h) for y in ys for x in xs]
```

```
def get_bboxes_for_tile(bboxes, class_ids, tile_x, tile_y, tile_w, tile_h,
```


min_visibility):

"""

Lọc và chuyển đổi bbox từ hệ tọa độ ảnh gốc sang hệ tọa độ tile.

Một bbox được giữ lại khi $\geq$ min_visibility diện tích của nó nằm trong tile.

Phần bbox nằm ngoài tile bị clip tại biên tile.

Args:

bboxes : list of [x_c, y_c, w, h] trong hệ ảnh gốc (pixel)

class_ids : list of int class id tương ứng

tile_x, tile_y, tile_w, tile_h : vị trí và kích thước tile trong ảnh gốc

min_visibility : tỉ lệ diện tích tối thiểu phải nằm trong tile

Returns:

(new_bboxes, new_class_ids) — bbox trong hệ tọa độ tile,

format [x_c, y_c, w, h]

"""

new_bboxes = []

new_class_ids = []

for bbox, cls_id in zip(bboxes, class_ids):

 x_c, y_c, w, h = bbox

 # Chuyển [x_c, y_c, w, h] $\rightarrow$ [xmin, ymin, xmax, ymax]

 xmin = x_c - w / 2.0

 ymin = y_c - h / 2.0

 xmax = x_c + w / 2.0

 ymax = y_c + h / 2.0

 # Tính phần giao nhau giữa bbox và tile

 inter_xmin = max(xmin, tile_x)

 inter_ymin = max(ymin, tile_y)

 inter_xmax = min(xmax, tile_x + tile_w)

```

inter_ymax = min(ymax, tile_y + tile_h)
# Bỏ qua nếu không có giao
if inter_xmax <= inter_xmin or inter_ymax <= inter_ymin:
    continue
inter_area = (inter_xmax - inter_xmin) * (inter_ymax - inter_ymin)
bbox_area = w * h
if bbox_area <= 0:
    continue
# Kiểm tra tỉ lệ hiển thị trong tile
if inter_area / bbox_area < min_visibility:
    continue
# Chuyển sang hệ tọa độ tile (gốc tại góc trên trái của tile)
new_xmin = inter_xmin - tile_x
new_ymin = inter_ymin - tile_y
new_xmax = inter_xmax - tile_x
new_ymax = inter_ymax - tile_y
new_x_c = (new_xmin + new_xmax) / 2.0
new_y_c = (new_ymin + new_ymax) / 2.0
new_w = new_xmax - new_xmin
new_h = new_ymax - new_ymin
new_bboxes.append([new_x_c, new_y_c, new_w, new_h])
new_class_ids.append(cls_id)
return new_bboxes, new_class_ids
def crop_and_save_tile(image_path, tx, ty, tw, th, output_dir, tile_idx):
    """
    Crop một tile từ ảnh gốc và lưu ra đĩa (JPEG, quality=95).
    Args:

```

image_path : đường dẫn ảnh gốc

tx, ty : offset (x, y) của tile trong ảnh gốc

tw, th : width, height của tile

output_dir : thư mục lưu tile

tile_idx : chỉ số tile (dùng để đặt tên file, 3 chữ số)

Returns:

tile_path : đường dẫn file tile đã lưu

"""

```
img = Image.open(image_path)
```

```
tile = img.crop((tx, ty, tx + tw, ty + th))
```

```
base_name = os.path.splitext(os.path.basename(image_path))[0]
```

```
tile_filename = f"{base_name}_tile_{tile_idx:03d}.jpg"
```

```
tile_path = os.path.join(output_dir, tile_filename)
```

```
tile.save(tile_path, "JPEG", quality=95)
```

```
return tile_path
```

```
def apply_global_nms(boxes, scores, iou_threshold=0.45, score_threshold=0.05):
```

"""

NMS (Non-Maximum Suppression) trên toàn bộ predictions sau khi gộp

boxes từ tất cả tiles. Loại bỏ box trùng lặp ở vùng overlap.

BẮT BUỘC phải gọi hàm này sau merge_tile_detections — nếu không,

1 người ở vùng overlap sẽ bị detect 2 lần (2 box chồng nhau).

Args:

boxes : list/array of [ymin, xmin, ymax, xmax] trong hệ ảnh gốc

scores : list/array of float confidence score tương ứng

iou_threshold : ngưỡng IoU để coi 2 box là trùng nhau

score_threshold: loại bỏ box có score thấp hơn ngưỡng này

Returns:

```

    (final_boxes, final_scores, num_det)

    final_boxes shape: (N, 4), format [ymin, xmin, ymax, xmax]
    """

    if len(boxes) == 0:
        return np.zeros((0, 4), dtype=np.float32), np.zeros(0, dtype=np.float32), 0
    boxes = np.array(boxes, dtype=np.float32)
    scores = np.array(scores, dtype=np.float32)

    # Lọc theo score threshold
    keep_mask = scores >= score_threshold
    boxes = boxes[keep_mask]
    scores = scores[keep_mask]

    if len(boxes) == 0:
        return np.zeros((0, 4), dtype=np.float32), np.zeros(0, dtype=np.float32), 0

    # Sắp xếp theo score giảm dần
    order = np.argsort(scores)[::-1]
    boxes = boxes[order]
    scores = scores[order]

    y1, x1, y2, x2 = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    areas = (y2 - y1) * (x2 - x1)

    kept = []
    suppressed = np.zeros(len(boxes), dtype=bool)
    for i in range(len(boxes)):
        if suppressed[i]:
            continue

        kept.append(i)

        # Tính IoU của box i với tất cả box phía sau (vectorized)
        inter_y1 = np.maximum(y1[i], y1[i + 1:])

```

```

inter_x1 = np.maximum(x1[i], x1[i + 1:])
inter_y2 = np.minimum(y2[i], y2[i + 1:])
inter_x2 = np.minimum(x2[i], x2[i + 1:])
inter_h = np.maximum(0.0, inter_y2 - inter_y1)
inter_w = np.maximum(0.0, inter_x2 - inter_x1)
inter_area = inter_h * inter_w
iou = inter_area / (areas[i] + areas[i + 1:] - inter_area + 1e-8)
suppressed[i + 1:][iou > iou_threshold] = True
final_boxes = boxes[kept]
final_scores = scores[kept]
return final_boxes, final_scores, len(kept)

```

Phần chuyển dữ liệu thành file CSV:

```

import xml.etree.ElementTree as ET
import os, glob
import pandas as pd
# [TILING - THÊM MỚI] import module tiling và thư viện PIL để crop ảnh
from tiling_utils import generate_tile_coords, get_bboxes_for_tile, crop_and_save_tile
if __name__ == '__main__':
    data_dir = '/home/huy/Documents/de_tai_tot_nghiep/Quadcopter Thermal.v4i.voc'
    anno_dir = os.path.join(data_dir, 'anno') # lay dia chi thuc muc anno
    img_dir = os.path.join(data_dir, 'imgs')
    tile_dir = os.path.join(data_dir, 'data_6_7_2026')
    os.makedirs(tile_dir, exist_ok=True)
    # [TILING - THÊM MỚI] thông số tiling — chỉnh ở đây nếu muốn thay đổi
    TILE_W = 640
    TILE_H = 512
    OVERLAP = 0.20 # 20% chồng lấp

```

```

# khoi tao dic gan nhan background voi id 0
class_mapping = {'background': 0}

# tao 1 mang de chua cac gia tri sau khi doc ra vao day
# [TILING - ĐỒI] mỗi phần tử trong xml_list giờ là 1 tile, không còn là 1 ảnh gốc
xml_list = []

# lay dia chi file xml trong anno
for xml_file in glob.glob(anno_dir + '/*.xml'):

    # lay vi tri cua file xml tren o dia
    tree = ET.parse(xml_file)

    # lay nut goc
    root = tree.getroot()

    # lay kích thước ảnh
    width_img = int(root.find('size/width').text)
    height_img = int(root.find('size/height').text)

    path_img = os.path.join(img_dir, root.find('filename').text) # lay dia chi cung
tung anh

    # Khoi tao danh sach rong cho tung anh de chua bndbox va class_id
    bboxes_of_img = []
    class_ids_of_img = []

    # truy cap vao phan tu object de doc nhan, bbox
    for member in root.findall('object'):

        # tim ten class cua tung object
        class_name = member.find('name').text

        class_id = class_mapping.setdefault(class_name, len(class_mapping))

        bndbox = member.find('bndbox')

        if bndbox is not None:

            xmin = int(bndbox.find('xmin').text)

```

```

ymin = int(bndbox.find('ymin').text)
xmax = int(bndbox.find('xmax').text)
ymax = int(bndbox.find('ymax').text)
w = xmax - xmin
h = ymax - ymin
x = xmin + w / 2.0
y = ymin + h / 2.0
# BỘ LỌC CHẶN BÓNG MA: Chỉ lấy những box có width và height > 0
if w > 0 and h > 0:
    bboxes_of_img.append([x, y, w, h])
    class_ids_of_img.append(class_id)

# -----
# [TILING - THAY ĐỔI CHÍNH]
# TRƯỚC: sau khi đọc xong object, append 1 dòng cho cả ảnh gốc
# SAU: chia ảnh thành N tiles, append N dòng (1 dòng/tile)
# -----

# Bước 1: Tạo tọa độ các tile
tile_coords = generate_tile_coords(width_img, height_img, TILE_W, TILE_H,
OVERLAP)

for tile_idx, (tx, ty, tw, th) in enumerate(tile_coords):
    # Bước 2: Lọc và chuyển đổi bbox sang hệ tọa độ tile
    tile_bboxes, tile_cls_ids = get_bboxes_for_tile(
        bboxes_of_img, class_ids_of_img,
        tx, ty, tw, th,
        min_visibility=0.3
    )

    # Bước 3: Crop tile và lưu ra đĩa (trả về đường dẫn file tile)

```

```

tile_path = crop_and_save_tile(path_img, tx, ty, tw, th, tile_dir, tile_idx)

# Bước 4: Ghi 1 dòng CSV cho tile này (kể cả tile không có object)
xml_list.append({
    'path_img' : tile_path,
    'bbox'     : tile_bboxes,
    'class_id' : tile_cls_ids
})

# -----

# Tao DataFrame truc tiep tu list da duoc gom nhom
grouped_df = pd.DataFrame(xml_list)

# [TILING - ĐỔI TÊN] lưu ra file CSV mới để không ghi đè CSV gốc
os.chdir(data_dir + '/csv_file/')
grouped_df.to_csv('data_tile_6_7_2026.csv', index=False)

#print(f'Xong! Đã tạo {len(xml_list)} tile records → data_information_tiled.csv")
print(f'Ảnh tile được lưu tại: {tile_dir}')

```

Phần chia train, valid và test:

```

from sklearn.model_selection import train_test_split

import pandas as pd
import os

if __name__ == "__main__":
    data_dir='/home/huy/Documents/RetinaNet_FreeAnchor_DIoU/Quadcopter
    Thermal.v4i.voc'

    csv_file = os.path.join(data_dir, 'csv_file')
    read_csv = pd.read_csv(csv_file+'data_tile_6_7_2026.csv')

    #lay tat ca ten trong path_img vao 1 mang
    unique_file = read_csv['path_img'].unique()

    #chia data train, valid, test

```



```

train_data,temp_data = train_test_split(unique_file,test_size=0.3,random_state=76)
#chia tap train thanh 70% va tap temp 30% (tu tap data unique)

valid_data,test_data = train_test_split(temp_data,test_size=1/3,random_state=35)
#chia tap valid 20% tap test 10% (tu tap temp)

train_df = read_csv[read_csv['path_img'].isin(train_data)]
valid_df = read_csv[read_csv['path_img'].isin(valid_data)]
test_df = read_csv[read_csv['path_img'].isin(test_data)]

print(f"Tong so anh: {len(unique_file)}")
print(f"so anh trong tap train: {len(train_data)}")
print(f"so anh trong tap test: {len(test_data)}")
print(f"so anh trong tap valid: {len(valid_data)}")

train_df.to_csv(csv_file+'/train_data_6_7_2026z.csv',index=False)
valid_df.to_csv(csv_file+'/valid_data_6_7_2026z.csv',index=False)
test_df.to_csv(csv_file+'/test_data_6_7_2026z.csv',index=False)

print('Chia data thanh cong')

```

Phần mạng trích xuất đặc trưng reset-40:

```
import tensorflow as tf
```

```
def Conv_block(x, filter, stride):
```

```
    """
```

```
    x: input
```

```
    filter: so luong filter
```

```
    stride: buoc nhay
```

```
    Day la block lam thay doi kich thuoc input (chi giam hoac khong giam)
```

```
    input va output se khong khop nhau, sau khi qua ham nay, input va output se dc can
    bang so luong kenh de co the cong voi nhau
```

```
    """
```

```
    x_skip = x
```

```
    f1, f2 = filter
```

```

# block dau tien

#neu filter stride la 1 thi chi la buoc nhay 1, neu stride la 2 thi no dai dien cho viec
maxpool

x = tf.keras.layers.Conv2D(filters=f1, kernel_size=(1, 1), strides=(stride, stride),
padding='valid',
kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.ReLU()(x)
# block thu 2

x = tf.keras.layers.Conv2D(filters=f1, kernel_size=(3, 3), strides=(1, 1),
padding='same',kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.ReLU()(x)
# block thu 3 (tang so kenh ban do dac trung)

x = tf.keras.layers.Conv2D(filters=f2, kernel_size=(1, 1), strides=(1, 1),
padding='valid',kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
x = tf.keras.layers.BatchNormalization()(x)
# tang so kenh x_skip de cong vao x tai block 3
x_skip = tf.keras.layers.Conv2D(filters=f2, kernel_size=(1, 1), strides=(stride,
stride), padding='valid',kernel_regularizer=tf.keras.regularizers.l2(0.001))(x_skip)
x_skip = tf.keras.layers.BatchNormalization()(x_skip)
# cong x_skip vao x
x = tf.keras.layers.Add()([x, x_skip])
x = tf.keras.layers.ReLU()(x)
return x #(so kenh ban do dac trung = f2)
def Res_id_block(x, filter):
    """
    resnet block khong thay doi kich thuoc

```

khởi này chỉ hoạt động khi input và output có cùng kích thước [w,h,c]

```
"""
```

```
x_skip = x #(so kênh x skip = f2 vì là return của hàm conv_block)
```

```
f1, f2 = filter
```

```
# block 1
```

```
x = tf.keras.layers.Conv2D(filters=f1, kernel_size=(1, 1), padding='valid',  
    kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
```

```
x = tf.keras.layers.BatchNormalization()(x)
```

```
x = tf.keras.layers.ReLU()(x)
```

```
# block 2
```

```
x = tf.keras.layers.Conv2D(filters=f1, kernel_size=(3, 3), padding='same',  
    kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
```

```
x = tf.keras.layers.BatchNormalization()(x)
```

```
x = tf.keras.layers.ReLU()(x)
```

```
# block 3
```

```
x = tf.keras.layers.Conv2D(filters=f2, kernel_size=(1, 1), padding='valid',  
    kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
```

```
x = tf.keras.layers.BatchNormalization()(x)
```

```
# add
```

```
x = tf.keras.layers.Add()([x, x_skip])
```

```
x = tf.keras.layers.ReLU()(x)
```

```
return x
```

```
def resnet_40_backbone():
```

```
"""
```

ResNet-40 backbone: giống ResNet-50 nhưng bỏ block 5 (P5/C5, Stride 32).

Chỉ xuất ra 2 feature map:

- block_3_out: C3 (Stride 8) -> P3 trong FPN

```

- block_4_out: C4 (Stride 16) -> P4 trong FPN
"""

# input size
input_dim = tf.keras.layers.Input(shape=(None, None, 3))

# su dung padding de giu lai bien anh truoc khi vao conv2d 7x7
x = tf.keras.layers.ZeroPadding2D(padding=(3, 3),
name='padding_zero')(input_dim)

# khoi 1
x = tf.keras.layers.Conv2D(filters=64, kernel_size=(7, 7), strides=(2, 2),
padding='valid', use_bias=False,
name='Conv7x7')(x)

x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.ReLU()(x)
x = tf.keras.layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2), padding='same',
name='overlapping')(x)

# khoi 2
x = Conv_block(x, (64, 256), 1) # 3 layers
x = Res_id_block(x, (64, 256)) # 3 layers
x = Res_id_block(x, (64, 256)) # 3 layers
# khoi 3 -> C3 / P3 (Stride 8)
x = Conv_block(x, (128, 512), 2) # 3 layers
x = Res_id_block(x, (128, 512)) # 3 layers
x = Res_id_block(x, (128, 512)) # 3 layers
x = Res_id_block(x, (128, 512)) # 3 layers
block_3_out = x

# khoi 4 -> C4 / P4 (Stride 16)
x = Conv_block(x, (256, 1024), 2) # 3 layers
x = Res_id_block(x, (256, 1024)) # 3 layers

```

```

x = Res_id_block(x, (256, 1024)) # 3 layers
x = Res_id_block(x, (256, 1024)) # 3 layers
x = Res_id_block(x, (256, 1024)) # 3 layers
x = Res_id_block(x, (256, 1024)) # 3 layers
block_4_out = x
# khoi 5 (P5) da bi xoa trong ResNet-40
model = tf.keras.models.Model(
    inputs=input_dim,
    outputs=[block_3_out, block_4_out],
    name='resnet_40'
)
return model

def head_of_model(output_filters, bias_init):
    """
    xay dung dau ra cho du doan class va bbox
    """
    head = tf.keras.Sequential([tf.keras.layers.Input(shape=[None, None, 256])])
    kernel_init = tf.initializers.RandomNormal(0.0, 0.01)
    for _ in range(4):
        head.add(tf.keras.layers.Conv2D(256,3,padding='same',
kernel_initializer=kernel_init))
        head.add(tf.keras.layers.ReLU())
        head.add(tf.keras.layers.Conv2D(output_filters,3,1,padding='same',
kernel_initializer=kernel_init, bias_initializer=bias_init))
    return head

Phần mạng FPN:
import keras.layers
import tensorflow as tf

```

```

from resnet_40 import resnet_40_backbone

class FeaturePyramid(keras.layers.Layer):
    """
    Xay dung dac trung kim tu thap tu cac ban do dac trung cua backbone.
    Su dung ResNet-40 (bo P5), chi xuất ra P3 va P4.
    """

    def __init__(self, backbone=None, **kwargs):
        super().__init__(name='FeaturePyramid', **kwargs)
        self.backbone = backbone if backbone else resnet_40_backbone()

        self.conv_c3_1x1 = keras.layers.Conv2D(256, 1, 1, padding='same',
name='conv_c3_1x1')

        self.conv_c4_1x1 = keras.layers.Conv2D(256, 1, 1, padding='same',
name='conv_c4_1x1')

        self.conv_c3_3x3 = keras.layers.Conv2D(256, 3, 1, padding='same',
name='conv_c3_3x3')

        self.conv_c4_3x3 = keras.layers.Conv2D(256, 3, 1, padding='same',
name='conv_c4_3x3')

        self.upsample_2x = keras.layers.UpSampling2D(2)

    def call(self, images, training=False):
        c3_output, c4_output = self.backbone(images, training=training)
        p3_output = self.conv_c3_1x1(c3_output)
        p4_output = self.conv_c4_1x1(c4_output)
        # Top-down pathway: p3 nhan them thong tin tu p4
        p3_output = p3_output + self.upsample_2x(p4_output)
        p3_output = self.conv_c3_3x3(p3_output)
        p4_output = self.conv_c4_3x3(p4_output)
        return p3_output, p4_output

```

Phần Anchor Free:

```
import tensorflow as tf
```

```
class Anchor_box:
```

```
    """
```

```
    Anchor-free point generator for feature map locations.
```

```
    Generates [cx, cy, stride, stride] for each location.
```

```
    """
```

```
    def __init__(self):
```

```
        self._num_anchors = 1
```

```
        self._strides = [2 ** i for i in range(3, 5)]
```

```
    def _get_anchors(self, feature_h, feature_w, level):
```

```
        ry = tf.range(feature_w, dtype=tf.float32) + 0.5
```

```
        rx = tf.range(feature_h, dtype=tf.float32) + 0.5
```

```
        X, Y = tf.meshgrid(ry, rx)
```

```
        centers = tf.stack([X, Y], axis=-1) * self._strides[level - 3]
```

```
        centers = tf.reshape(centers, [-1, 2])
```

```
        stride_tensor = tf.fill([tf.shape(centers)[0], 2], tf.cast(self._strides[level-3],  
tf.float32))
```

```
        anchors = tf.concat([centers, stride_tensor], axis=-1)
```

```
        return anchors
```

```
    def get_anchors(self, img_h, img_w):
```

```
        anchors = [
```

```
            self._get_anchors(
```

```
                tf.math.ceil(img_h/2 ** i),
```

```
                tf.math.ceil(img_w/2 ** i),
```

```
                i,
```

```
            )
```

```

        for i in range(3, 5)
    ]

    return tf.concat(anchors, axis=0)

```

Phần IoU:

```
import tensorflow as tf
```

```
def convert_to_corners(boxes):
```

```
    """
```

```
    chuyen bbox tu dang [x,y,w,h] sang [xmin,ymin,xmax,ymax]
```

```
    sau do noi cac gia tri nay thanh dang tensor
```

```
    """
```

```
    x_center,y_center,w,h = tf.split(boxes,4,axis=-1)
```

```
    """
```

```
    tach tung cot cua 1 mang thanh tensor 1 cot
```

```
    vd:
```

```
    [50,50,100,100]
```

```
    [100,100,200,200]
```

```
    -> sau khi bien doi se cho ra dang
```

```
    tf.tensor([
```

```
    [50]
```

```
    [100]],shape(2,1)
```

```
    """
```

```
    #print(x_center,y_center,w,h)
```

```
    x_min = x_center - (w/2.0)
```

```
    y_min = y_center - (h/2.0)
```

```
    x_max = x_center + (w/2.0)
```

```
    y_max = y_center + (h/2.0)
```

```
    return tf.concat([x_min,y_min,x_max,y_max],axis=-1)
```



```

def compute_iou(truth_box, predict_box):
    """
    tb = truth_box
    pb = predict box
    """
    #chuyen toa do
    box_tb = convert_to_corners(truth_box)
    box_pb = convert_to_corners(predict_box)
    #tach tao do lu,rd
    lu_box_tb = box_tb[:, :2] #lay tat ca hang, sau do lay 2 cot dau tien
    rd_box_tb = box_tb[:, -2:] #lay tat ca hang, sau do lay 2 cot cuoi cung
    lu_box_pb = box_pb[:, :2]
    rd_box_pb = box_pb[:, -2:]
    lu_box_tb = tf.expand_dims(lu_box_tb, 0)
    rd_box_tb = tf.expand_dims(rd_box_tb, 0)
    lu_box_pb = tf.expand_dims(lu_box_pb, 1)
    rd_box_pb = tf.expand_dims(rd_box_pb, 1)
    #tinh toa do phan giao nhau
    Intersection_lu = tf.math.maximum(lu_box_tb, lu_box_pb)
    Intersection_rd = tf.math.minimum(rd_box_tb, rd_box_pb)
    #wh cua phan giao
    Intersection_wh = tf.math.maximum(0.0, Intersection_rd - Intersection_lu) #neu
    khong giao thi = 0
    #print(Intersection_wh)
    #dien tich phan giao
    area_Intersection = Intersection_wh[:, :, 0] * Intersection_wh[:, :, 1]
    #dien tich phan gom (Union)

```

```

    area_box_tb = (rd_box_tb[:, :, 0] - lu_box_tb[:, :, 0]) * (rd_box_tb[:, :, 1] -
lu_box_tb[:, :, 1])
    area_box_pb = (rd_box_pb[:, :, 0] - lu_box_pb[:, :, 0]) * (rd_box_pb[:, :, 1] -
lu_box_pb[:, :, 1])
    area_Union = tf.math.maximum(area_box_tb+area_box_pb-area_Intersection,1e-8)
    #tinh Iou
    IoU = tf.math.divide_no_nan(area_Intersection,area_Union)
    return IoU

```

Phần LabelEncoder:

```
import tensorflow as tf
```

```
class LabelEncoder:
```

```
    def __init__(self):
```

```
        self._box_variance = tf.convert_to_tensor(
            [0.1, 0.1, 0.2, 0.2], dtype=tf.float32
        )
```

```
    def _compute_box_target(self, points, matched_gt_boxes):
```

```
        # points: [cx, cy, stride, stride]
        # matched_gt_boxes: [gt_cx, gt_cy, gt_w, gt_h]
        box_target_xy = (matched_gt_boxes[:, :2] - points[:, :2]) / points[:, 2:]
        safe_gt_wh = tf.math.maximum(matched_gt_boxes[:, 2:], 1e-7)
        box_target_wh = tf.math.log(safe_gt_wh / points[:, 2:])
        box_target = tf.concat([box_target_xy, box_target_wh], axis=-1)
        return box_target / self._box_variance

```

```
    def _encode_sample(self, gt_boxes, gt_classes, points):
```

```
        if tf.shape(gt_boxes)[0] == 0:
            return tf.zeros_like(points), tf.fill([tf.shape(points)[0]], -1.0)
        gt_centers = gt_boxes[:, :2]
        point_centers = points[:, :2]

```

```

pc_x = tf.expand_dims(point_centers[:, 0], 1)
pc_y = tf.expand_dims(point_centers[:, 1], 1)
gc_x = tf.expand_dims(gt_boxes[:, 0], 0)
gc_y = tf.expand_dims(gt_boxes[:, 1], 0)
gw = tf.expand_dims(gt_boxes[:, 2], 0)
gh = tf.expand_dims(gt_boxes[:, 3], 0)
inside_x = tf.logical_and(pc_x >= gc_x - gw/2.0, pc_x <= gc_x + gw/2.0)
inside_y = tf.logical_and(pc_y >= gc_y - gh/2.0, pc_y <= gc_y + gh/2.0)
is_inside = tf.logical_and(inside_x, inside_y)
diff = tf.expand_dims(point_centers, 1) - tf.expand_dims(gt_centers, 0)
distances = tf.reduce_sum(tf.square(diff), axis=-1)
closest_point_idx = tf.argmin(distances, axis=0)
is_closest = tf.equal(
    tf.expand_dims(tf.range(tf.shape(points)[0], dtype=tf.int64), 1),
    tf.expand_dims(closest_point_idx, 0)
)
is_assigned = tf.logical_or(is_inside, is_closest)
gt_area = gw * gh
area_matrix = tf.where(is_assigned, gt_area, tf.fill(tf.shape(is_assigned),
float('inf')))
min_area_idx = tf.argmin(area_matrix, axis=1)
min_area = tf.reduce_min(area_matrix, axis=1)
positive_mask = tf.less(min_area, float('inf'))
matched_gt_boxes = tf.gather(gt_boxes, min_area_idx)
matched_gt_classes = tf.gather(gt_classes, min_area_idx)
target_classes = tf.cast(tf.fill([tf.shape(points)[0]], -1.0), tf.float32)
target_classes = tf.where(positive_mask, tf.cast(matched_gt_classes, tf.float32),
target_classes)

```

```

        target_boxes = self._compute_box_target(points, matched_gt_boxes)
        return target_boxes, target_classes

    def encode_batch(self, batch_images, gt_boxes, gt_classes, anchor_boxes):
        target_boxes, target_classes = tf.map_fn(
            fn=lambda x: self._encode_sample(x[0], x[1], anchor_boxes),
            elems=(gt_boxes, gt_classes),
            fn_output_signature=(tf.float32, tf.float32)
        )
        return batch_images, target_boxes, target_classes

```

Phần mô hình RetinaNet:

```

import tensorflow as tf

from FPN import FeaturePyramid
from resnet_40 import head_of_model

import numpy as np
import keras.layers

class RetinaNet(keras.Model):
    """A subclassed Keras model implementing the RetinaNet architecture.

    Attributes:
        num_classes: Number of classes in the dataset.
        backbone: The backbone to build the feature pyramid from.
        Currently supports ResNet50 only.
    """
    def __init__(self, num_classes, backbone=None, num_anchors=1, **kwargs):
        super().__init__(name="RetinaNet", **kwargs)
        self.fpn = FeaturePyramid(backbone)
        self.num_classes = num_classes
        self.num_anchors = num_anchors

```

```

prior_probability = tf.constant_initializer(-np.log((1 - 0.01) / 0.01))

self.cls_head = head_of_model(self.num_anchors * num_classes,
prior_probability) #phan du doan nhan

self.box_head = head_of_model(self.num_anchors * 4, "zeros") # phan du doan
box

def call(self, image, training=False):
    features = self.fpn(image, training=training)
    N = tf.shape(image)[0]
    cls_outputs = []
    box_outputs = []
    for feature in features:
        box_outputs.append(tf.reshape(self.box_head(feature), [N, -1, 4]))
        cls_outputs.append(
            tf.reshape(self.cls_head(feature), [N, -1, self.num_classes])
        )
    cls_outputs = tf.concat(cls_outputs, axis=1)
    box_outputs = tf.concat(box_outputs, axis=1)
    return tf.concat([box_outputs, cls_outputs], axis=-1)

#boxloss retina net

class RetinaNetBoxLoss(tf.losses.Loss):
    """Implements DIOU loss"""
    def __init__(self, anchors):
        super().__init__(
            reduction="none", name="RetinaNetBoxLoss"
        )
        self.anchors = anchors
        self._box_variance = tf.convert_to_tensor(
            [0.1, 0.1, 0.2, 0.2], dtype=tf.float32

```

```

)

def _decode_box_predictions(self, encoded_boxes):
    # encoded_boxes: [batch, num_anchors, 4]
    # self.anchors: [num_anchors, 4]
    box_variance = tf.cast(self._box_variance, encoded_boxes.dtype)
    encoded_boxes = encoded_boxes * box_variance
    anchors = tf.cast(self.anchors, encoded_boxes.dtype)
    anchors = tf.expand_dims(anchors, axis=0) # [1, num_anchors, 4]
    anchor_cx_cy = anchors[..., :2]
    anchor_stride = anchors[..., 2:]
    # Decode centers
    decoded_cx_cy = encoded_boxes[..., :2] * anchor_stride + anchor_cx_cy
    # Decode width and height
    decoded_w_h = tf.math.exp(encoded_boxes[..., 2:]) * anchor_stride
    # Convert back to x1, y1, x2, y2 format for DIoU calculation
    decoded_x1_y1 = decoded_cx_cy - decoded_w_h / 2
    decoded_x2_y2 = decoded_cx_cy + decoded_w_h / 2
    return tf.concat([decoded_x1_y1, decoded_x2_y2], axis=-1)

def call(self, y_true, y_pred):
    y_true_decoded = self._decode_box_predictions(y_true)
    y_pred_decoded = self._decode_box_predictions(y_pred)
    b1_x1, b1_y1, b1_x2, b1_y2 = tf.split(y_true_decoded, 4, axis=-1)
    b2_x1, b2_y1, b2_x2, b2_y2 = tf.split(y_pred_decoded, 4, axis=-1)
    # Intersection
    inter_x1 = tf.maximum(b1_x1, b2_x1)
    inter_y1 = tf.maximum(b1_y1, b2_y1)
    inter_x2 = tf.minimum(b1_x2, b2_x2)

```

```

inter_y2 = tf.minimum(b1_y2, b2_y2)
inter_w = tf.maximum(0.0, inter_x2 - inter_x1)
inter_h = tf.maximum(0.0, inter_y2 - inter_y1)
inter_area = inter_w * inter_h

# Union
b1_w = tf.maximum(0.0, b1_x2 - b1_x1)
b1_h = tf.maximum(0.0, b1_y2 - b1_y1)
b2_w = tf.maximum(0.0, b2_x2 - b2_x1)
b2_h = tf.maximum(0.0, b2_y2 - b2_y1)
b1_area = b1_w * b1_h
b2_area = b2_w * b2_h
union_area = tf.maximum(b1_area + b2_area - inter_area, 1e-7)
iou = inter_area / union_area

# Enclosing box
enclose_x1 = tf.minimum(b1_x1, b2_x1)
enclose_y1 = tf.minimum(b1_y1, b2_y1)
enclose_x2 = tf.maximum(b1_x2, b2_x2)
enclose_y2 = tf.maximum(b1_y2, b2_y2)
enclose_w = tf.maximum(0.0, enclose_x2 - enclose_x1)
enclose_h = tf.maximum(0.0, enclose_y2 - enclose_y1)

# Diagonal length squared of enclosing box
c_squared = tf.square(enclose_w) + tf.square(enclose_h) + 1e-7

# Distance squared between centers
b1_cx = b1_x1 + b1_w / 2
b1_cy = b1_y1 + b1_h / 2
b2_cx = b2_x1 + b2_w / 2
b2_cy = b2_y1 + b2_h / 2

```

```

rho_squared = tf.square(b2_cx - b1_cx) + tf.square(b2_cy - b1_cy)
diou = iou - (rho_squared / c_squared)
loss = 1.0 - diou
return tf.squeeze(loss, axis=-1)

class RetinaNetClassificationLoss(tf.losses.Loss):
    """Implements Focal loss"""
    def __init__(self, alpha, gamma):
        super().__init__(
            reduction="none", name="RetinaNetClassificationLoss"
        )
        self._alpha = alpha
        self._gamma = gamma
    def call(self, y_true, y_pred):
        cross_entropy = tf.nn.sigmoid_cross_entropy_with_logits(
            labels=y_true, logits=y_pred
        )
        probs = tf.nn.sigmoid(y_pred)
        alpha = tf.where(tf.equal(y_true, 1.0), self._alpha, (1.0 - self._alpha))
        pt = tf.where(tf.equal(y_true, 1.0), probs, 1 - probs)
        loss = alpha * tf.pow(1.0 - pt, self._gamma) * cross_entropy
        return tf.reduce_sum(loss, axis=-1)

class RetinaNetLoss(tf.losses.Loss):
    """Wrapper to combine both the losses"""
    #base alpha 0.25, gamma 2.0
    """
    gamma la tham so tap trung, neu mo hinh doan nham vat the khac la nguoi thi tang
    chi so nay len

```


alpha là tham số cân bằng, nếu mô hình bắt sót quá nhiều (recall thấp) thì tăng chỉ số này lên để ưu tiên người hơn

```
"""
```

```
def __init__(self, anchors, num_classes=80, alpha=0.25, gamma=2.0):
    super().__init__(reduction="sum_over_batch_size", name="RetinaNetLoss")
    self._clf_loss = RetinaNetClassificationLoss(alpha, gamma)
    self._box_loss = RetinaNetBoxLoss(anchors)
    self._num_classes = num_classes

def call(self, y_true, y_pred):
    y_pred = tf.cast(y_pred, dtype=tf.float32)
    box_labels = y_true[:, :, :4]
    box_predictions = y_pred[:, :, :4]
    cls_labels = tf.one_hot(
        tf.cast(y_true[:, :, 4], dtype=tf.int32),
        depth=self._num_classes,
        dtype=tf.float32,
    )
    cls_predictions = y_pred[:, :, 4:]
    positive_mask = tf.cast(tf.greater(y_true[:, :, 4], -1.0), dtype=tf.float32)
    ignore_mask = tf.cast(tf.equal(y_true[:, :, 4], -2.0), dtype=tf.float32)
    clf_loss = self._clf_loss(cls_labels, cls_predictions)
    box_loss = self._box_loss(box_labels, box_predictions)
    # Cắt bỏ những vùng không cần thiết (Nền và Ignore)
    clf_loss = tf.where(tf.equal(ignore_mask, 1.0), 0.0, clf_loss)
    box_loss = tf.where(tf.equal(positive_mask, 1.0), box_loss, 0.0)
    # THÊM NORMALIZER TẠI ĐÂY: Đếm số lượng Positive Anchor trên mỗi ảnh
    normalizer = tf.reduce_sum(positive_mask, axis=-1)
```

```

normalizer = tf.maximum(1.0, normalizer) # Ép tối thiểu là 1 để tránh lỗi chia cho
0
# Chia tổng Loss cho Normalizer để ổn định Gradient
clf_loss = tf.reduce_sum(clf_loss, axis=-1) / normalizer
box_loss = tf.reduce_sum(box_loss, axis=-1) / normalizer
return clf_loss + box_loss

```

Phần train mô hình:

```

import os
os.environ['TF_DATA_AUTOTUNE_RAM_BUDGET'] = '500000000'
import tensorflow as tf
from tensorflow.keras import mixed_precision
from Anchor_box import Anchor_box
from Label_encode import LabelEncoder
from Xu_ly_du_lieu import preprocessing_data_before_training
from RetinaNet import RetinaNet, RetinaNetLoss
from resnet_40 import resnet_40_backbone
import pandas as pd
import ast
def pack_targets(img, bdb, cls):
    # Lấy target_boxes (4 cột) và target_classes (1 cột) từ encoder
    # Mã hóa nhãn (Bounding box và Class) bằng bộ Anchor Box (đã được tối ưu nhỏ
    lại)
    _, target_boxes, target_classes = label_encoder.encode_batch(img, bdb, cls,
all_anchor)
    # Ép kiểu target_classes về float32 để đồng nhất với target_boxes
    target_classes = tf.cast(target_classes, tf.float32)
    # Mở rộng chiều của target_classes để nó có dạng (Batch, Num_Anchor, 1)
    target_classes = tf.expand_dims(target_classes, axis=-1)

```

```

# Hàn 2 cái lại thành 1 cục y_true duy nhất có 5 cột (Batch, Num_Anchor, 5)
y_true = tf.concat([target_boxes, target_classes], axis=-1)
# Trả về ĐÚNG 2 món: Ảnh (Input) và Nhãn gộp (Target) cho Keras
return img, y_true

if __name__ == '__main__':
    gpus = tf.config.experimental.list_physical_devices('GPU')
    if gpus:
        try:
            for gpu in gpus:
                tf.config.experimental.set_memory_growth(gpu, True)
        except RuntimeError as e:
            print(e)
    policy = mixed_precision.Policy('mixed_float16')
    mixed_precision.set_global_policy(policy)
    base_dir =
'/home/huy/Documents/RetinaNet_FreeAnchor_DIoU_resnet40/Quadcopter
Thermal.v4i.voc'

    target_size = 512 # [TILING - ĐỔI] 640 → 512: tile 640×512 resize về 512×512
    batch_size = 8 # [TILING - ĐỔI] 4 → 8: tile nhỏ hơn → tăng batch_size được
    # ---KHOI TAO ANCHOR BOX ---#
    anchor_gene = Anchor_box()
    # tao anchor
    all_anchor = anchor_gene.get_anchors(img_h=target_size, img_w=target_size)
    # khoi tao label encoder
    label_encoder = LabelEncoder()
    # --- XU LY DATASET VA GAN NHAN ---#
    # Đọc đường dẫn dữ liệu CSV (Dữ liệu đã được tiling/cắt nhỏ)

```

```

train_csv_path =
'/home/huy/Documents/RetinaNet_FreeAnchor_DIoU_resnet40/Quadcopter
Thermal.v4i.voc/csv_file/train_data_6_7_2026.csv'

val_csv_path =
'/home/huy/Documents/RetinaNet_FreeAnchor_DIoU_resnet40/Quadcopter
Thermal.v4i.voc/csv_file/valid_data_6_7_2026.csv'

raw_train_data =
preprocessing_data_before_training.change_string2number(train_csv_path)

raw_val_data =
preprocessing_data_before_training.change_string2number(val_csv_path)

train_dataset =
preprocessing_data_before_training.create_dataset_from_dataframe(raw_train_data).s
huffle(
    buffer_size=1000)

val_dataset =
preprocessing_data_before_training.create_dataset_from_dataframe(raw_val_data)

# Đọc ảnh, resize và thêm Data Augmentation chuyên cho ảnh nhiệt (Thermal)

train_dataset=train_dataset.map(preprocessing_data_before_training.read_img_and_la
bel,num_parallel_calls=tf.data.AUTOTUNE)

train_dataset = train_dataset.map(
    lambdaimg,bdb,cls:
preprocessing_data_before_training.resize_and_pad_img(img, bdb, cls, target_size),
    num_parallel_calls=tf.data.AUTOTUNE
)

# [TỐI ƯU QUADCOPTER] Áp dụng Data Augmentation (Lật ngang, độ sáng,
tương phản)

# Giúp mô hình bắt được người ở mọi hướng và trong điều kiện nhiệt độ môi trường
khác nhau

```

```

train_dataset = train_dataset.map(
    preprocessing_data_before_training.data_augment,
    num_parallel_calls=tf.data.AUTOTUNE
)
val_dataset=
val_dataset.map(preprocessing_data_before_training.read_img_and_label,num_parallel_calls=tf.data.AUTOTUNE)

val_dataset=val_dataset.map(lambda img,bdb,cls:
preprocessing_data_before_training.resize_and_pad_img(img, bdb, cls, target_size),
    num_parallel_calls=tf.data.AUTOTUNE)
train_dataset = train_dataset.padded_batch(
    batch_size=batch_size,
    padding_values=(0.0, 1e-8, -2),
    drop_remainder=True # cat bo nhung anh le khong cung 1 batch
)
train_dataset = train_dataset.map(
    pack_targets,
    num_parallel_calls=tf.data.AUTOTUNE
)
val_dataset = val_dataset.padded_batch(
    batch_size=batch_size,
    padding_values=(0.0, 1e-8, -2),
    drop_remainder=True # cat bo nhung anh le khong cung 1 batch
)
val_dataset = val_dataset.map(
    pack_targets,
    num_parallel_calls=tf.data.AUTOTUNE
)

```

```

# [FIX RAM]: Chốt cứng buffer_size=2 thay vì để AUTOTUNE bòn rút RAM
train_dataset = train_dataset.prefetch(buffer_size=2)
val_dataset = val_dataset.prefetch(buffer_size=2)

# -----#

#   Khoi tao mo hinh   #
train_df = pd.read_csv(train_csv_path)
all_labels = []

for cid in train_df['class_id'].apply(ast.literal_eval):
    all_labels.extend(cid)

num_class = len(set(all_labels))
num_classes = num_class

resnet40_backbone = resnet_40_backbone()

model = RetinaNet(num_classes=num_classes, backbone=resnet40_backbone)

# [TỐI ƯU LOSS & GRADIENT] Bổ sung clipnorm=10.0 để tránh bùng nổ Gradient
khi dùng Loss CIoU/DIoU

# Learning rate đặt ở mức nhỏ (1e-4) do bài toán fine-tuning trên vật thể nhỏ thường
rất nhạy cảm

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4, clipnorm=10.0)
loss_fn = RetinaNetLoss(anchors=all_anchor, num_classes=num_classes)

# 3. Compile model

model.compile(optimizer=optimizer, loss=loss_fn)

# 4. Thiết lập Callbacks để lưu trọng số và tự dừng

weight_folder = os.path.join(base_dir, 'weight_store')
os.makedirs(weight_folder, exist_ok=True)

callbacks_list = [

    # Tự động ngắt hệ thống nếu Loss văng ra NaN (Bảo vệ tiến trình train)
    tf.keras.callbacks.TerminateOnNaN(),

```

```

# Giảm Learning Rate khi mô hình có dấu hiệu chững lại (Plateau)
# Giúp mô hình hội tụ tốt hơn ở các epoch cuối
tf.keras.callbacks.ReduceLROnPlateau(
    monitor="loss",
    factor=0.5,
    patience=3,
    min_lr=1e-7,
    verbose=1
),
tf.keras.callbacks.ModelCheckpoint(
    filepath=weight_folder + "/RetinaNet_anchorfree_DIoU_resnet40.weights.h5",
    # [TILING - ĐỔI] không ghi đè model cũ
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=True,
    verbose=1,
),
tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    verbose=1,
    restore_best_weights=True
)
]
EPOCHS = 200
hist = model.fit(
    train_dataset,

```

```

validation_data=val_dataset,
epochs=EPOCHS,
callbacks=callbacks_list,
)
hist_df = pd.DataFrame(hist.history)
model_store_folder = os.path.join(base_dir, 'model_store')
hist_store_folder = os.path.join(base_dir, 'hist_store')
os.makedirs(hist_store_folder, exist_ok=True)
hist_df.to_csv(hist_store_folder + '/RetinaNet_anchorfree_DIoU_resnet40.csv')

```

Phần dự đoán kết quả với ảnh:

```

"""

```

this file use for datasets that dont have ground truth.

file du doan nay dung cho du doan loai du lieu co nhan.

```

"""

```

```

import sys

```

```

import os

```

```

# Thêm thư mục gốc của dự án vào sys.path để Python tìm được các module:

```

```

# Anchor_box, RetinaNet, resnet_40, Xu_ly_du_lieu

```

```

# (Cần thiết vì file này nằm trong thư mục con 'Quadcopter Thermal.v4i.voc/')

```

```

sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))

```

```

import tensorflow as tf

```

```

import pandas as pd

```

```

import numpy as np

```

```

import matplotlib.pyplot as plt

```

```

import matplotlib.patches as patches

```

```

from Anchor_box import Anchor_box

```

```

from RetinaNet import RetinaNet

```



```

from resnet_40 import resnet_40_backbone

import ast

# [TILING - THÊM MỚI] import các hàm tiling từ tiling_utils
from Xu_ly_du_lieu.tiling_utils import generate_tile_coords, apply_global_nms
from tensorflow.keras import mixed_precision

# --- CAU HINH ---

TARGET_SIZE = 512 # kích thước mỗi tile sau khi resize (không đổi)
BATCH_SIZE = 1

# [TILING - ĐỔI] trở sang weight của model mới train với tiled data
WEIGHT_PATH = '/home/huy/Documents/RetinaNet_FreeAnchor_DIoU/Quadcopter
Thermal.v4i.voc/weight_store/RetinaNet_anchorfree_DIoU.weights.h5'
TEST_CSV = '/home/huy/Documents/RetinaNet_FreeAnchor_DIoU/Quadcopter
Thermal.v4i.voc/csv_file/test_data_6_7_2026.csv'

gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError as e:
        print(e)

policy = mixed_precision.Policy('mixed_float16')
mixed_precision.set_global_policy(policy)

def decode_box_predictions(anchor_boxes, box_predictions):
    """
    Giai ma toa do tu delta sang [ymin, xmin, ymax, xmax] cho NMS
    """

    # Tach toa do Anchor (x_center, y_center, w, h)
    anchor_boxes = tf.cast(anchor_boxes, box_predictions.dtype)

```

```

a_x, a_y, a_w, a_h = tf.split(anchor_boxes, 4, axis=-1)
# Tach Delta du doan
t_x, t_y, t_w, t_h = tf.split(box_predictions, 4, axis=-1)
# [BAN VA 1]: Nhan nguoc lai voi variances da nen luc train
# Phai trung khop voi file LabelEncoder luc train nhe bro
t_x = t_x * 0.1
t_y = t_y * 0.1
t_w = t_w * 0.2
t_h = t_h * 0.2
# Giai ma Center X, Y
x = t_x * a_w + a_x
y = t_y * a_h + a_y
# Giai ma Width, Height
w = tf.exp(t_w) * a_w
h = tf.exp(t_h) * a_h
# Chuyen ve dang goc [xmin, ymin, xmax, ymax]
x1, y1 = x - w / 2.0, y - h / 2.0
x2, y2 = x + w / 2.0, y + h / 2.0
# Tra ve y-first cho ham combined_non_max_suppression cua TF
return tf.concat([y1, x1, y2, x2], axis=-1)
def get_inference_model(num_classes):
    backbone = resnet_40_backbone()
    model = RetinaNet(num_classes=num_classes, backbone=backbone)
    # Khoi tao shape cho model truoc khi load weight
    model(tf.zeros((1, TARGET_SIZE, TARGET_SIZE, 3)))
    model.load_weights(WEIGHT_PATH)
    return model

```

[TILING - HÀM MỚI]

```
def run_tiled_inference(model, image_path, all_anchors,
                        tile_w=640, tile_h=512, overlap=0.20,
                        score_threshold=0.7):
```

"""

Chạy inference trên ảnh gốc bằng cách chia thành các tile, inference từng tile, gộp lại và áp dụng Global NMS để loại bỏ box trùng ở vùng overlap. Trả về boxes trong hệ tọa độ ảnh gốc (không cần scale lại bằng ratio). ratio trả về = 1.0 để tương thích với code vẽ GT box trong __main__.

Args:

model : RetinaNet model đã load weights
 image_path : đường dẫn ảnh gốc (1280×1024)
 all_anchors : anchor boxes cho TARGET_SIZE×TARGET_SIZE
 tile_w, tile_h: kích thước mỗi tile (pixel trong ảnh gốc)
 overlap : tỉ lệ chồng lấp giữa các tile (0.0 – 1.0)
 score_threshold: lọc box có confidence thấp hơn ngưỡng này

Returns:

(orig_img_uint8, final_boxes, final_scores, dummy_classes, num_det, ratio=1.0)
 final_boxes shape: (N, 4) — [ymin, xmin, ymax, xmax] trong hệ ảnh gốc

"""

Bước 1: Load ảnh gốc

```
img_raw = tf.io.read_file(image_path)
img = tf.image.decode_jpeg(img_raw, channels=3)
img = tf.cast(img, tf.float32)
orig_shape = tf.shape(img)
orig_h = int(orig_shape[0])
```

```

orig_w = int(orig_shape[1])

# Bước 2: Tạo danh sách tọa độ tile
tile_coords = generate_tile_coords(orig_w, orig_h, tile_w, tile_h, overlap)
all_boxes = []
all_scores = []
for (tx, ty, tw, th) in tile_coords:
    # Crop tile từ ảnh gốc
    tile = img[ty:ty + th, tx:tx + tw]

    # Resize và pad tile về TARGET_SIZE × TARGET_SIZE (giống
    # resize_and_pad_img)
    tile_shape = tf.cast(tf.shape(tile)[:2], dtype=tf.float32)
    t_h, t_w = tile_shape[0], tile_shape[1]
    tile_ratio = tf.cast(TARGET_SIZE, tf.float32) / tf.math.maximum(t_h, t_w)
    new_h = tf.cast(t_h * tile_ratio, tf.int32)
    new_w = tf.cast(t_w * tile_ratio, tf.int32)
    tile_resized = tf.image.resize(tile, [new_h, new_w])
    tile_padded = tf.image.pad_to_bounding_box(tile_resized, 0, 0, TARGET_SIZE,
    TARGET_SIZE)

    tile_input = tf.expand_dims(tile_padded, axis=0)

    # Chạy model trên tile
    predictions = model.predict(tile_input, verbose=0)
    box_preds = predictions[:, :, :4]
    class_preds = predictions[:, :, 4:]

    decoded_boxes = decode_box_predictions(all_anchors, tf.cast(box_preds,
    tf.float32))

    class_probs = tf.nn.sigmoid(tf.cast(class_preds, tf.float32))

    # NMS nội tile (lọc thô, threshold thấp để không bỏ sót)

```

```

nms_boxes, nms_scores, _, valid_det =
tf.image.combined_non_max_suppression(
    tf.expand_dims(decoded_boxes, axis=2),
    class_probs,
    max_output_size_per_class=50,
    max_total_size=50,
    iou_threshold=0.3,
    score_threshold=score_threshold,
    clip_boxes=False
)
ratio_val = float(tile_ratio)
n_det = int(valid_det[0])
# Bước 3: Chuyển tọa độ boxes về hệ ảnh gốc
# model space (TARGET_SIZE) → tile gốc space (÷ ratio)
# tile gốc space → ảnh gốc space (+ tile offset)
for i in range(n_det):
    ymin, xmin, ymax, xmax = [float(v) for v in nms_boxes[0][i]]
    all_boxes.append([
        ymin / ratio_val + ty, # ymin trong ảnh gốc
        xmin / ratio_val + tx, # xmin trong ảnh gốc
        ymax / ratio_val + ty, # ymax trong ảnh gốc
        xmax / ratio_val + tx, # xmax trong ảnh gốc
    ])
    all_scores.append(float(nms_scores[0][i]))
# Bước 4: Global NMS — BẮT BUỘC để loại box trùng ở vùng overlap
final_boxes, final_scores, num_det = apply_global_nms(
    all_boxes, all_scores,

```

```

        iou_threshold=0.3,
        score_threshold=score_threshold
    )
    # Trả về ảnh gốc (uint8) và boxes trong hệ tọa độ ảnh gốc
    orig_img_uint8 = tf.cast(img, tf.uint8).numpy()
    dummy_classes = np.zeros(num_det, dtype=np.float32)
    # ratio = 1.0 vì boxes đã ở hệ ảnh gốc → code vẽ GT trong __main__ hoạt động đúng
    return orig_img_uint8, final_boxes, final_scores, dummy_classes, num_det, 1.0

# [TILING - KẾT THÚC HÀM MỚI]
=====

if __name__ == '__main__':
    test_df = pd.read_csv(TEST_CSV)
    all_labels = []
    for cid in test_df['class_id'].apply(ast.literal_eval):
        all_labels.extend(cid)
    num_class = len(set(all_labels))
    anchor_gene = Anchor_box()
    all_anchors = anchor_gene.get_anchors(img_h=TARGET_SIZE,
img_w=TARGET_SIZE)
    model = get_inference_model(num_class)
    for i in range(100):
        row = test_df.iloc[i]
        img_path = row['path_img']
        gt_boxes = ast.literal_eval(row['bbox'])
        # [TILING - ĐỔI] dùng run_tiled_inference thay cho run_inference
        # img giờ là ảnh gốc 1280×1024 (thay vì 512×512 padded)
        # boxes đã ở hệ tọa độ ảnh gốc, ratio = 1.0
        img, boxes, scores, classes, num_det, ratio = run_tiled_inference(

```

```

    model, img_path, all_anchors, score_threshold=0.7
)
print(f"--- Anh thu {i+1} ---")
print(f"So luong vat the tim thay: {num_det}")
if num_det > 0:
    # [TILING - ĐỒI] scores là numpy array, không gọi .numpy()
    print(f"Diem tin tuong (Scores): {scores[:num_det]}")
fig, ax = plt.subplots(1, figsize=(8, 6))
ax.imshow(img)
for j in range(num_det):
    # [TILING - GIỮ NGUYÊN] boxes vẫn là [ymin, xmin, ymax, xmax],
    # nhưng giờ trong hệ ảnh gốc 1280×1024 thay vì 512×512
    ymin, xmin, ymax, xmax = float(boxes[j][0]), float(boxes[j][1]),
float(boxes[j][2]), float(boxes[j][3])
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin, linewidth=2,
edgecolor='cyan',facecolor='none')
    ax.add_patch(rect)
    ax.text(xmin, ymin - 5, f"Pred: {scores[j]:.2f}", color='cyan', fontsize=10,
backgroundcolor='black')
# Ve Ground Truth (Mau Do)
# [TILING - GIỮ NGUYÊN] ratio = 1.0 nên x_c * ratio = x_c (tọa độ ảnh gốc)
for box in gt_boxes:
    x_c, y_c, w, h = box
    x_c_scaled, y_c_scaled = x_c * ratio, y_c * ratio
    w_scaled, h_scaled = w * ratio, h * ratio
    rect_gt = patches.Rectangle((x_c_scaled - w_scaled / 2.0, y_c_scaled - h_scaled
/ 2.0), w_scaled, h_scaled, linewidth=1, edgecolor='red', facecolor='none', linestyle='--
')

```

```
ax.add_patch(rect_gt)
plt.title(f"Test Image {i + 1} - Found {num_det} objects")
plt.axis('off')
plt.show()
```